
ECG Anomaly Detection and Arrhythmia Classification

AUTHOR

A.K.S.S. Wimalasena

SUPERVISOR

Ms. Sachini Karunarathne

Department of Mathematics

University of Ruhuna



University of Ruhuna

Department of Mathematics

Acknowledgements

I sincerely thank my supervisor, Ms. Sachini Karunaratne, for her guidance, constructive comments, and encouragement throughout this research. I also thank the staff of the Department of Mathematics, University of Ruhuna, for providing the academic environment needed to complete this work.

I am grateful to the researchers and organisations that maintain PhysioNet and the MIT-BIH databases. Their carefully annotated physiological data made the experimental part of this study possible. Finally, I thank my family and friends for their patience and support during the design, implementation, testing, and writing of this report.

Matara, September 2026
A.K.S.S. Wimalasena

Abstract

Long ECG recordings contain many beats, making manual review difficult. This study developed a single-lead electrocardiogram (ECG) system to detect abnormal beats and classify them into five supported groups. A binary Extra Trees model first separates normal and abnormal beats. A second Extra Trees model assigns an abnormal subtype. The six output codes are N, PVC, PAC, LBBB, RBBB, and AFib. These are study-specific annotation groups; in particular, AFib includes atrial flutter, and some groups include escape beats.

A convolutional autoencoder was trained on the first stored ECG channel of the MIT-BIH Normal Sinus Rhythm Database, using 15 subjects for fitting and three for validation. Its weights were then fixed. The supervised models used modified Lead II (MLII) data from 46 MIT-BIH Arrhythmia Database records, representing 45 subject clusters. Each beat was described by 223 waveform, spectral, and statistical features, 15 RR-interval features, and four autoencoder residual features. Supervised windows were filtered and standardised separately.

The final 20% of accepted beats in each record formed the temporal test. Earlier beats were used for model selection and fitting, with a 16-beat gap before each evaluation boundary. Because the test segment had been viewed during earlier development, this was a retrospective evaluation of known patients, with a risk of optimistic performance estimates.

Binary accuracy was 98.81% on 20,969 beats, and conditional subtype accuracy was 97.63% on 7,958 supported abnormal beats. Whole-system accuracy was 98.07% and macro F1 was 0.9524 on 20,043 supported beats. Mean accuracy across subjects was 98.03%, with a subject-clustered 95% confidence interval of 96.29–99.25%. However, PAC recall was only 70.63%. A limited external check used 5,416 beats from two INCART records belonging to one patient. Whole-system accuracy was 92.85%, while binary precision fell to 64.22%.

The system met the aggregate accuracy and interval-width targets in the retrospective test. It did not meet the recall target for every class. Independent patient evaluation and real-sensor testing are still needed.

Contents

List of Figures	viii
List of Tables	ix
1 Introduction	1
1.1 Background	1
1.2 Research Problem	4
1.3 Objectives and expected outcomes	4
1.4 Research questions	5
1.5 Organisation of the report	5
2 Literature Review	6
2.1 ECG datasets and evaluation protocols	6
2.2 Supervised heartbeat classification	7
2.3 Normal-reference reconstruction	7
2.4 Class imbalance and statistical validation	8
2.5 Research gap and contribution	8
3 Data and Preparation	11
3.1 Data Sources and Roles	11
3.2 Data Cleaning	14
3.2.1 Cleaning Checklist	14
3.2.2 Window Construction	15
3.2.3 Cleaning Yield	16
3.2.4 Preparation during prediction	16
3.3 Data Preparation	17
3.3.1 Prepared Objects	17
3.3.2 Splits	18
3.3.3 Exact Model Allocation	19
3.3.4 Feeding the Frozen System	20
4 Theoretical Foundations	21
4.1 Why ECG evidence has more than one form	21
4.2 From voltage to a comparable four-second window	22
4.3 Feature extraction theory	23
4.4 RR timing theory	25

4.5	Normal-only reconstruction theory	26
4.5.1	What the encoder and decoder actually compute	27
4.5.2	How the autoencoder lowers reconstruction loss	27
4.6	Hierarchical classification theory	29
4.7	Why Extra Trees fits this feature design	30
4.7.1	How the two supervised models are fitted	31
4.8	Class and patient imbalance theory	31
4.9	Temporal patient adaptation theory	32
4.9.1	Why training, validation, and testing have different jobs	32
4.10	From saved models to a live ECG decision	32
4.11	Validation theory	33
4.12	Summary of the theoretical framework	35
5	Methodology	36
5.1	System overview	36
5.2	Input signal and beat window	37
5.3	Three trained components	38
5.4	Normal-only autoencoder	39
5.4.1	Purpose	39
5.4.2	Network structure	41
5.4.3	Encoder block	42
5.4.4	Bottleneck and decoder mathematics	42
5.4.5	Normal-reference data and checkpoint	43
5.4.6	How the training update changes network weights	43
5.4.7	Four autoencoder residual features	44
5.5	Final 242-value feature vector	45
5.6	How an Extra Trees model learns	46
5.6.1	Why some training rows have more weight	47
5.6.2	The mathematical question at each tree node	47
5.6.3	From a tree leaf to an ensemble score	48
5.7	Binary abnormality detector	48
5.8	Abnormal-subtype classifier	49
5.9	Final decision logic	50
5.10	Mathematical training and inference procedure	51
5.10.1	Notation, targets, and separation of data	51
5.10.2	Model 1: learning and freezing the autoencoder	52
5.10.3	Core calculation shared by training and prediction	54
5.10.4	Model 2: fitting and selecting the binary detector	55
5.10.5	Model 3: fitting the abnormal-subtype classifier	57
5.10.6	Applying the saved models to an unseen beat	58
5.10.7	End-to-end case study using real ECG segments	59
5.11	Why this architecture was selected	73

6	Experimental Method	74
6.1	Method overview	74
6.2	Research objective and success conditions	75
6.3	Input data and feature construction	75
6.4	Normal-reference autoencoder training procedure	76
6.5	Temporal split	76
6.6	Model selection and final fitting	77
6.7	Evaluation order	79
6.8	Metrics	80
6.9	Subject-clustered confidence interval	81
6.10	Exploratory tests against the 90% target	82
6.11	Reproducibility and leakage controls	82
7	Results and Statistical Validation	83
7.1	Evaluation populations and sample sizes	83
7.2	Model selection	84
7.3	Main temporal results	85
7.4	Uncertainty and subject variation	86
7.5	Errors by stage and class	87
7.6	Limited external INCART check	89
7.7	Answers to the research questions	90
8	Discussion and Conclusion	91
8.1	Main findings	91
8.2	Interpretation of the findings	91
8.3	Achievement of the objectives	92
8.4	Effect of the preprocessing revision	92
8.5	Limitations	92
8.6	Further research	93
8.7	Conclusion	93
	Bibliography	94
A	Reproducibility Record	98
A.1	Experiment files	98
A.2	Frozen constants	98
A.3	Recorded evaluation targets	98
A.4	Safety statement	98
A.5	Verification and historical record	99

List of Figures

1.1	Main components of a simplified ECG beat. Shape features describe the P–QRS–T pattern, while RR features describe timing between beats. This is an educational drawing, not a diagnostic template.	1
1.2	Real four-second ECG examples from the MIT–BIH Arrhythmia Database: normal record 100, PVC record 208, PAC record 232, LBBB record 109, RBBB record 118, and AFib record 203. The dashed line marks the annotated central R peak. The examples are used for explanation and are not clinical diagnostic templates.	3
3.1	Dataset roles and evaluation boundaries. The healthy NSRDB channel creates the frozen normal model. MITDB MLII creates the supervised feature table and is split chronologically into development and temporal-test regions. . . .	12
3.2	MLII polarity and the channel used by each model.	13
3.3	Cleaning real ECG for the two roles. The healthy example is the first channel of NSRDB record 16265. The supervised-input example is MLII from MITDB record 100. Both become one 512-sample input, but their documented channel identities remain different.	17
3.4	Preparation of normal-reference windows and labelled MLII beats.	20
4.1	Two evidence scales used by the final model. Morphology describes the local shape of a beat; RR timing describes rhythm context around the beat.	21
4.2	Learning map for the final ECG system.	22
4.3	Feature-extraction operations behind the final 242-value vector. The tree models learn from compact numerical evidence, not from raw ECG samples directly.	24
4.4	Normal-only reconstruction principle. A larger residual can indicate novelty, but it is evidence only; the final arrhythmia decision is made by the supervised hierarchy.	26
4.5	Autoencoder and Extra Trees training mechanisms.	28
4.6	Probability logic of the final hierarchy. The subtype model is conditional: it is used only after the abnormality gate is passed.	29
4.7	Theoretical reason for subject-clustered validation. Clustered resampling prevents one long record from being treated as many independent people.	34
5.1	Exact runtime order of the final ECG system. The autoencoder supplies features; the supervised tree models make the final class decision.	36

5.2	R-peak-aligned beat window used by the final model. The R peak is at sample index 200, giving the model pre-QRS context and post-QRS recovery information.	38
5.3	Training and use of the saved three-component model.	39
5.4	Autoencoder architecture with verified tensor dimensions.	41
5.5	Autoencoder fitting and validation loss.	44
5.6	Real healthy and abnormal ECG examples through the frozen autoencoder. The residual between input and reconstruction is converted into four numerical features for the supervised detector.	45
5.7	Feature-vector layout used by both supervised models. The same 242-value vector is passed first to the abnormality detector and, if required, to the subtype classifier.	46
5.8	Worked example of entropy reduction in a tree.	48
5.9	Real fitting and temporal-test ECG segments.	61
5.10	Real ECG through one training update and the saved autoencoder.	63
5.11	Real test PVC from source ECG to subtype scores.	66
5.12	Real temporal-test examples of all six outputs.	71
6.1	Chronological model selection, refitting, and final testing.	74
6.2	Temporal split audit for every eligible MITDB MLII record. Blue beats are available for final fitting, yellow beats are the boundary gap before the test, and red beats are the held-out final 20% temporal test.	77
6.3	Model selection on the 64–80% validation region. The binary detector is selected by balanced accuracy and F1; the subtype classifier is selected by macro F1 and accuracy.	79
6.4	Populations used for the three evaluation questions.	80
7.1	Three evaluation questions and their test populations.	83
7.2	Reference-class composition of the corrected temporal test.	84
7.3	Pooled temporal scores after the preprocessing correction.	85
7.4	Subject-clustered confidence intervals.	86
7.5	Corrected whole-system accuracy across 45 known-patient clusters.	87
7.6	True and correctly classified supported abnormal beats. PAC remains below the 90% recall target.	88
7.7	Whole-system confusion matrix for 20,043 supported temporal beats.	89

List of Tables

1.1	ECG patterns underlying the six output groups considered in this study. . . .	2
2.1	Main papers referenced in this study and their contribution.	9
3.1	Data sources and their strictly separated roles.	11
3.2	Mapping from MITDB expert annotations to project outputs.	14
3.3	Cleaning checklist applied before a signal can become a model input.	15
3.4	Training, validation, and temporal-test data supplied to each model.	19
4.1	Technologies in the final evidence path and the mathematical task performed by each. The web interface displays results and does not train the models. . .	33
4.2	How the theory maps to the implemented final system.	35
5.1	Architecture, training target, and output of each fitted component.	38
5.2	Organised feature groups in the final architecture.	46
5.3	Illustrative allocation of a record containing 1,000 accepted beats.	52
5.4	Illustrative validation comparison of three binary-score thresholds.	56
5.5	Real ECG segments used in the training-to-prediction case study.	60
5.6	All 15 measured RR features for the real temporal-test PVC.	65
5.7	Recorded validation measurements selecting the real saved models.	68
5.8	Complete route of the real test PVC through the first saved binary tree. . . .	69
5.9	Complete route of the real test PVC through the first saved subtype tree. . . .	70
5.10	Measured predictions for the six real illustrative test segments.	72
5.11	Reason for each major final-architecture decision.	73
6.1	Exact evidence supplied to the final supervised models.	75
7.1	Distinct denominators for the three evaluation questions.	84
7.2	Corrected temporal results. Confidence intervals estimate equal-subject accuracy, not pooled beat accuracy.	85
7.3	Whole-system performance by supported abnormal output group.	87
7.4	Corrected model on complete INCART I01 and I02 recordings.	90
A.1	Settings used for the final experiment.	99

1 Introduction

1.1 Background

An electrocardiogram (ECG) records the small voltage changes that occur when electrical activity moves through the heart. Electrodes on the body record this activity from different directions. In an ECG trace, the horizontal axis shows time and the vertical axis shows voltage. ECG analysis is useful because the pattern changes when a heartbeat starts in a different place, happens at a different time, or travels through the heart in a different way.

Components of one ECG beat. A normal heartbeat starts when the sinoatrial node creates an impulse. The P wave shows atrial depolarisation. The short, sharp QRS complex mainly shows ventricular depolarisation. The T wave shows ventricular repolarisation. The time between two neighbouring R peaks is called the RR interval. It gives information about heart rate and rhythm regularity [1]. Figure 1.1 shows these parts in a simple way. In real recordings, the same beat can look inverted, stretched, or noisy because of the lead, the patient, the electrodes, and the recording conditions.

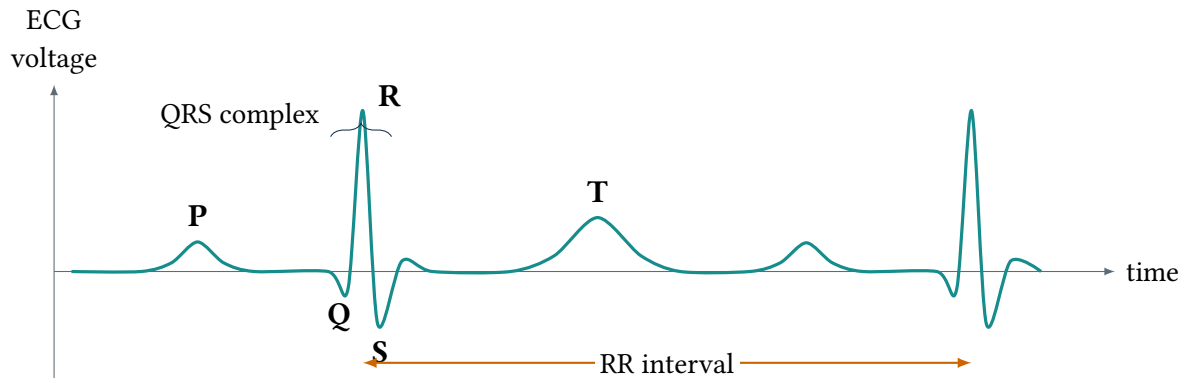


Figure 1.1: Main components of a simplified ECG beat. Shape features describe the P–QRS–T pattern, while RR features describe timing between beats. This is an educational drawing, not a diagnostic template.

Arrhythmia and the classes considered. Arrhythmias affect the timing or sequence of the heart’s electrical activity. Conduction disorders can also change the ECG waveform. This study uses one normal-like output group and five abnormal output groups. Table 1.1 summarises the main physiological cause and visible ECG behaviour for each one. These descriptions explain the underlying ECG patterns. The study’s annotation groups are broader than these clinical descriptions and are defined in Chapter 3. A clinical diagnosis requires the full ECG and patient context [2, 3].

Table 1.1: ECG patterns underlying the six output groups considered in this study.

Pattern	Main cause	Typical ECG features
Normal sinus rhythm	The impulse begins in the sinoatrial node and follows the normal conduction path.	Repeated P–QRS–T patterns and generally regular RR intervals. It supplies the reference against which unusual beats are compared.
Premature ventricular contraction (PVC)	A premature impulse begins in a ventricle instead of following the normal path from the atria.	The beat occurs early and often has a wide, unusual QRS, followed by a pause. Detection is important because it is a clear ventricular abnormality and can occur repeatedly.
Premature atrial contraction (PAC)	A premature impulse begins in an atrial location outside the sinoatrial node.	The beat occurs early; the P wave may change, while the QRS can remain narrow and similar to normal. Its subtle shape makes PAC difficult to separate from normal beats.
Left bundle branch block (LBBB)	Electrical conduction through the left bundle branch is delayed or blocked.	The ventricles activate in an altered sequence, producing a broad, changed QRS. Recognition is important because the pattern can be associated with underlying cardiac disease.
Right bundle branch block (RBBB)	Electrical conduction through the right bundle branch is delayed or blocked.	The later part of the QRS changes and can show a second peak in some leads. It must be separated from LBBB because the delayed side is different.
Atrial fibrillation (AFib)	Atrial electrical activity becomes rapid and disorganised rather than following one regular atrial impulse.	Consistent P waves disappear and RR intervals become irregular. Identification is important because AFib is linked with clinically important complications and requires rhythm-level evidence.

Figure 1.2 shows real labelled ECG segments from the MIT–BIH Arrhythmia Database instead of drawings. Each panel comes from the final temporal-test portion of its record. For each stated record and class, the displayed window is the one closest to that record-and-class group’s mean waveform by squared distance. This rule selects an example close to the group mean. The signals were resampled to 128 Hz, standardised, and aligned so the labelled R peak is at 1.5625 seconds.

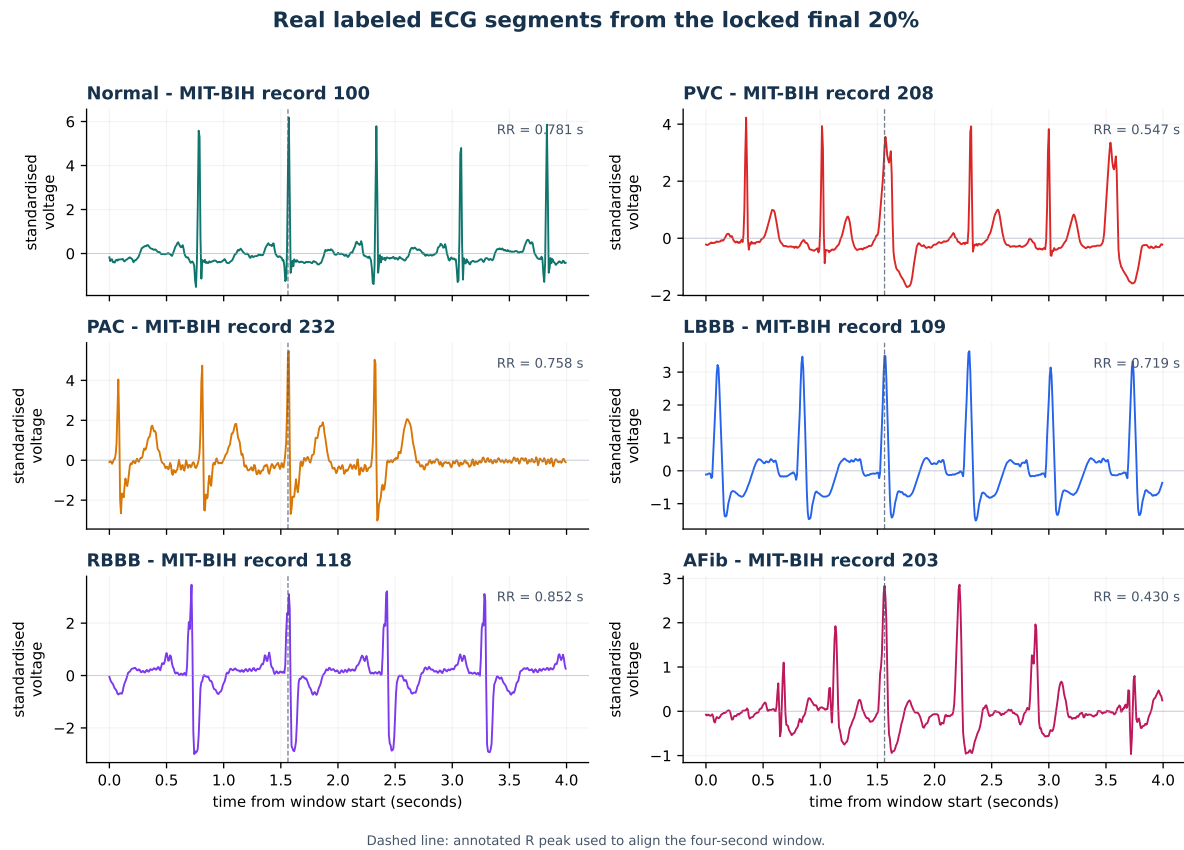


Figure 1.2: Real four-second ECG examples from the MIT-BIH Arrhythmia Database: normal record 100, PVC record 208, PAC record 232, LBBB record 109, RBBB record 118, and AFib record 203. The dashed line marks the annotated central R peak. The examples are used for explanation and are not clinical diagnostic templates.

These examples show why one simple threshold is not enough. PVC, LBBB, and RBBB mainly change beat shape. PAC can occur early while still looking close to a normal beat. AFib is often clearer from changing intervals and rhythm context. For this reason, the final system combines modified Lead II (MLII) waveform shape, frequency information, RR history, and disagreement with a learned normal reconstruction.

Finding these patterns matters because long ECG recordings can contain too many beats for continuous manual review. An automated research system can mark suspicious regions, count events, and help a reviewer focus attention. At the same time, an error can miss an abnormality or create an unnecessary alarm. Therefore, accuracy, per-class recall, confidence intervals, and clear limits are all needed. The system developed here is for research evaluation only. It is not a replacement for a clinician.

1.2 Research Problem

Main research problem: How accurately can a two-stage ECG system combine normal-reference reconstruction, beat shape, and heartbeat timing to detect abnormal beats and classify five supported abnormal groups in later recordings from known patients? Can it exceed 90% whole-system accuracy while meeting the study's confidence-interval and per-class recall targets?

The main problem contains four connected sub-problems:

1. ECG shape varies between patients and leads, so harmless personal morphology can resemble an abnormality.
2. A two-stage system can fail at the abnormality gate or at subtype classification; a missed abnormal beat never reaches the second stage.
3. Normal and arrhythmia classes have unequal numbers of beats and patients, so ordinary accuracy can hide failure on a rare class.
4. Thousands of neighbouring beats from one patient are correlated. Evaluation must separate model fitting from testing and estimate uncertainty using complete patient clusters. Any earlier exposure to the test data must also be reported.

The final experiment handles these problems with a chronological split. The first 64% of each MIT-BIH Arrhythmia Database (MITDB) record trains candidate models, the next 16% selects the models and threshold, and the final 20% is used for temporal testing. The present model fitting excludes test rows, but the segment was observed during earlier development. Percentages count accepted beats in chronological order, rather than exact recording duration. The result is therefore a retrospective patient-adapted temporal evaluation, with possible selection optimism and no confirmatory claim about completely new patients.

1.3 Objectives and expected outcomes

Main objective. The main objective is to develop and statistically evaluate an ECG system that first detects abnormal beats and then classifies five supported abnormal groups in later ECG from known patients.

Specific objectives.

1. Train a normal-only autoencoder using the first ECG channel from all 18 MIT-BIH Normal Sinus Rhythm Database subjects, with separate fitting and validation subjects, and use reconstruction errors as features.
2. Use the earlier 80% of every MITDB record containing MLII for supervised development and exclude the final 20% from fitting and selection in the rerun.
3. Build a binary abnormality detector using ECG morphology, spectrum, RR timing, and autoencoder-residual features.

4. Build a second classifier for PVC, PAC, LBBB, RBBB, and AFib using the same MLII signal and RR context with a delay until the required signal and next R peak are available.
5. Measure abnormality-detection accuracy, conditional subtype accuracy, and complete six-class system accuracy separately.
6. Report the true and correctly classified count for every supported abnormal group.
7. Estimate uncertainty with equal-subject accuracy, a complete-subject bootstrap confidence interval, and subject-level statistical tests.

Expected outcomes. The planned outcomes are:

- a reproducible, deployable two-stage research prototype;
- abnormality-detection, subtype, and whole-system point accuracies above 90% on the fixed temporal test;
- a whole-system 95% confidence interval whose lower bound exceeds 90% and whose total width is no more than three percentage points;
- per-arrhythmia results that reveal whether every supported abnormal class exceeds 90% recall; and
- a clear account of the data, statistical, hardware, and clinical limits of the model.

1.4 Research questions

- RQ1** How does the frozen NSRDB reconstruction branch contribute evidence to the supervised hierarchy, and what evidence supports its independent benefit?
- RQ2** What are binary, conditional-subtype, and complete six-class performance on the retrospective temporal evaluation?
- RQ3** What do equal-subject accuracy and subject-clustered uncertainty show about variation across known patients?
- RQ4** Do the measured accuracy, interval precision, and class recalls meet the study targets, and what does the external check show?

1.5 Organisation of the report

Chapters 2 and 3 review related work and define the data and labels. Chapters 4 and 5 explain the theoretical foundations and methodology, including mathematical model training and inference on unseen beats. Chapter 6 describes the experiment, Chapter 7 presents the results, and Chapter 8 discusses the findings and future work.

2 Literature Review

This chapter reviews research on ECG abnormality detection, heartbeat classification, and evaluation when earlier ECG from the same patient is available. The review focuses on four areas: ECG datasets and evaluation protocols, supervised heartbeat classification, normal-only anomaly detection, and statistical validation. It is a focused narrative review, not a systematic review or meta-analysis. Papers were selected because they explain a design choice, show an important alternative, or reveal a risk in patient-independent evaluation.

Headline accuracy is not used to rank the papers. ECG studies often use different leads, labels, record lengths, patient splits, and output units. For example, a patient-specific beat classifier, a 12-lead diagnostic model, and the six-class system in this report answer different questions. Their accuracy values cannot be compared directly without accounting for these differences. This matters because a high score may come from an easier class mapping, or from placing beats from the same person in both training and test data.

2.1 ECG datasets and evaluation protocols

PhysioNet provides open physiological databases and common software tools [4]. The MIT-BIH Arrhythmia Database (MITDB) became a widely used benchmark because it contains expert beat annotations and two ECG channels [2, 5]. The MIT-BIH Normal Sinus Rhythm Database provides 18 real long-term recordings from subjects without significant arrhythmias [6]. It is suitable for normal-only learning, but its headers identify the channels only as ECG1 and ECG2. The present study therefore uses one fixed first channel and reports this uncertainty instead of assigning an unsupported anatomical lead name.

The evaluation protocol is as important as the database. A beat-wise random split can place very similar beats from one patient in both training and test sets. An inter-patient split instead tests whether the learned boundary transfers to a different person. De Chazal et al. established an influential inter-patient heartbeat study using ECG morphology and heartbeat-interval features [3]. Later research continued to treat patient variation as a central problem rather than assuming that all beats are independent [7, 8].

This evidence supports a clearly defined evaluation protocol instead of a single ambiguous score. The final experiment keeps chronological order, uses the earlier 80% of each record for development and the final 20% for testing. Boundary gaps separate the regions, and percentages count accepted beats. Although the rerun preserves chronology, earlier development already exposed the final segment. Its results are retrospective temporal evidence with possible selection optimism. The separate NSRDB cohort is used only for the normal autoencoder; no healthy record trains either supervised tree model.

2.2 Supervised heartbeat classification

Early automatic ECG systems used measured morphology and timing. De Chazal et al. combined waveform information with RR intervals, showing why beat shape and beat timing should be treated as complementary evidence [3]. This idea remains useful even when the classifier changes: morphology helps describe PVC and bundle-branch beats, while RR history helps describe premature beats and irregular rhythms.

Huang et al. studied inter-patient separation of normal, LBBB, and RBBB beats with ensemble classifiers [9]. That study is particularly relevant because bundle-branch morphology can persist through a record. However, its three-class task is narrower than the present N/PVC/PAC/LBBB/RBBB/AFib task. Its findings support considering morphology and ensemble learning, but its accuracy does not provide a direct benchmark for the broader task in this study.

Feature-based models have two practical advantages for this dataset. They make the evidence easier to inspect, and their settings can be adjusted when few patients represent a class. Their weakness is that hand-designed summaries may miss useful waveform detail. This trade-off led to the combined design used here: pooled waveform and RR features are kept, while an autoencoder adds learned normal-reconstruction evidence.

One-dimensional convolutional neural networks can learn ECG morphology directly. Kiranyaz et al. developed a patient-specific real-time classifier [10]. Its patient adaptation is useful when labelled beats from the target person exist. It is therefore directly relevant to the final patient-adapted setting, but it does not answer performance before seeing a new patient. Acharya et al. classified arrhythmias from ECG segments with a convolutional network [11]. Segment-level work demonstrates the value of learned temporal features, but its label definition and split must match the target study before accuracy can be compared.

Large deep-learning studies have also reported strong performance on ambulatory single-lead recordings and 12-lead diagnostic ECGs [12, 13]. These works show the potential of deep networks when many patients and labels are available. They also show why dataset scale matters. Their databases, lead systems, and diagnostic outputs are different from the small six-class MITDB experiment, so they are evidence for a method family rather than a performance baseline for this report.

Sequence models use neighbouring beats to add rhythm context. Mousavi et al. compared intra-patient and inter-patient heartbeat classification with a sequence-to-sequence approach [7]. Zhang et al. added an adversarial subject classifier so that the learned representation would contain less patient identity [8]. These approaches address generalisation directly, but they also need enough independent patients. The present study has only 45 eligible subject clusters, so the final system uses a simpler tree hierarchy. This is a data-availability decision, not a claim that deep learning is generally inferior.

2.3 Normal-reference reconstruction

Autoencoders learn an internal representation by reconstructing their input [14]. Denoising training corrupts the input but retains a clean target, encouraging the network to learn stable

structure rather than an exact copy [15]. In anomaly detection, the model is often trained on normal sequences and reconstruction error is used as evidence that a new sequence differs from the learned normal pattern [16, 17].

Temporal convolutional autoencoders use convolution and dilation to cover longer time patterns efficiently. Thill et al. evaluated this idea on time-series anomaly tasks that included ECG [18]. ECG-specific studies have also investigated convolutional variational and adversarial autoencoders [19, 20]. Together, these studies support using a convolutional autoencoder for normal ECG representation.

Reconstruction error alone does not establish a diagnosis. A high-capacity model may reconstruct an abnormal input well, while noise or a different electrode position may increase the error of a normal input. For that reason, this report does not use a single reconstruction threshold for its final classification. It uses four residual summaries as features in a supervised abnormal detector. The normal-only idea is kept, but the final decision can combine residual, morphology, spectral, and RR evidence.

2.4 Class imbalance and statistical validation

ECG datasets contain unequal class counts and unequal numbers of patients per class. A high overall accuracy can therefore hide failure on a rare arrhythmia. Precision–recall analysis is more informative than ROC analysis in strongly imbalanced binary problems [21], while class-balanced learning can increase the influence of rare classes [22]. This thesis therefore reports sensitivity, specificity, balanced accuracy, macro F1, and per-class recall in addition to accuracy.

Thousands of beats from a small number of subjects do not become thousands of independent patients. Resampling individual beats can underestimate uncertainty and produce confidence intervals that are too narrow. Patient-cluster bootstrap methods keep within-patient dependence [23]. The final experiment gives each of 45 subject clusters equal weight and resamples complete subjects. This also follows modern guidance to report the data split, model development, and validation population clearly [24].

2.5 Research gap and contribution

The reviewed studies cover normal-reference reconstruction, waveform and RR features, and methods that reduce dependence on patient identity. They also address bundle-branch classification and statistical analysis of clustered data. However, they study different tasks. Some use different labels or more leads; others adapt to a known patient or detect an anomaly without assigning a subtype.

This study brings these methods together in a two-stage system and evaluates both its component models and its complete output. Its contribution is the integration and evaluation of the following elements:

1. learn normal reconstruction from one fixed channel of all 18 real MIT–BIH normal-sinus recordings;

2. use a chronological 80/20 experiment for patient-adapted temporal evidence;
3. exclude the final test portion from selection in the rerun and report its earlier use during development; and
4. report separate abnormal, subtype, whole-system, patient-level, and uncertainty results.

Table 2.1 summarises the papers that contributed most directly to the data protocol, model architecture, and evaluation method used in this research.

Table 2.1: Main papers referenced in this study and their contribution.

Paper title	Author(s)	Main contribution used in this research
MIT-BIH Normal Sinus Rhythm Database [6]	George B. Moody	Provides the 18 real long-term recordings without significant arrhythmias used for normal-only autoencoder development; its generic channel names also motivate explicit reporting of lead uncertainty.
Automatic Classification of Heartbeats Using ECG Morphology and Heartbeat Interval Features [3]	Philip de Chazal, Maria O'Dwyer, and Richard B. Reilly	Established the value of combining ECG morphology with RR intervals and carefully defining the evaluation population. This supports the morphology, timing, and reporting choices of the present system.
Reducing the Dimensionality of Data with Neural Networks [14]	Geoffrey E. Hinton and Ruslan R. Salakhutdinov	Provides the representation-learning basis of the autoencoder used to learn normal ECG structure.
Extracting and Composing Robust Features with Denoising Autoencoders [15]	Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol	Supports adding noise to the input while keeping a clean reconstruction target, so the autoencoder learns stable normal patterns.
Temporal Convolutional Autoencoder for Unsupervised Anomaly Detection in Time Series [18]	Markus Thill, Wolfgang Konen, Hao Wang, and Thomas Baeck	Supports temporal convolution, dilation, and reconstruction error for detecting unusual time-series patterns, including ECG signals.
Inter- and Intra-patient ECG Heartbeat Classification for Arrhythmia Detection: A Sequence to Sequence Deep Learning Approach [7]	Sajad Mousavi and Fatemeh Afghah	Shows the importance of neighbouring-beat rhythm context and clearly separates intra-patient from inter-patient evaluation.
Interpatient ECG Heartbeat Classification with an Adversarial Convolutional Neural Network [8]	Jing Zhang, Aiping Liu, Deng Liang, Xun Chen, and Min Gao	Explains patient-identity leakage and why results from known-patient temporal tests must not be presented as new-patient performance.

Paper title	Author(s)	Main contribution used in this research
Detection of Inter-Patient Left and Right Bundle Branch Block Heartbeats in ECG Using Ensemble Classifiers [9]	Huifang Huang, Jie Liu, Qiang Zhu, Ruiping Wang, and Guangshu Hu	Supports morphology and ensemble classifiers for LBBB and RBBB, while showing that this specialised three-class task is narrower than the task in this report.
The Precision–Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets [21]	Takaya Saito and Marc Rehmsmeier	Justifies reporting precision, recall, and F1 because the ECG classes are imbalanced and accuracy alone can hide failure.
Bootstrap Estimation of Diagnostic Accuracy with Patient-Clustered Data [23]	Carolyn M. Rutter	Provides the reason for resampling complete patient records instead of treating correlated beats as independent observations.
TRIPOD+AI Statement: Updated Guidance for Reporting Clinical Prediction Models That Use Regression or Machine Learning Methods [24]	Gary S. Collins et al.	Supports transparent reporting of the patient split, model-development process, temporal evaluation, limitations, and intended use.

The next chapter defines the data sources and annotation groups used to put these design choices into practice. The study evaluates their combination; it does not establish that any single component improves performance on its own.

3 Data and Preparation

This chapter identifies the ECG sources, explains how the signals are prepared, and shows how examples enter model fitting, validation, and the temporal test. Keeping these roles separate prevents healthy data or test labels from being used in the wrong part of the study.

3.1 Data Sources and Roles

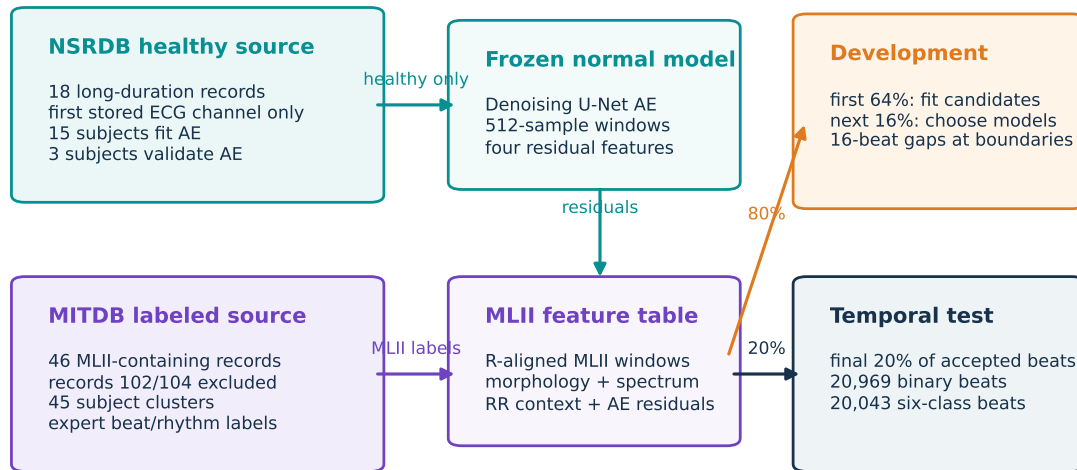
Two public PhysioNet databases supply model development data. A separate, limited external INCART check is reported in the results chapter. The MIT-BIH Normal Sinus Rhythm Database (NSRDB) supplies normal-reference ECG for the autoencoder. The MIT-BIH Arrhythmia Database (MITDB) supplies labelled normal and abnormal beats for the detector, subtype classifier, and final temporal evaluation [4, 6, 5].

Table 3.1: Data sources and their strictly separated roles.

Source	Available data	Use in this study
MIT-BIH Normal Sinus Rhythm DB (NSRDB)	18 real long-term ECG recordings from five men aged 26–45 and 13 women aged 20–50, with no significant arrhythmias; two stored channels named ECG1 and ECG2; 128 Hz; approximately 23–26 hours per record [6].	The complete first stored ECG channel from every record is processed. Fifteen subjects fit the normal-only autoencoder and three different subjects validate it and determine the normal residual threshold.
MIT-BIH Arrhythmia DB (MITDB)	48 approximately 30-minute, two-channel ambulatory ECG records from 47 subjects (25 men aged 32–89 and 22 women aged 23–89), sampled at 360 Hz, with expert beat and rhythm annotations [5, 2].	Only the 46 records containing MLII are eligible. Their earlier 80% supplies supervised development and their final 20% supplies the temporal test. These records represent 45 subject clusters because records 201 and 202 belong to one person.

Figure 3.1 summarises these roles. NSRDB supplies normal-reference windows for the autoencoder, while MITDB supplies labelled MLII beats for supervised development and temporal testing.

Dataset roles and evaluation boundaries



The temporal test uses later signal from the same MITDB patients; it is not an unseen-patient test.

Figure 3.1: Dataset roles and evaluation boundaries. The healthy NSRDB channel creates the frozen normal model. MITDB MLII creates the supervised feature table and is split chronologically into development and temporal-test regions.

Why NSRDB is used for healthy training. NSRDB contains real long-duration ECG from subjects reported to have no significant arrhythmias. This matches the autoencoder idea: the network sees normal structure without supervised arrhythmia labels. This is a nominally normal source cohort, not an annotation-verified guarantee that every window is normal. The preparation code does not exclude windows using NSRDB beat annotations. All 18 subjects and their complete recordings enter the cleaning process, giving 437.49 hours of source ECG. Complete recordings also expose the model to normal within-day variation instead of a short selected excerpt.

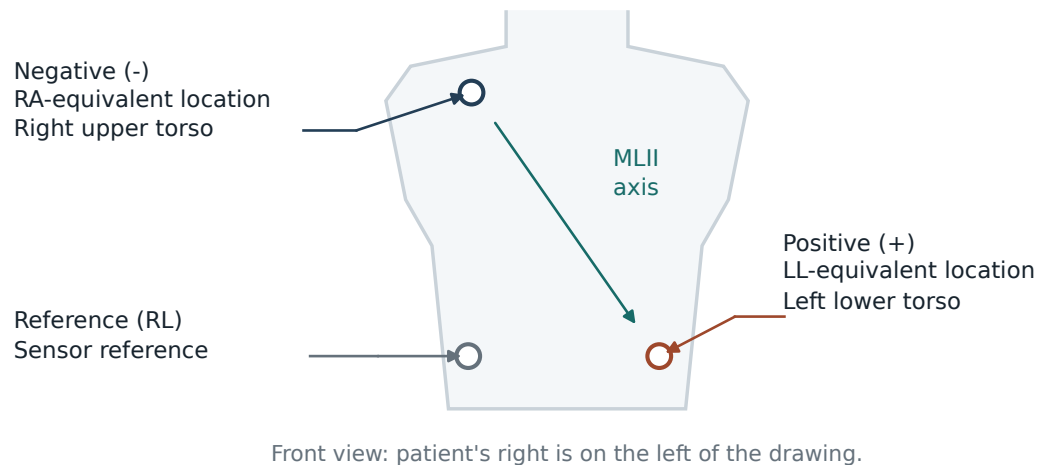
Exactly one channel is used. NSRDB headers call the two waveforms ECG1 and ECG2; they do not identify either as a standard anatomical lead. This study therefore selects the first stored channel consistently and describes it as the *first NSRDB ECG channel*. It is not renamed Lead II. The second channel never enters autoencoder fitting, validation, threshold estimation, or residual calculation.

The supervised branch uses MLII only. Modified Lead II is a bipolar torso lead with a negative RA-equivalent electrode on the right upper torso and a positive LL-equivalent electrode on the left lower torso. Figure 3.2 shows this polarity schematically; the separate

reference electrode is shown for the project's sensor connection. It does not represent a second ECG channel. PhysioNet describes the modified torso lead and its generally prominent normal QRS complexes [25, 5]. The loader selects MLII by its header name rather than assuming a channel number. MLII is stored first in 45 eligible records; record 114 stores V5 first and MLII second. Records 102 and 104 contain V5 and V2 but no MLII and are excluded. Consequently, the detector, classifier, temporal test, and installed replay all receive one MLII waveform.

Both data streams contain one channel, but their lead identities differ: the autoencoder learns healthy structure from the first NSRDB channel, while the supervised hierarchy operates on MLII. This source difference is a limitation because the electrode placement for the NSRDB channel is not documented.

A MLII polarity and torso placement



B One channel per input; channel identities differ

Normal autoencoder: NSRDB

First stored channel: ECG1
Anatomical placement is unspecified.
Used to learn a normal ECG reference.

Supervised models: MITDB

Select MLII by its header name.
Record 114: MLII is the second channel.
Exclude 102 and 104: neither has MLII.

Figure 3.2: MLII polarity and the channel used by each model. Panel A is an anterior-view schematic of the modified Lead-II axis, with the project's reference connection shown separately; it is not an exact map of the database electrode coordinates [25]. Panel B distinguishes the NSRDB ECG1 normal reference from the MLII input used by the supervised models.

Reference labels. MITDB expert annotations are mapped to the six project outputs as shown in Table 3.2. AFib is determined from the rhythm annotation around a beat; the other targets mainly use beat symbols.

Table 3.2: Mapping from MITDB expert annotations to project outputs.

Output	MITDB evidence	Interpretation
N	N, e, j outside AFib/AFL	normal-like/escape group
PVC	V, E	ventricular premature or escape beat
PAC	A, a, J, S	supraventricular premature beat
LBBB	L	left bundle branch block beat
RBBB	R	right bundle branch block beat
AFib	normal-like beat inside AFIB/AFL rhythm	atrial fibrillation/flutter context

The class codes are project groupings, not exact clinical diagnoses or the standard AAMI five-class scheme. N includes atrial and junctional escape beats; PVC includes ventricular escape beats; PAC includes nodal and supraventricular premature beats. AFib combines fibrillation and flutter rhythm annotations only when the beat symbol maps to N. Other supported beat types retain their morphology label during AFIB/AFL. Consequently, AFib recall here is neither pure AF sensitivity nor episode-level AF detection.

Unsupported symbols remain abnormal in binary evaluation and are excluded from six-class scoring. At runtime, a gated unsupported abnormality still receives one of the five available subtype labels. The prepared MLII dataset contains 104,755 accepted beats: 63,073 in group N and 41,682 abnormal. Of these, 100,072 have a supported six-class label.

3.2 Data Cleaning

Data preparation converts the two ECG sources into a common input shape: one four-second signal with 512 samples. The supervised preparation rules were fixed for the rerun and are also used during session prediction. NSRDB retains its original preparation procedure. Figure 3.3 shows the full transformation for one healthy NSRDB segment and one MITDB MLII beat.

3.2.1 Cleaning Checklist

Table 3.3 lists the preparation steps and their purpose. Rejected windows do not enter model fitting or evaluation.

Table 3.3: Cleaning checklist applied before a signal can become a model input.

Step	What is done	Why it matters
Required channel	NSRDB must provide ECG1; MITDB must provide MLII. Missing required signals stop processing.	Prevents accidental use of ECG2, V5, V2, or another non-matching lead.
Sampling rate	NSRDB remains at 128 Hz. MITDB MLII is resampled from 360 Hz to 128 Hz with polyphase anti-aliasing.	Makes every four-second model window exactly 512 samples.
Band-pass filter	A Butterworth band-pass with prototype order four and cutoffs 0.5–50 Hz uses forward–backward filtering. For supervised inputs this is applied within the four-second window [26, 27].	Reduces baseline drift and high-frequency noise while avoiding filter-induced phase delay.
Amplitude standardisation	Each supervised MLII window is converted to approximately zero mean and unit standard deviation; NSRDB retains its archived scaling.	Reduces gain differences while preserving the relative P–QRS–T shape.
Window quality check	Corrected supervised windows must be finite and non-flat. Both neighbouring R peaks must be observed.	Rejects unusable inputs. These checks do not establish clinical signal quality or remove all motion artifacts.

3.2.2 Window Construction

The autoencoder and supervised hierarchy build windows in different ways, but the final array shape is the same. The autoencoder receives consecutive, non-overlapping four-second windows from the first stored NSRDB channel. The supervised MLII windows are aligned to a beat: 200 samples before the R peak and 312 samples starting at the R peak. During offline experiments, expert annotations give the R-peak location for alignment and later scoring. During live use, detected R peaks replace those annotations.

For samples $x_1, \dots, x_n$ inside one prepared model window, amplitude standardisation is

$$z_i = \frac{x_i - \mu}{\sigma + 10^{-8}}, \quad \mu = \frac{1}{n} \sum_{i=1}^n x_i. \quad (3.1)$$

The corrected supervised pipeline filters each extracted window and applies this operation locally. The original experiment instead standardised each complete MITDB record before

extracting windows. That used test-region signal statistics during preparation and differed from sensor chunks. The results reported here use the local preparation procedure. NSRDB remains the frozen reference checkpoint with its archived input scaling. The real-data audit in Section 5.10.7 shows that an archived NSRDB window is not unit variance; the current cache-building code must not be treated as proof of the older cache's normalisation.

3.2.3 Cleaning Yield

The healthy NSRDB recordings produce many candidate windows because complete records are used instead of short excerpts. Across the 18 complete NSRDB records, 393,744 candidate windows were formed. The quality checks kept 345,850 windows and rejected 47,894. These accepted windows were then split into 288,673 autoencoder fitting windows and 57,177 validation windows.

MITDB cleaning produces beat-aligned MLII examples. The prepared MLII dataset contains 104,755 beats: 63,073 in group N and 41,682 abnormal. Unsupported abnormal symbols are still used for binary abnormality evaluation, but they are excluded from six-class subtype scoring.

3.2.4 Preparation during prediction

The corrected hierarchy applies the same local window filtering, standardisation and feature order during fitting, offline scoring and installed use. Database replay first resamples the signal. Sensor acquisition supplies 128 Hz samples. Labels never enter a prediction. The live backend waits until there is enough signal around a detected R peak and enough RR context. It then feeds one prepared MLII window to the frozen system. Neither supervised model is retrained during prediction.

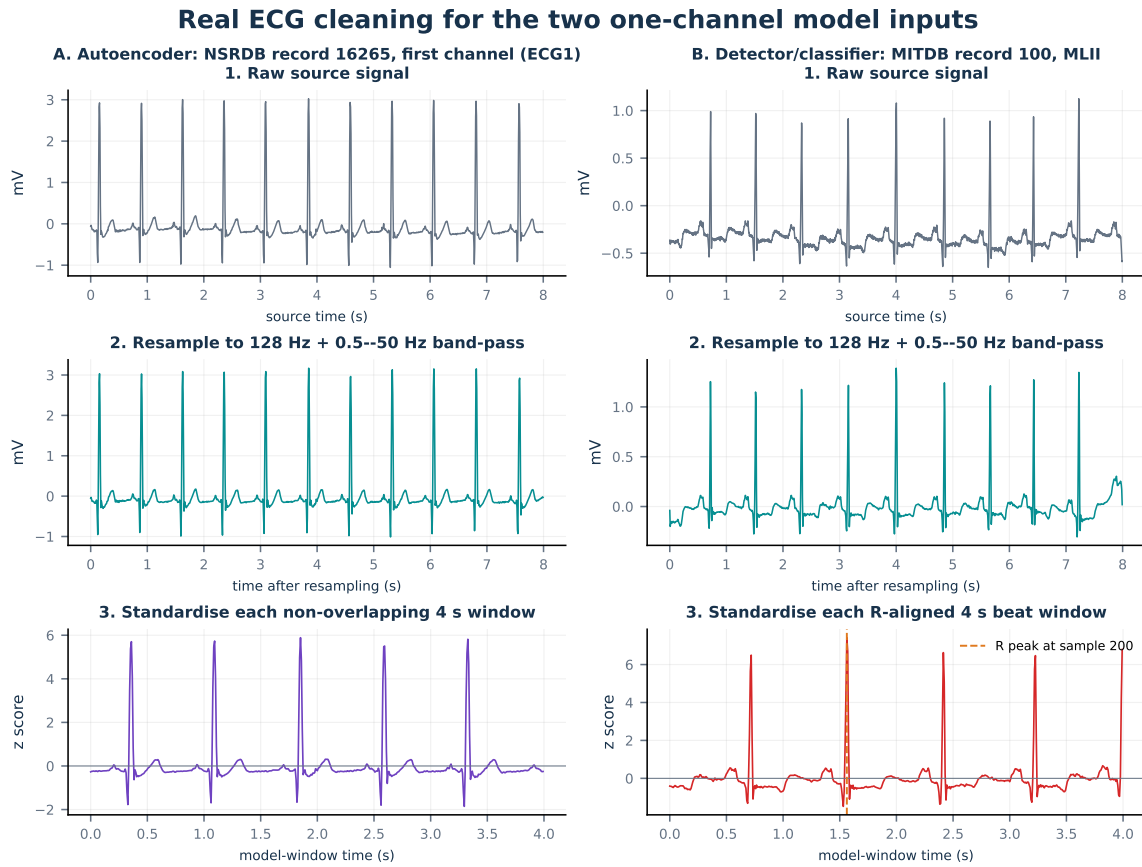


Figure 3.3: Cleaning real ECG for the two roles. The healthy example is the first channel of NSRDB record 16265. The supervised-input example is MLII from MITDB record 100. Both become one 512-sample input, but their documented channel identities remain different.

3.3 Data Preparation

Data preparation decides which cleaned examples are allowed to fit a model, which examples choose model settings, and which examples are held back for the final temporal result. Figure 3.4 shows this flow from raw source to frozen prediction.

3.3.1 Prepared Objects

After cleaning, the study keeps two kinds of prepared object:

- **NSRDB window:** a normal-reference [1, 512] window from the first stored ECG channel. It has no arrhythmia label and is used only by the normal autoencoder.
- **MITDB beat:** an R-aligned MLII [1, 512] window with record identity, binary normal/abnormal truth, supported subtype truth when available, and RR timing context.

The autoencoder input tensor has shape $[B, 1, 512]$. The supervised detector and subtype

classifier do not receive raw samples alone. Each MITDB beat is converted into the same 242-value MLII feature vector:

$$\begin{aligned} 242 \text{ values} &= 230 \text{ morphology, spectrum, statistics, and base-RR values} \\ &+ 8 \text{ causal RR-history values} \\ &+ 4 \text{ autoencoder residual values.} \end{aligned}$$

The subtype model is fitted only on the five supported abnormal classes: PVC, PAC, LBBB, RBBB, and AFib.

3.3.2 Splits

The autoencoder split is subject-disjoint. A seeded split assigns 15 complete NSRDB records to fitting and three different records (16420, 16483, and 16795) to validation. This produces 288,673 fitting windows and 57,177 validation windows. Validation selects the checkpoint and estimates the normal residual threshold. There is no separate healthy test because the autoencoder is frozen before it contributes residual features to MITDB evaluation.

The MITDB split is chronological inside every eligible record:

- the first 64% fits candidate supervised models;
- the next 16% selects the model structures and abnormal-score threshold;
- a 16-beat gap is removed before both the validation and test boundaries during selection; and
- the final 20% evaluates the completed frozen hierarchy.

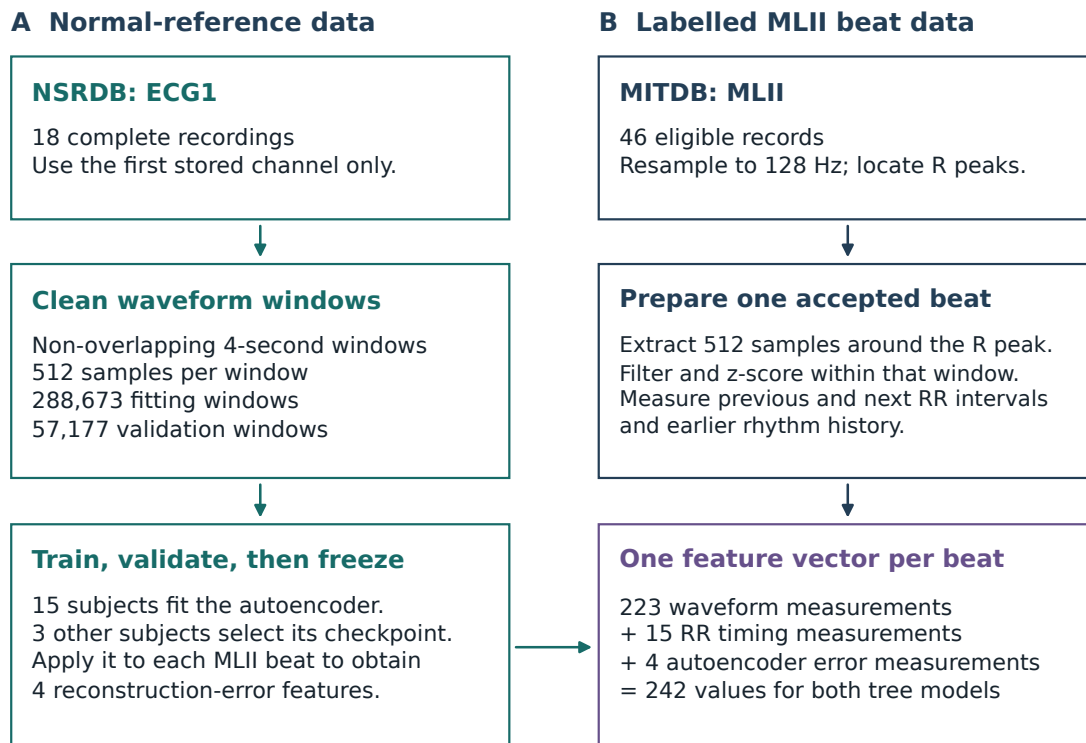
Percentages count accepted beats in chronological order, not exact elapsed time. Final refitting uses the earlier 80% while retaining the gap before the test. The procedure checks that signal windows do not overlap across each boundary. The test had been observed during earlier development, making this a retrospective known-patient evaluation with possible optimism.

3.3.3 Exact Model Allocation

Table 3.4: Training, validation, and temporal-test data supplied to each model.

Model	Candidate training	Validation	Final fitting and testing
Autoencoder	288,673 first-channel windows from 15 complete NSRDB records	57,177 windows from 3 different complete NSRDB records	Frozen while every MLII test beat produces four residual features
Abnormality detector	66,285 MLII beats: 40,469 normal and 25,816 abnormal	16,029 beats: 9,664 normal and 6,365 abnormal	Fitted on 83,050 development beats; tested on 20,969 temporal-test beats
Subtype classifier	22,844 supported abnormal MLII beats	5,651 supported abnormal beats	Fitted on 28,758 supported abnormal beats; conditionally tested on 7,958 supported abnormal beats

The supported six-output test contains 20,043 beats: 12,085 N, 1,527 PVC, 589 PAC, 1,481 LBBB, 1,383 RBBB, and 2,978 AFib. Another 926 unsupported abnormal beats remain in binary testing but not six-class scoring.



Expert labels are stored separately from the 242 input features.

Earlier MITDB beats develop the models; later beats evaluate their predictions.
The first and last eligible beat are excluded when a neighbouring RR is unavailable.

Figure 3.4: Preparation of normal-reference windows and labelled MLII beats. Panel A develops the frozen autoencoder from subject-disjoint NSRDB fitting and validation data. Panel B uses local filtering and standardisation for each MITDB beat, then combines waveform, RR timing, and frozen-autoencoder residual features. Expert labels remain separate from the model input.

3.3.4 Feeding the Frozen System

During autoencoder fitting, Gaussian noise with standard deviation 0.03 is added to each input while the clean NSRDB window remains the target. During supervised fitting, record-and-class weights reduce domination by long records and common classes. During prediction, one MLII window, its observed previous and next RR intervals, and its recent RR history are converted into the same 242-value vector used in the temporal experiment. A decision of N ends classification for that beat. An abnormal decision passes the same vector to the subtype model. The next chapter explains the theory behind these features and the two-stage decision.

4 Theoretical Foundations

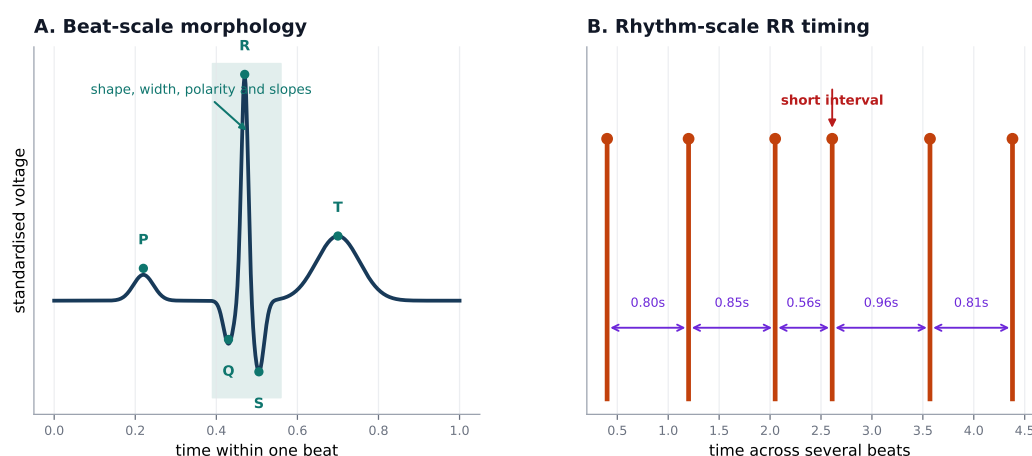
This chapter explains the theory behind the final ECG system. The system combines MLII waveform shape, RR timing, and autoencoder residuals in two supervised Extra Trees ensembles. These components provide complementary information about beat shape and rhythm.

The explanation follows one beat from measured voltage to the final label. The equations define the main operations and explain how they contribute to classification. The three trained components are a normal-reference autoencoder, a normal-versus-abnormal tree ensemble, and an abnormal-subtype tree ensemble. The autoencoder learns by changing neural weights to lower reconstruction error; the tree ensembles learn by choosing weighted, entropy-reducing splits. These are different forms of training.

4.1 Why ECG evidence has more than one form

An ECG is a voltage signal over time. For arrhythmia classification, useful evidence appears at two related scales. The first is beat-scale morphology: what the P–QRS–T shape looks like around one R peak. The second is rhythm timing: when the beat occurs relative to neighbouring beats. The final model uses both because they answer different questions.

Two ECG evidence scales used by the final model



The final feature vector joins these scales with four normal-reconstruction residuals.

Figure 4.1: Two evidence scales used by the final model. Morphology describes the local shape of a beat; RR timing describes rhythm context around the beat.

PVC, LBBB, and RBBB can strongly affect QRS shape, width, slopes, and polarity. PAC and AFib may be easier to detect when timing evidence is also present: premature beats create short

RR intervals, pauses create long RR intervals, and AFib creates repeated irregularity. Published heartbeat-classification work also shows the value of combining waveform morphology with heartbeat-interval features [3, 7].

How one ECG beat becomes a supported output

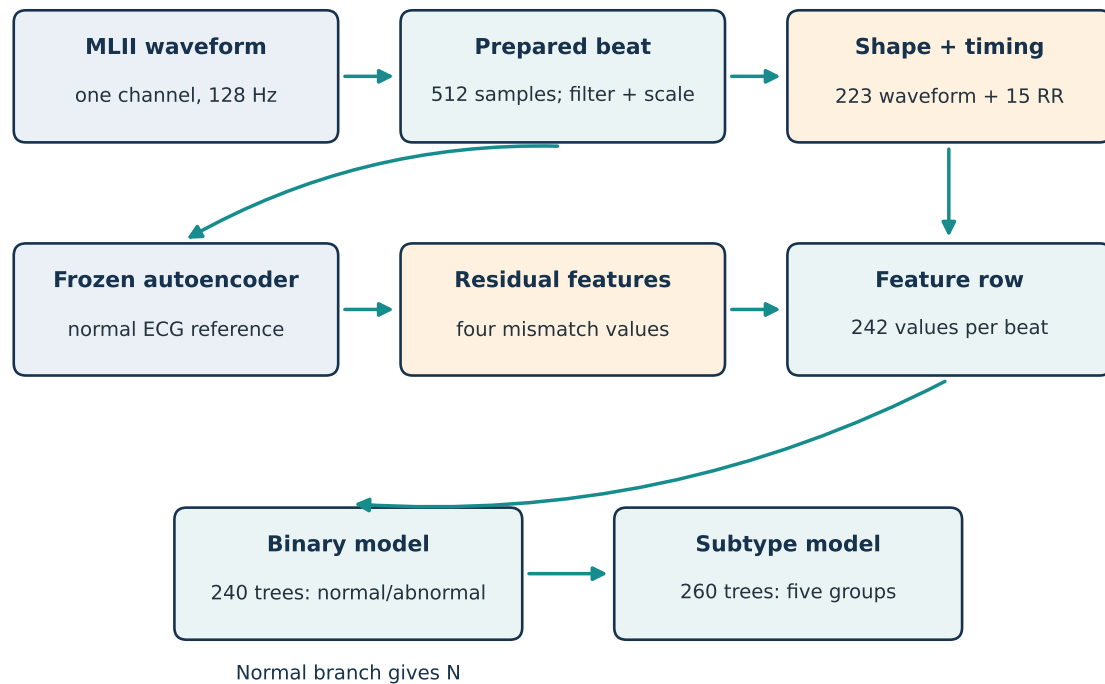


Figure 4.2: Reading map for the final system. ECG signal preparation produces one 512-sample window. Three complementary feature groups make one 242-value row. The frozen autoencoder creates residual features; the two fitted tree ensembles make the final decision. Labels are used during supervised fitting, never as input features at prediction time.

4.2 From voltage to a comparable four-second window

An electrode channel measures a sequence of voltage differences. Sampling at 128 samples per second turns four seconds into $T = 4(128) = 512$ numbers. The MITDB waveform is originally sampled at 360 Hz and is resampled to 128 Hz before beat extraction. Anti-aliasing in the resampling step prevents high-frequency content from appearing falsely at lower frequencies. The normal-reference NSRDB channel is already sampled at 128 Hz. Its header calls the first channel ECG1; the supervised and installed beat input is MLII. These channel names should not be treated as interchangeable anatomical lead names [5, 6].

For supervised examples, an observed R peak is placed at sample 200. Thus a window contains 200 samples before and 312 samples from that point onward. The R peak is a useful alignment anchor because many important morphological changes occur in or near the QRS

complex. The normal-reference autoencoder instead receives consecutive, non-overlapping four-second windows; it does not need beat labels or R-peak annotations. Figure 4.2 shows where the two streams meet.

The corrected supervised path applies a Butterworth band-pass filter with prototype order four and 0.5–50 Hz cutoffs to *each* 512-sample window. Forward–backward filtering removes the phase shift that a one-directional filter would introduce at an R peak. The lower cutoff attenuates slowly changing baseline, and the upper cutoff attenuates faster interference. It does not imply that all noise or motion artifact has disappeared [26, 27]. The NSRDB preparation filters longer recording blocks before making windows; its archived windows retain their historical scaling, as audited in Section 5.10.7. The local standardisation equation below describes the corrected supervised MLII path.

If u_t is the filtered voltage in one window, the model receives

$$x_t = \frac{u_t - \bar{u}}{s_u + 10^{-8}}, \quad \bar{u} = \frac{1}{T} \sum_{t=1}^T u_t, \quad s_u = \sqrt{\frac{1}{T} \sum_{t=1}^T (u_t - \bar{u})^2}. \quad (4.1)$$

Subtracting the window mean removes a local offset. Dividing by its standard deviation reduces gain differences between recordings. The waveform keeps its *relative* shape and timing, but absolute millivolt amplitude is no longer directly available to the model. Flat or non-finite windows are rejected. Because scaling is calculated inside the current window, later test signal statistics are not needed when preparing an earlier training beat.

4.3 Feature extraction theory

The raw 512-sample MLII window contains more detail than the final tree models need. Small changes in R-peak alignment or noise can change many raw sample values. The system therefore converts the window into compact measurements that keep useful ECG evidence while reducing unnecessary sample-level detail.

Transparent operations that create the final feature vector

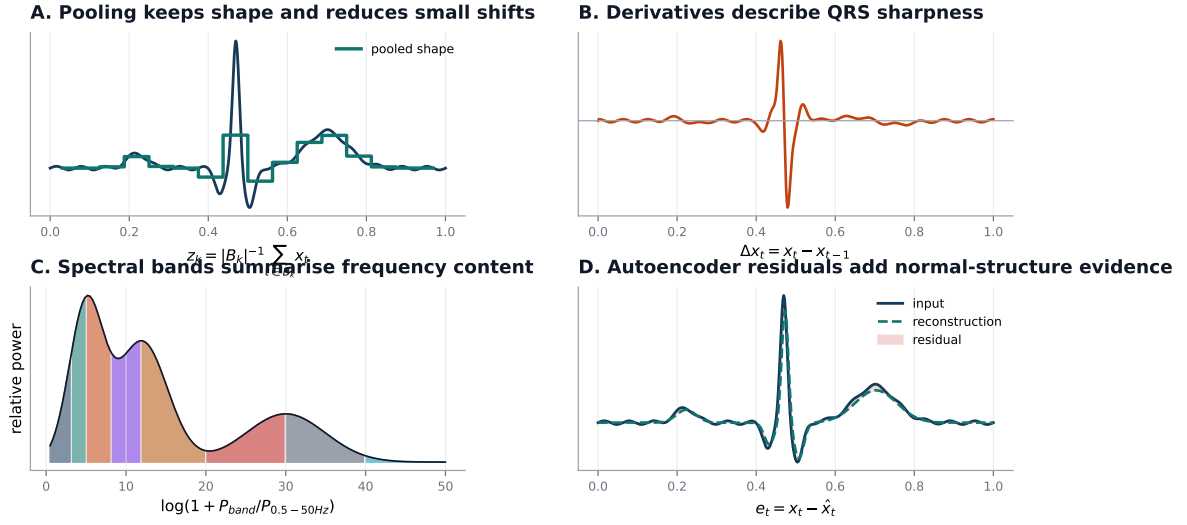


Figure 4.3: Feature-extraction operations behind the final 242-value vector. The tree models learn from compact numerical evidence, not from raw ECG samples directly.

For a pooled region B_k , the pooled value is

$$z_k = \frac{1}{|B_k|} \sum_{t \in B_k} x_t. \quad (4.2)$$

Pooling keeps the average shape in a small region and makes the representation less sensitive to a one- or two-sample shift. The final system uses 64 whole-window pooled values and 80 central morphology pooled values.

The derivative

$$\Delta x_t = x_t - x_{t-1} \quad (4.3)$$

describes how quickly voltage changes. This matters because QRS abnormalities often change slopes and sharpness. The system keeps 60 pooled derivative values from the central beat region.

Frequency features describe how signal energy is spread from 0.5 to 50 Hz. For a frequency band b , the relative band feature is

$$s_b = \log \left(1 + \frac{P_b}{P_{0.5-50}} \right), \quad (4.4)$$

where P_b is the power inside the band and $P_{0.5-50}$ is total power in the ECG band used by the feature extractor. This band-limited view follows the general ECG idea of keeping physiological content while reducing drift and high-frequency interference [1, 26].

Eleven summary statistics add scale, range, energy, skewness, and kurtosis. Together, the waveform, derivative, spectral, and statistical values form 223 non-RR, non-autoencoder features.

The spectrum is computed with a discrete Fourier transform. For a 512-sample window, the transform coefficient at frequency index k is

$$X_k = \sum_{t=0}^{T-1} x_t \exp\left(-\frac{2\pi i k t}{T}\right), \quad P_k = |X_k|^2. \quad (4.5)$$

The implementation sums P_k in eight frequency bands between 0.5 and 50 Hz and divides each sum by total power in that range before applying $\log(1 + \cdot)$. The logarithm reduces differences between the relative-power values. These values describe the distribution of variation by frequency; they are not separate clinical diagnoses. The eleven statistics include mean, standard deviation, minimum, maximum, range, mean absolute value, root-mean-square amplitude, derivative spread and maximum slope, skewness, and excess kurtosis. Skewness describes asymmetry and kurtosis describes unusually large deviations. The exact count is $64 + 80 + 60 + 8 + 11 = 223$. This count matches the implemented feature vector.

4.4 RR timing theory

Let r_i be the time of the R peak of beat i . The RR interval is

$$RR_i = r_i - r_{i-1}. \quad (4.6)$$

RR values describe rhythm. A short interval can suggest a premature beat. A long interval can suggest a pause. Repeated interval changes support rhythm irregularity.

The final live system classifies a beat only after it has enough context to compute the same RR information used in the final experiment. It uses seven base RR values, including previous RR, next RR, local mean RR, interval ratios, local coefficient of variation, and heart rate. It then appends eight history-based values from already completed intervals: median, interquartile range, median absolute deviation, RMSSD, pNN50, relative range, trend, and latest interval change.

RMSSD is one example:

$$\text{RMSSD} = \sqrt{\frac{1}{K-1} \sum_{j=2}^K (RR_j - RR_{j-1})^2}. \quad (4.7)$$

The next-RR requirement is important. It delays the streaming decision until the following peak is observed and allows the same RR features to be used in offline and live processing.

For clarity, the seven base values are previous RR, next RR, mean of up to ten earlier intervals, previous/mean, next/mean, interval coefficient of variation, and heart rate $60/RR_{\text{previous}}$. The eight additional values summarise up to twenty already completed intervals. The interquartile range is the 75th minus the 25th percentile; median absolute deviation is the median distance from the median; and pNN50 is the fraction of successive interval differences exceeding 0.05 seconds. A trend and latest relative change capture direction. These quantities provide rhythm context without giving a class label to the model. “Next RR” is available only after the following R peak is observed, so this system makes a delayed beat decision rather than an instantaneous R-peak decision.

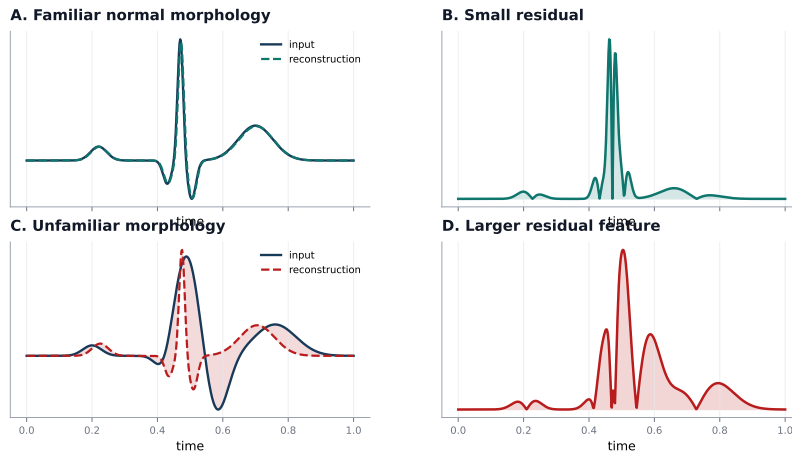
4.5 Normal-only reconstruction theory

An autoencoder contains an encoder f_θ and decoder g_ϕ . It is trained to reconstruct a normal ECG window x :

$$\hat{x} = g_\phi(f_\theta(x)), \quad \min_{\theta, \phi} \frac{1}{T} \sum_{t=1}^T (x_t - \hat{x}_t)^2. \quad (4.8)$$

This follows the representation-learning idea of autoencoders [14]. The final autoencoder is trained with a denoising objective: small Gaussian noise is added to the input while the target remains the clean normal window. Denoising encourages stable structure instead of exact noise copying [15].

Normal-only reconstruction: residual is evidence, not diagnosis



The supervised detector receives four residual summaries and decides together with morphology and RR evidence.

Figure 4.4: Normal-only reconstruction principle. A larger residual can indicate novelty, but it is evidence only; the final arrhythmia decision is made by the supervised hierarchy.

If the autoencoder has learned common normal structure, an unfamiliar or abnormal beat may be reconstructed less accurately. This is the one-class anomaly detection idea [16]. It is not a guarantee. A flexible decoder may reconstruct an abnormal waveform well, and noise may create a large residual for a normal waveform. Therefore, the final system uses autoencoder mismatch as four supervised input features, not as the final diagnosis rule.

Let $e_t = x_t - \hat{x}_t$ be the residual. The four residual features are:

$$e_{\text{MAE}} = \frac{1}{T} \sum_t |e_t|, \quad (4.9)$$

$$e_{\text{MSE}} = \frac{1}{T} \sum_t e_t^2, \quad (4.10)$$

$$e_{\text{local}} = \frac{1}{|Q|} \sum_{t \in Q} |e_t|, \quad (4.11)$$

$$e_{\Delta} = \frac{1}{T-1} \sum_{t=2}^T |e_t - e_{t-1}|. \quad (4.12)$$

Here Q is the central 240-sample region used by the feature extractor, defined in Chapter 5. The four features measure whole-window mismatch, squared mismatch, central morphology mismatch, and residual sharpness. The final architecture appends these four values to the MLII waveform and RR features.

4.5.1 What the encoder and decoder actually compute

The implemented autoencoder is a one-dimensional convolutional network. A convolution learns a small template and slides it across time. For output channel j at time t , a simplified convolution is

$$h_{j,t} = b_j + \sum_c \sum_{m=0}^{K-1} W_{j,c,m} x_{c,t+m}. \quad (4.13)$$

The learnable numbers $W_{j,c,m}$ and b_j determine which short patterns activate the channel. A first layer may respond to a steep QRS slope; later layers can combine such responses over longer time spans. The actual network uses four encoder blocks to reduce time resolution from 512 to 32 positions while increasing the channel count to 256. Four decoder blocks restore the waveform. Figure 5.4 gives the verified tensor sizes.

Several architectural functions have distinct purposes. Group normalisation stabilises intermediate activations. LeakyReLU keeps a non-zero slope for negative activations, which helps gradients propagate. Residual connections add a block's input to its transformed output, providing a shorter route for both signal and gradient flow. Dilation rates 1, 2, 4, and 8 widen the bottleneck's temporal view without proportionally increasing parameters. Decoder skip connections pass selected encoder detail forward, scaled by 0.5 in this checkpoint; this helps restore ECG shape but also limits direct copying. Dropout randomly suppresses activations during fitting to discourage over-reliance on individual pathways. The last layer is linear so it can produce both positive and negative standardised voltages [28, 29, 30].

4.5.2 How the autoencoder lowers reconstruction loss

Let $x^{(i)}$ be a clean normal window and let $\epsilon_t^{(i)} \sim \mathcal{N}(0, 0.03^2)$ be freshly drawn Gaussian noise. The training input is $\tilde{x}^{(i)} = x^{(i)} + \epsilon^{(i)}$, while the target is still $x^{(i)}$. For a mini-batch B the actual fitting objective is

$$\mathcal{L}_B(\theta, \phi) = \frac{1}{|B|T} \sum_{i \in B} \sum_{t=1}^T \left[x_t^{(i)} - g_\phi(f_\theta(\tilde{x}^{(i)}))_t \right]^2. \quad (4.14)$$

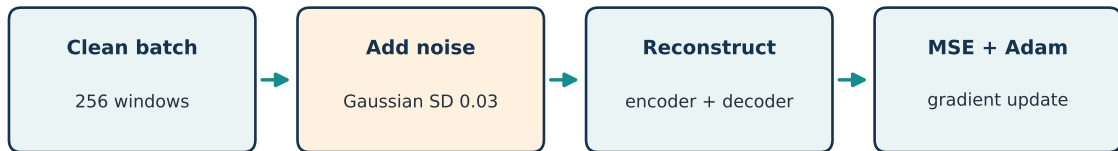
Squaring makes large reconstruction errors count more strongly. The network must predict clean structure from a slightly perturbed signal, so simply copying the input noise is a poor solution. For one output sample, the derivative of squared error with respect to reconstruction is proportional to $2(\hat{x}_t - x_t)$: an output above the target is pushed downward and one below the target is pushed upward. Backpropagation applies the chain rule through the decoder and encoder to find the effect of every learned weight [15].

The fitting loop repeatedly performs four operations: clear old gradients, compute reconstructed windows, backpropagate $\mathcal{L}_B$, and take one Adam step. Adam tracks moving averages of gradients and squared gradients; schematically, the update is

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k, \quad v_k = \beta_2 v_{k-1} + (1 - \beta_2) g_k^2, \quad \theta_{k+1} = \theta_k - \eta \frac{\widehat{m}_k}{\sqrt{\widehat{v}_k} + \varepsilon}. \quad (4.15)$$

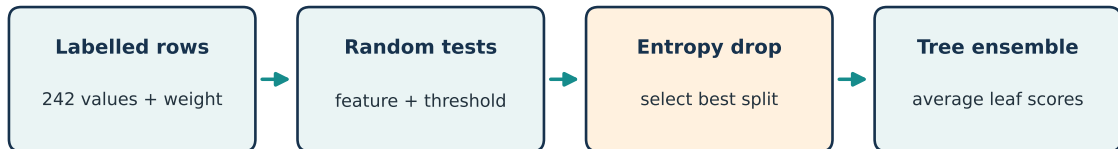
Here g_k is the mini-batch gradient, η is the learning rate, and hats indicate bias correction. The same operation updates decoder parameters ϕ . Thus “reducing loss” means changing millions of network weights in a direction estimated to reduce expected reconstruction error; it does not mean manually adjusting a threshold [31].

A. Autoencoder: repeated weight updates



Repeat for each batch; validate on held-out clean windows after each epoch.

B. Extra Trees: recursive node splits



Repeat splitting within each tree; compare finished models on validation data.

Figure 4.5: Two distinct learning mechanisms. The autoencoder repeatedly changes weights from mini-batch reconstruction gradients; an Extra Trees model fixes each node by comparing randomly proposed splits using weighted entropy. Only the autoencoder has an epoch-by-epoch reconstruction-loss curve.

The concrete run used 288,673 fitting windows from 15 NSRDB subjects and 57,177 windows from three different validation subjects. It used batches of 256, Adam with $\eta = 0.001$, and two epochs (two passes through fitting windows). Validation uses the clean window as both input and target, with dropout disabled. The checkpoint with the smaller validation mean squared error is kept. The saved measurements are:

Epoch	Noisy-input fitting MSE	Clean-input validation MSE
1	0.057639	0.061195
2	0.041976	0.061193

Training MSE fell substantially, while validation MSE changed by only about 0.0000023. The two curves are measured with different input conditions, so their gap is not a direct estimate of overfitting. The near-flat validation result supports selecting epoch 2 by the stated rule, but it is weak evidence of improved reconstruction on new normal subjects. The measured curves are in Figure 5.5. The frozen network is then applied to MLII windows; its residuals are four features, not a clinical diagnosis.

4.6 Hierarchical classification theory

The hierarchy divides classification into two stages. The first model assigns a normal or abnormal label. Beats assigned to the abnormal group then enter the subtype model.

Hierarchical probability logic in the final model

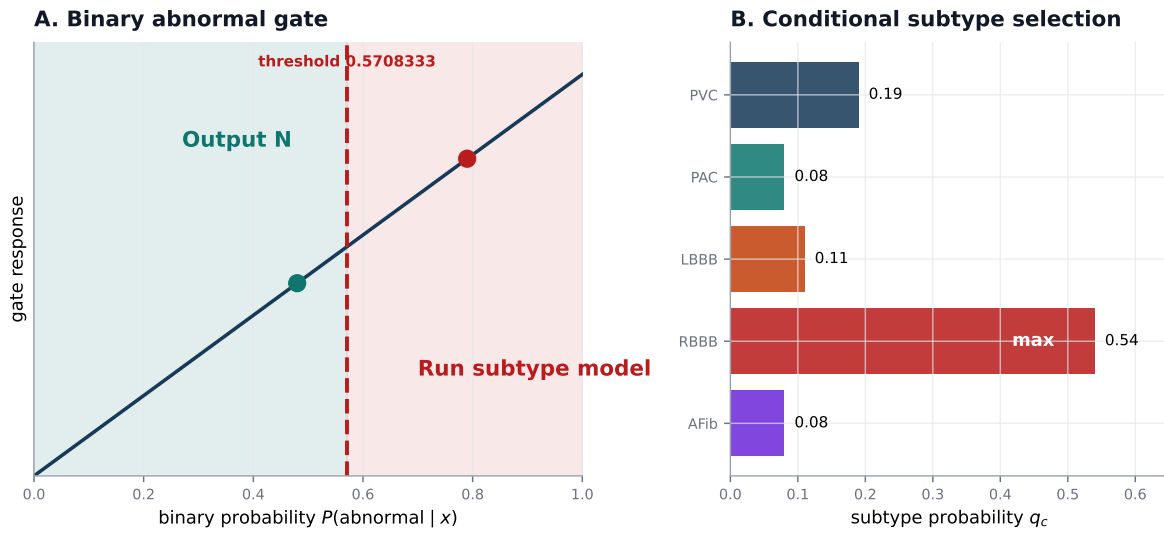


Figure 4.6: Probability logic of the final hierarchy. The subtype model is conditional: it is used only after the abnormality gate is passed.

For beat i , the binary detector returns an abnormal score p_i from its 242-value feature vector. This score estimates abnormal-class membership but has not been independently calibrated as a clinical probability. The final threshold rule is

$$a_i = \mathbb{I}[p_i > 0.5708333]. \quad (4.16)$$

If $a_i = 0$, the output is N. If $a_i = 1$, the subtype model selects the highest probability among PVC, PAC, LBBS, RBBB, and AFib:

$$\hat{y}_i = \arg \max_{c \in \{1, \dots, 5\}} q_{ic}. \quad (4.17)$$

Here the maximisation already returns an index 1–5. Unsupported abnormal symbols contribute to binary testing and are excluded from subtype training and six-class scoring. At inference, every gated beat receives a supported label. There is no validated unknown-class rejection mechanism.

The tree output is an ensemble score obtained from training-label proportions in reached leaves. It should not be read as a clinically calibrated disease probability. In particular, the subtype score q_{ic} is conditional on taking the abnormal branch; a large q_{ic} does not repair a missed binary gate. This is why the report evaluates the binary detector, the conditional subtype model, and the complete six-class decision separately.

4.7 Why Extra Trees fits this feature design

The final 242-value vector mixes pooled waveform values, derivatives, spectral power, signal statistics, RR timing, and autoencoder residuals. Their relationship to class labels is non-linear. Extra Trees fits this setting by building many randomised decision trees and averaging their probability outputs [32].

At a decision-tree node S , let w_i be the training weight of beat i . The weighted fraction of class c and its entropy are

$$p_c(S) = \frac{\sum_{i \in S} w_i \mathbb{1}[y_i = c]}{\sum_{i \in S} w_i}, \quad H(S) = - \sum_c p_c(S) \log_2 p_c(S), \quad (4.18)$$

with $0 \log_2 0$ defined as zero. Entropy is high when weighted classes are mixed and zero when a node contains only one class. A proposed split sends rows to left and right child nodes. Its weighted impurity decrease is

$$\Delta H = H(S) - \frac{W_L}{W_S} H(S_L) - \frac{W_R}{W_S} H(S_R), \quad W_A = \sum_{i \in A} w_i. \quad (4.19)$$

For an unweighted toy node with four normal and four abnormal beats, entropy is one bit. A split putting all normals left and all abnormals right reduces the two child entropies to zero, giving $\Delta H = 1$ bit. Real nodes usually give a smaller decrease. The tree does not take gradient steps: it selects a split, partitions the rows, and repeats recursively. Extra Trees randomly selects candidate features and thresholds at each node, then chooses the best candidate under this criterion [32].

Each tree ends with leaf nodes. A beat follows feature-threshold questions from root to leaf, and the leaf's class mixture supplies a score. The ensemble averages these scores over its trees. Random candidate splits make individual trees differ; averaging reduces the instability of any one tree. The binary model contains 240 trees and the subtype model contains 260 trees. Tree count describes ensemble size, whereas an epoch describes one complete pass through training data in neural-network fitting.

This theory also explains why Extra Trees is suitable for the final data scale. MITDB contributes many beats but only 45 subject clusters. A very high-capacity end-to-end neural classifier could learn patient identity instead of robust ECG rules [8]. The Extra Trees hierarchy

is still high-capacity, but it uses transparent tabular evidence and is easier to audit against the final temporal protocol. This does not mean that tree ensembles are always better than deep learning.

4.7.1 How the two supervised models are fitted

The binary model receives every eligible training beat, including abnormal beats whose detailed symbol is outside the five supported subtype groups. Its target is $y_i^{(B)} \in \{0, 1\}$, meaning normal or abnormal. The subtype model receives only supported abnormal training beats and has target $y_i^{(C)} \in \{\text{PVC, PAC, LBBB, RBBB, AFib}\}$. Both models use the same 242 measurements: 223 morphology, spectral, and statistical values, 15 RR values, and 4 frozen-autoencoder residuals. The labels determine fitting targets and validation scores; they do not enter this input vector.

The binary candidates vary tree count, maximum depth, leaf size, and record weighting power. The selected candidate has 240 fully grown trees, at least one sample per leaf, entropy splits, and approximately $\sqrt{242}$ candidate features at a split. The subtype candidates vary similar settings; the selected model has 260 fully grown trees and samples approximately one third of the 242 features at each split. These settings were chosen from the earlier validation region, then the selected structures were refitted using all eligible development beats. The subtype refit still excludes N and unsupported abnormal categories.

The binary model averages its trees' abnormal scores to obtain p_i . Validation tries the distinct output scores as thresholds (along with 0, 0.5, and 1). The primary ranking is balanced accuracy, followed by F1 and further tie-breaks. The selected threshold is 0.5708333. A threshold is an *operating point*: lowering it generally catches more abnormal beats but also increases false alarms. It is not a weight learned by backpropagation. The subtype candidates are ranked by validation macro F1, then accuracy. Figure 6.3 plots those measured validation comparisons.

4.8 Class and patient imbalance theory

ECG beat datasets are imbalanced in two ways. Some classes are rare, and some patients contribute many more beats than others. Without weighting, a model can mostly learn common normal beats or a few large records.

The final training code uses record-aware class weights. For class c in subject cluster r , the weight is proportional to

$$w_{cr} = \frac{1}{n_c} \left(\frac{n_c}{R_c n_{cr}} \right)^\alpha, \quad (4.20)$$

where n_c is total support for class c , R_c is the number of subject clusters containing class c , and n_{cr} is the number of class- c beats in subject cluster r (records 201 and 202 are combined). Weights are normalised to mean one and clipped to $[0.05, 20]$. The first factor supports class balancing; the second factor reduces domination by a single record. The exponent α controls

how strongly record balancing is applied and is selected in validation. Class-balanced learning is important when rare classes would otherwise have little influence [22].

4.9 Temporal patient adaptation theory

ECG morphology contains patient-specific information such as body geometry, electrode placement, baseline QRS shape, conduction pattern, and background rhythm. If earlier labelled ECG from a patient is available during development, the model can adapt to that patient's baseline. This shared patient information can make classification easier than it would be for a patient whose ECG was absent from development.

This is the theory behind the final temporal-holdout design. It is useful for monitoring after a labelled calibration period, but it changes the claim. The final result is a known-patient temporal-continuation result. It should not be reported as unseen-patient generalisation unless a patient-disjoint experiment is run. The 16-beat boundary gap reduces direct overlap at the split boundary, but it does not remove shared patient identity.

4.9.1 Why training, validation, and testing have different jobs

Training data changes model parameters: neural weights for the autoencoder, and split rules and leaf scores for each tree ensemble. Validation data chooses among checkpoints, tree settings, and the binary operating threshold. The temporal test then estimates performance after those choices are fixed. If test labels were used to choose the best threshold, its reported score would be optimistically biased. In this final rerun, the test did not select the threshold or candidate, but its labels had been seen during earlier development. The result is therefore a retrospective known-patient estimate, not a fresh confirmatory test. Figure 6.1 shows the chronological allocation and boundary gaps.

4.10 From saved models to a live ECG decision

The implemented workflow joins signal acquisition, digital processing, model inference, and reporting. An AD8232 front end senses a single ECG channel; the Arduino demonstration sketch samples its analog output at approximately 128 Hz and reports lead-off status [33]. The demonstration hardware does not itself certify that its electrode placement equals the MITDB MLII configuration. Database replay instead starts with labelled MITDB MLII data. Both paths must provide the expected sampling rate and one usable channel before the trained inference pipeline can run.

The live analyser keeps a rolling signal buffer, searches for R peaks, and waits until the beat window and the following RR interval are observed. It then applies the local preparation in Equation 4.1, builds the 242-value vector, and loads the frozen model bundle. With detected peaks at sample indices s_i , the interval in seconds is

$$RR_i = \frac{s_i - s_{i-1}}{128}. \quad (4.21)$$

Peak-detection mistakes change both alignment and RR evidence; the offline test used reference annotations for alignment, so live results may differ. The delay follows from the required input: the window includes 312 samples starting at the R peak (about 2.44 seconds), and the next R peak must also be observed. The larger of those requirements determines when a beat is ready to score.

Table 4.1: Technologies in the final evidence path and the mathematical task performed by each. The web interface displays results and does not train the models.

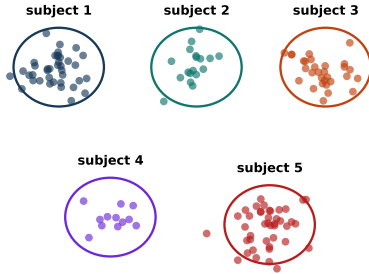
Technology	Role in the implemented system
AD8232 and Arduino	Analog acquisition, lead-off indication, and approximate 128 Hz sample timing in the sensor demonstration
WFDB and PhysioNet	Read ECG samples, lead headers, beat symbols, and rhythm annotations for reproducible research datasets
NumPy and SciPy	Arrays, resampling, Butterworth filtering, peak detection, Fourier spectrum, and numerical feature calculations [27]
PyTorch	Convolutional autoencoder, automatic differentiation, mini-batch MSE, and Adam weight updates [34]
scikit-learn	Weighted Extra Trees fitting, ensemble scores, and validation metrics
Backend and interface	Load saved checkpoints, build live features, apply the fixed gate and subtype rule, and display beat results

4.11 Validation theory

Beat-level accuracy alone is not enough. If one patient contributes 2,000 beats, those beats are related. They are not equivalent to 2,000 independent patients. The final validation therefore reports beat metrics and also uses subject-clustered uncertainty.

Why validation must respect subject clusters

A. Many beats, but fewer independent subjects



B. Bootstrap resamples subjects, not individual beats

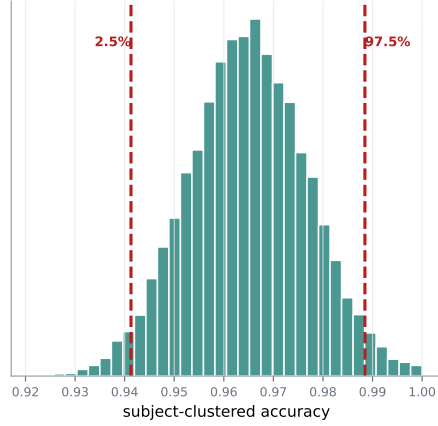


Figure 4.7: Theoretical reason for subject-clustered validation. Clustered resampling prevents one long record from being treated as many independent people.

Balanced accuracy gives equal weight to class recalls, so normal beats cannot hide poor abnormal recall. Macro F1 gives each class equal importance, so a failure on a rare class is visible. Precision–recall style measures are also important under class imbalance [21]. For confidence intervals, the final experiment resamples complete subject clusters rather than individual beats, following the general diagnostic-statistics principle that clustered data need clustered analysis [23].

For binary detection, TP , TN , FP , and FN denote true positives, true negatives, false positives, and false negatives. Sensitivity is $TP/(TP + FN)$ and specificity is $TN/(TN + FP)$. Balanced accuracy is their mean. This gives equal importance to recognising abnormal and normal beats even when one group is larger. For a class c , precision asks what fraction of predicted c is truly c ; recall asks what fraction of true c is found. Their harmonic mean is

$$F1_c = \frac{2 \text{Precision}_c \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}, \quad \text{MacroF1} = \frac{1}{C} \sum_{c=1}^C F1_c. \quad (4.22)$$

A confusion matrix counts every true/predicted class pair, making a high overall accuracy easier to interpret. The final test also computes equal-subject uncertainty by resampling complete patient clusters and recalculating the statistic. This treats patients, rather than individual beats, as the independent units for the interval. It cannot correct for the earlier observation of the test set or demonstrate generalisation to new patients.

4.12 Summary of the theoretical framework

Table 4.2: How the theory maps to the implemented final system.

Theory concept	Implemented in the final system
Beat-scale morphology	512-sample MLII window with pooled full-window, central, and derivative features
Rhythm timing	15 RR features; live scoring waits until next-RR context is available
Normal-only reconstruction	frozen NSRDB autoencoder supplies four residual features
Hierarchical probability	240-tree binary gate first; 260-tree subtype model only for gated abnormal beats
Class/patient imbalance	record-aware class weighting with validation-selected record-power parameter
Temporal adaptation	earlier MLII timeline trains/selects the model; final 20% of accepted beats is tested
Subject-aware uncertainty	subject-clustered bootstrap over 45 clusters, with records 201 and 202 treated as one cluster

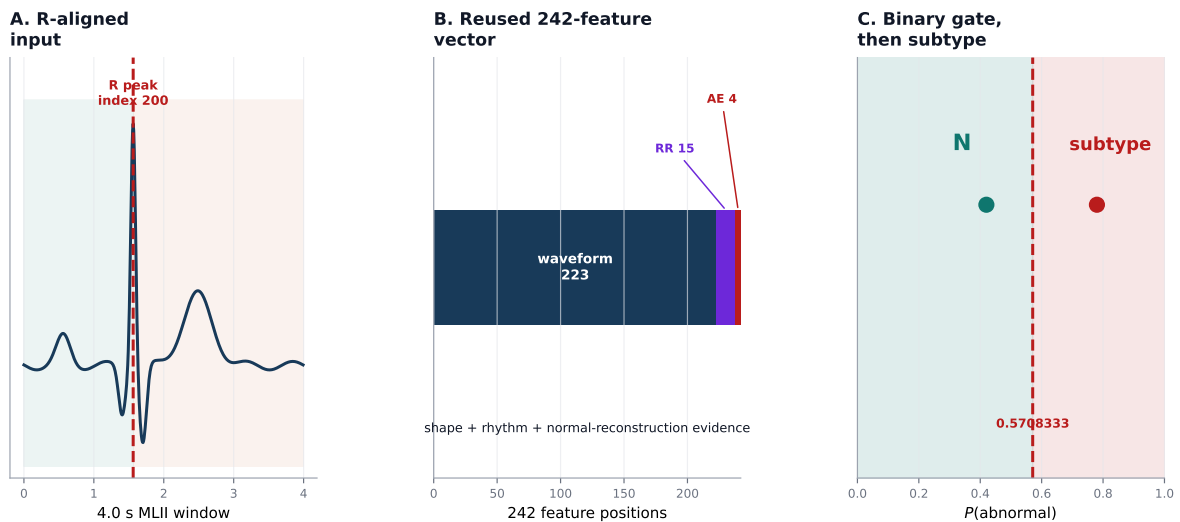
5 Methodology

This chapter presents the computational methodology, model training, and prediction procedure implemented in the ECG prototype. The system uses one MLII channel and three trained components: a normal-reference autoencoder, a binary abnormality detector, and an abnormal subtype classifier. The two supervised models use the same 242-value feature vector. All model parameters remain fixed during prediction. Section 5.10 connects the individual calculations into a mathematical account of fitting each model and applying the saved hierarchy to previously unscored ECG. Chapter 6 gives the experimental protocol and evaluation measures.

5.1 System overview

For each detected MLII heartbeat, the system takes a 512-sample window aligned to the R peak, builds waveform, rhythm, and normal-reconstruction features, sends those features to a binary abnormality gate, and sends only gated abnormal beats to a five-class subtype classifier.

Final ECG system: from one MLII beat to one decision



The autoencoder supplies four residual features. It does not make the final class decision alone.

Figure 5.1: Exact runtime order of the final ECG system. The autoencoder supplies features; the supervised tree models make the final class decision.

5.2 Input signal and beat window

The deployed system accepts one ECG channel. For database replay and final testing that channel is MIT-BIH MLII, selected by signal name. Records 102 and 104 are excluded because they do not contain MLII. For sensor sessions, the API requires the user to declare `lead_name='MLII'` and rejects other declared leads. This rule checks the declared lead against the supervised training signal. It cannot verify the physical electrode placement.

The classifier assigns one output to each eligible beat. The live backend collects MLII samples, detects R peaks, and waits until enough rhythm context is available. For a scorable beat, it extracts a fixed 512-sample window at 128 Hz:

- 200 samples before the R peak;
- 312 samples starting at the R peak;
- total duration of 4.0 seconds.

This same window geometry is used in the final temporal-holdout experiment and in the installed session pipeline.

Let s_i be the sample index of the current R peak in the 128 Hz signal. The extracted beat is the ordered vector

$$\mathbf{u}^{(i)} = (v_{s_i-200}, v_{s_i-199}, \dots, v_{s_i+311}) \in \mathbb{R}^{512}. \quad (5.1)$$

Filtering and per-window standardisation produce the model input $\mathbf{x}^{(i)}$. For a filtered window u , this scaling is

$$x_t = \frac{u_t - \mu_u}{\sigma_u + 10^{-8}}, \quad \mu_u = \frac{1}{512} \sum_{t=1}^{512} u_t, \quad \sigma_u = \sqrt{\frac{1}{512} \sum_{t=1}^{512} (u_t - \mu_u)^2}. \quad (5.2)$$

The subtraction removes a local voltage offset and the division reduces recording-gain differences. The model still sees relative waveform shape. Each training and live beat is prepared from its own window, so preparing an earlier beat does not require statistics from a later test segment.

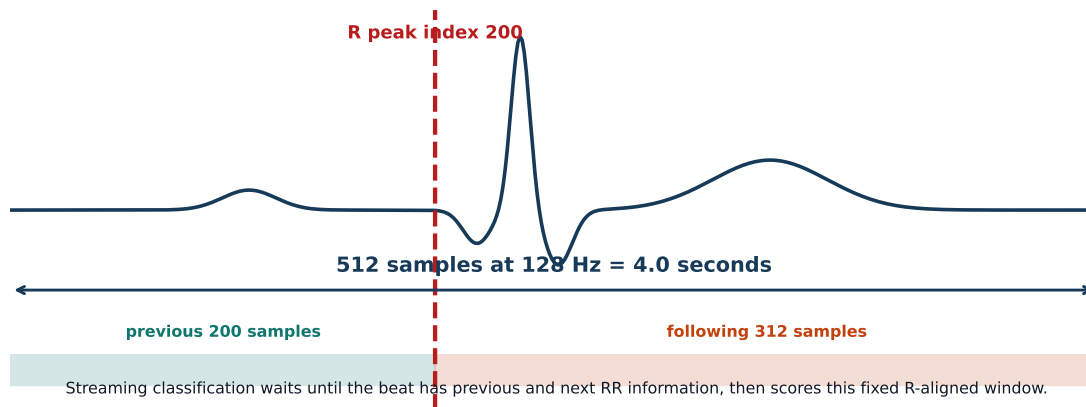


Figure 5.2: R-peak-aligned beat window used by the final model. The R peak is at sample index 200, giving the model pre-QRS context and post-QRS recovery information.

5.3 Three trained components

The final system uses three trained components, stored as an autoencoder checkpoint and a separate hierarchy bundle:

1. a frozen normal-only autoencoder;
2. a binary Extra Trees abnormality detector;
3. a five-class Extra Trees abnormal-subtype classifier.

Table 5.1: Architecture, training target, and output of each fitted component.

Component	Input and architecture	What fitting learns	Output used next
Normal autoencoder	One [1, 512] ECG window; convolutional encoder, dilated bottleneck, decoder	Weights that reconstruct clean NSRDB windows from noisy inputs	Reconstructed waveform; four residual features
Binary detector	One 242-value feature row; 240 Extra Trees	Weighted normal/abnormal split rules and leaf scores	Abnormal score p_i compared with the fixed threshold
Subtype classifier	The same 242-value row; 260 Extra Trees	Weighted split rules among five supported abnormal classes	Five class scores; highest score becomes the subtype

The autoencoder is trained first and then frozen. The two supervised tree models are then fitted on MLII features from the temporal development portion of MIT–BIH Arrhythmia Database records.

A Offline training produces three fixed components

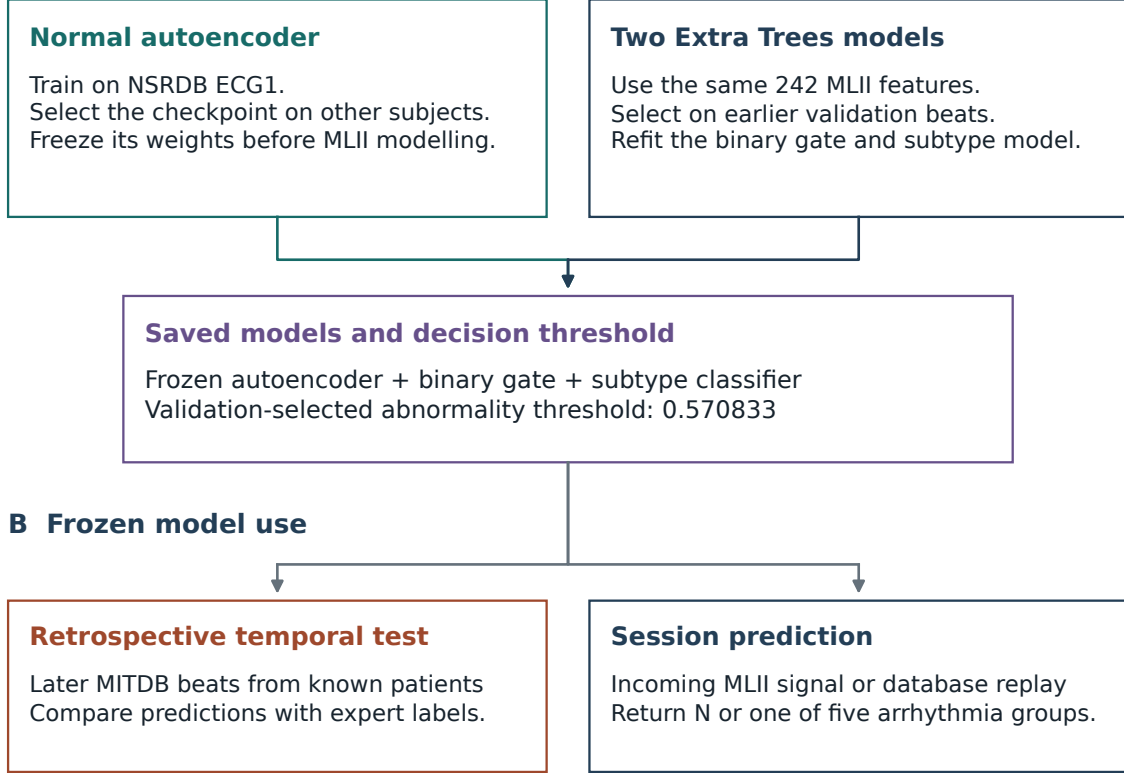


Figure 5.3: Training and use of the saved three-component model. Offline development produces the frozen autoencoder, binary gate, subtype classifier, and validation-selected threshold. The same fixed components support retrospective temporal evaluation and session prediction; session use does not update its parameters.

5.4 Normal-only autoencoder

5.4.1 Purpose

The autoencoder learns to reconstruct normal-reference ECG windows. Its reconstruction errors provide four input features for the supervised models. The final classification depends on the combined feature vector.

For an input window $x \in \mathbb{R}^{512}$, the autoencoder output is

$$\hat{x} = g_{\phi}(f_{\theta}(x)), \quad (5.3)$$

where f_{θ} is the encoder and g_{ϕ} is the decoder. Training uses a denoising objective: Gaussian noise with standard deviation 0.03 is added to the input, but the target remains the clean

standardised window. For a mini-batch $\mathcal{B}$, write

$$\tilde{x}^{(i)} = x^{(i)} + \epsilon^{(i)}, \quad \epsilon_t^{(i)} \sim \mathcal{N}(0, 0.03^2), \quad \hat{x}^{(i)} = g_\phi(f_\theta(\tilde{x}^{(i)})). \quad (5.4)$$

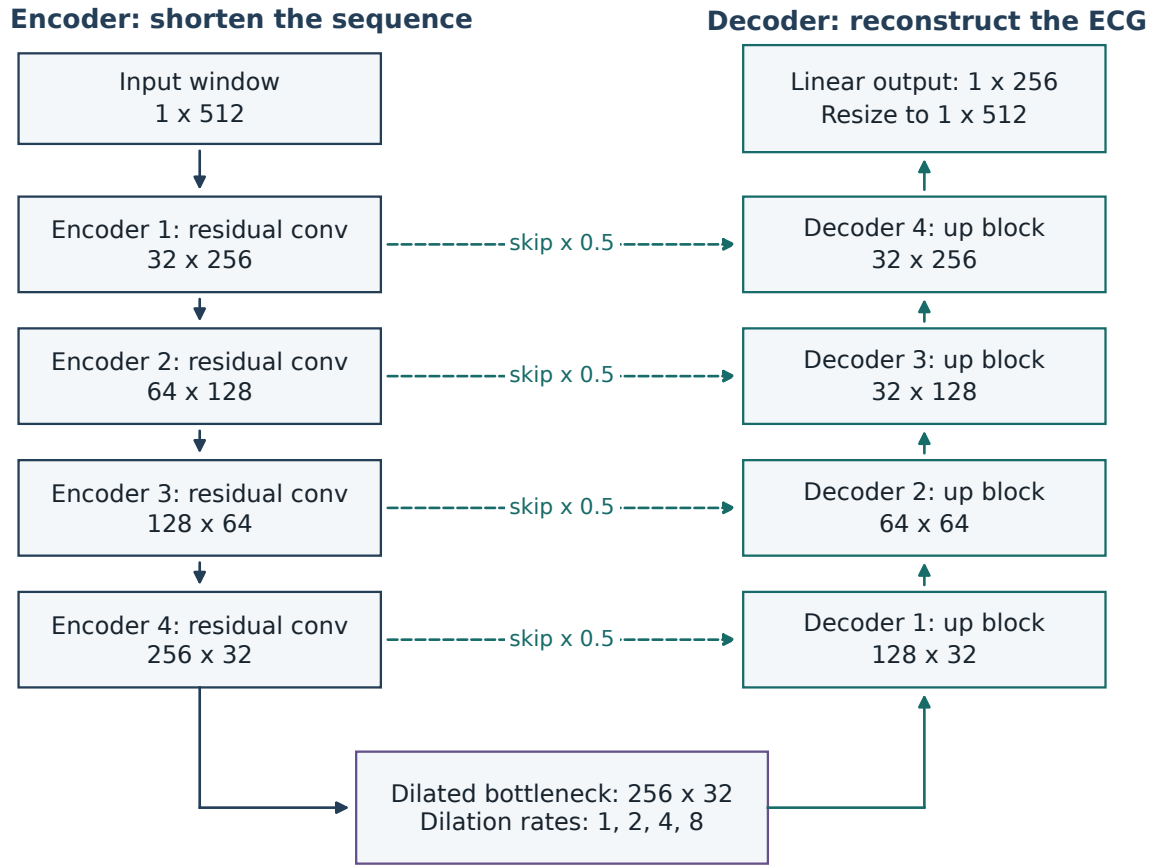
The exact training objective is the mean squared difference from the *clean* target across samples and windows:

$$\mathcal{L}_{AE}(\theta, \phi) = \frac{1}{512|\mathcal{B}|} \sum_{i \in \mathcal{B}} \sum_{t=1}^{512} (x_t^{(i)} - \hat{x}_t^{(i)})^2. \quad (5.5)$$

The network receives noisy input while the loss compares against clean input. It must recover repeatable ECG structure rather than copy the added noise [15]. At prediction time no artificial noise is added: the same network reconstructs the prepared MLI beat x directly.

As a small illustration, if a clean three-sample target is $(0, 1, 0)$ and a reconstruction is $(0.1, 0.8, -0.1)$, the three squared errors are 0.01, 0.04, 0.01 and their mean is 0.02. The real loss applies the same arithmetic to every one of the 512 samples in every batch window.

5.4.2 Network structure



Dimensions are channels x time samples; the batch dimension is omitted.

Dashed arrows carry encoder features to matching decoder lengths.

Each up block resizes to its skip length; final interpolation restores 512 samples.

Figure 5.4: Autoencoder architecture with tensor dimensions verified by a forward pass through the 2,680,641-parameter implementation. Read down the encoder, through the bottleneck, and up the decoder. Dashed connections concatenate encoder features scaled by 0.5; decoder outputs match their skip lengths before the final interpolation restores the 512-sample waveform.

Each residual encoder block contains two kernel-seven convolutions, Group Normalisation, LeakyReLU, dropout, and a residual projection. Four kernel-three bottleneck blocks use dilation 1, 2, 4, and 8. Dilation increases the receptive field without deleting time samples. The decoder uses transposed convolutions and skip connections scaled by 0.5. Each up block resizes its output to match the corresponding encoder skip. The first up block therefore returns length 32, the fourth returns length 256, and final linear interpolation restores length 512. The figure reports these actual tensor dimensions. The final layer is linear because the standardised ECG signal contains positive and negative values.

5.4.3 Encoder block

A one-dimensional convolution applies learned filters along time. For input channels c , output channel j , kernel offset m , and time t , a simplified layer operation is

$$h_{j,t} = b_j + \sum_c \sum_{m=0}^{K-1} W_{j,c,m} x_{c,t+m}. \quad (5.6)$$

The learned coefficients W detect local patterns such as slopes and peaks. Each encoder block uses a kernel width $K = 7$ and stride two in its first convolution. With input length L , padding p , dilation d , and stride s , the output length is

$$L_{\text{out}} = \left\lfloor \frac{L + 2p - d(K - 1) - 1}{s} + 1 \right\rfloor. \quad (5.7)$$

Here $p = 3$, $d = 1$, and $s = 2$, giving $512 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 32$ time positions. The corresponding channel counts are $1 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$. The tensor at the deepest encoder stage is $[B, 256, 32]$, where B is the batch size. This is a 16-fold reduction in the *time axis*; 256 channels and the skip connections mean it should not be interpreted as a 16-fold reduction in total stored information.

Within a block, Group Normalisation standardises activations within eight channel groups. LeakyReLU then applies

$$\text{LeakyReLU}(a) = \begin{cases} a, & a \geq 0, \\ 0.1a, & a < 0. \end{cases} \quad (5.8)$$

Negative internal activations therefore retain a small gradient. A learned 1×1 projection makes the input shape match the block output before they are added. In abbreviated form the residual computation is

$$h_{\text{out}} = \text{Dropout}_{0.2}[\text{LeakyReLU}(F(h_{\text{in}}) + P(h_{\text{in}}))], \quad (5.9)$$

where F is the two-convolution path and P is the identity or projection path. Adding $P(h_{\text{in}})$ lets information and gradients travel through a shorter route. Dropout randomly masks internal activations during fitting; it is disabled during validation and prediction [28, 30].

5.4.4 Bottleneck and decoder mathematics

Four bottleneck convolutions use kernel width three and dilation $d \in \{1, 2, 4, 8\}$. A dilated convolution reads positions $t - d, t, t + d$ instead of only $t - 1, t, t + 1$. It therefore combines progressively wider temporal context without further shrinking the 32-position axis. Each bottleneck operation adds its input back to its output.

The decoder uses transposed convolutions to increase temporal resolution. At each stage it also concatenates the corresponding encoder output e_j with a scale factor of 0.5:

$$d_j = U_j(\text{concat}[\text{resize}(\text{up}(d_{j-1}), |e_j|), 0.5e_j]). \quad (5.10)$$

U_j denotes the following convolution, normalisation, activation, and dropout operations. The resize matches the skip tensor's actual time length. In this implementation the four decoder stages end at lengths 32, 64, 128, and 256; linear interpolation restores length 512 before returning the waveform. The final 1×1 convolution produces one output channel and has no sigmoid or other output activation. These details explain how local normal structure can be reconstructed while preserving the input shape.

5.4.5 Normal-reference data and checkpoint

The autoencoder checkpoint was developed from complete recordings of all 18 subjects in the MIT-BIH Normal Sinus Rhythm Database. Fifteen subjects supply 288,673 fitting windows and three different subjects supply 57,177 validation windows. The selected NSRDB header channel is ECG1. The database does not document this channel as anatomical Lead II, so the report treats it as a normal ECG reference channel rather than renaming it MLII [6].

The final checkpoint uses Adam, learning rate 0.001, batch size 256, and two epochs. Validation MSE changed from 0.061195 after epoch 1 to the best saved value, 0.061193, after epoch 2. The autoencoder artifact also stores a normal-residual threshold of 0.158893, but the final hierarchy does not use that threshold as the final arrhythmia rule.

5.4.6 How the training update changes network weights

For one predicted sample, the derivative of Equation 5.5 is

$$\frac{\partial \mathcal{L}_{AE}}{\partial \hat{x}_t^{(i)}} = \frac{2(\hat{x}_t^{(i)} - x_t^{(i)})}{512|\mathcal{B}|}. \quad (5.11)$$

An output greater than its clean target has a positive derivative; an output below the target has a negative derivative. Backpropagation applies the chain rule through the decoder, bottleneck, and encoder to obtain derivatives for all learnable filters and biases. If ψ collects those parameters and $g_k = \nabla_{\psi} \mathcal{L}_{AE}$ is the gradient at training step k , Adam uses running estimates of the gradient and its squared magnitude:

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k, \quad v_k = \beta_2 v_{k-1} + (1 - \beta_2) g_k^2, \quad \psi_{k+1} = \psi_k - \eta \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \varepsilon}. \quad (5.12)$$

Here $\eta = 0.001$, the hats are bias-corrected running estimates, and the division is elementwise. This is how fitting loss can decrease across mini-batches. Validation windows receive no weight update; their MSE is used only to select the saved epoch [31].

The recorded fitting MSE fell from 0.057639 to 0.041976, but the clean-input subject-disjoint validation MSE changed only from 0.061195 to 0.061193. Because training uses noisy inputs and validation uses clean inputs, their levels are not directly comparable as an overfitting measure. The second epoch is retained because it has the lowest measured validation loss, not because the much larger fitting-loss decrease proves improved generalisation.

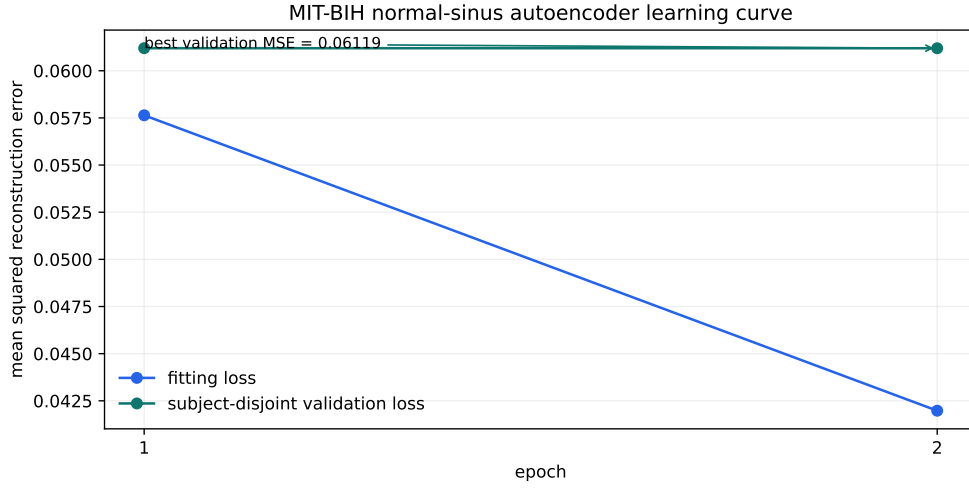


Figure 5.5: Autoencoder fitting and subject-disjoint validation loss. The checkpoint with the lowest validation loss is retained before the supervised MLII stages are trained.

5.4.7 Four autoencoder residual features

Let $e_t = x_t - \hat{x}_t$ be the reconstruction residual. The final feature system uses four scalar residual summaries:

$$f_1 = \frac{1}{512} \sum_t |e_t| \quad \text{whole-window MAE,} \quad (5.13)$$

$$f_2 = \frac{1}{512} \sum_t e_t^2 \quad \text{whole-window MSE,} \quad (5.14)$$

$$f_3 = \frac{1}{240} \sum_{t=105}^{344} |e_t| \quad \text{central morphology MAE,} \quad (5.15)$$

$$f_4 = \frac{1}{511} \sum_{t=2}^{512} |e_t - e_{t-1}| \quad \text{residual-change MAE.} \quad (5.16)$$

These equations use one-based sample indices. The central range corresponds to the zero-based code slice 104 : 344 and covers the QRS region and nearby waveform. The residual-change feature becomes large when the reconstruction misses sharp slopes. These four values are appended to the transparent MLII and RR features.

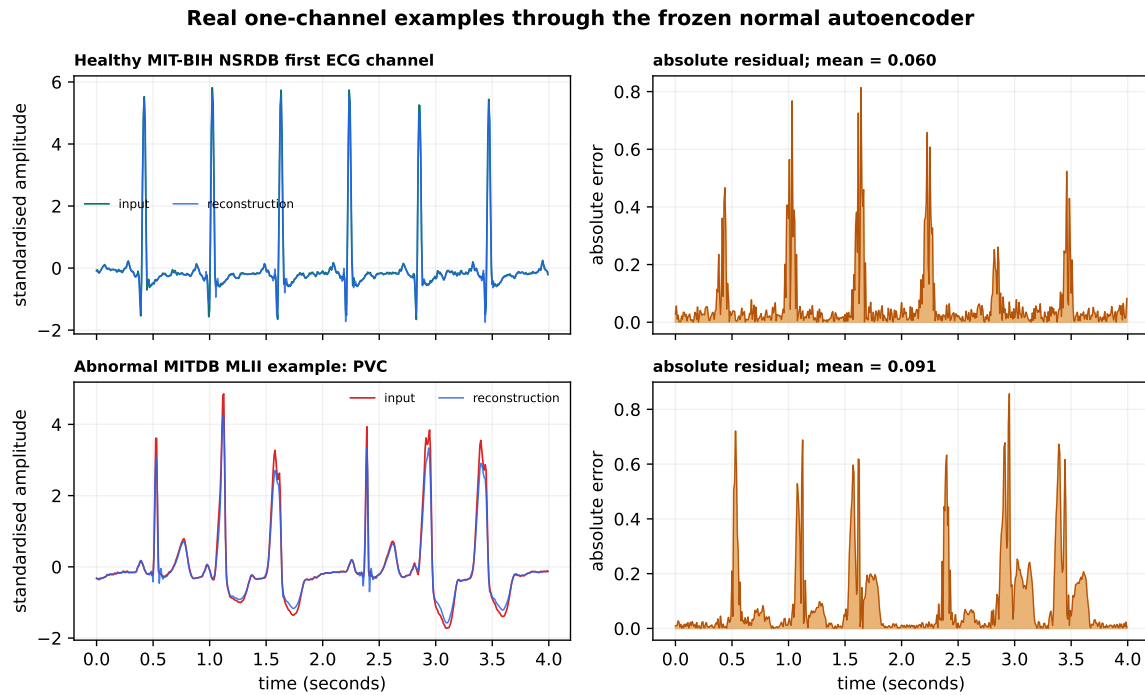


Figure 5.6: Real healthy and abnormal ECG examples through the frozen autoencoder. The residual between input and reconstruction is converted into four numerical features for the supervised detector.

5.5 Final 242-value feature vector

The saved final bundle declares 242 total features, 15 RR features, required autoencoder features, and four autoencoder residual values. Therefore every supervised prediction uses:

$$\begin{aligned}
 &223 \text{ waveform/spectrum/statistics values} \\
 &\quad + 15 \text{ RR timing values} \\
 &\quad + 4 \text{ autoencoder residual values} = 242.
 \end{aligned} \tag{5.17}$$

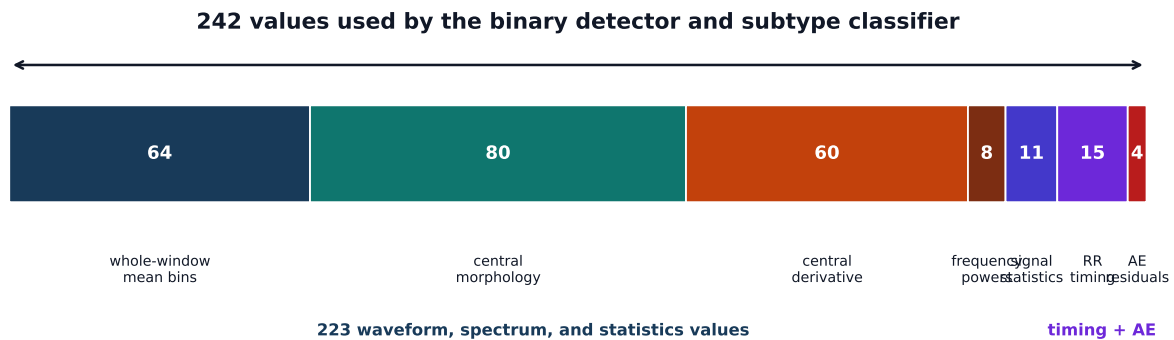


Figure 5.7: Feature-vector layout used by both supervised models. The same 242-value vector is passed first to the abnormality detector and, if required, to the subtype classifier.

Table 5.2: Organised feature groups in the final architecture.

Group	Values	What it tells the model
Whole-window mean bins	64	broad four-second MLII waveform shape
Central morphology bins	80	P–QRS–T shape around the R peak
Central derivative bins	60	sharp slopes and rapid QRS changes
Frequency-band powers	8	relative energy from 0.5 to 50 Hz
Signal statistics	11	scale, range, energy, skewness, and kurtosis
RR timing context	15	previous/next interval, local rate, variability, median, interquartile range, MAD, RMSSD, pNN50, range ratio, trend, and latest RR change
Autoencoder residuals	4	how strongly the beat disagrees with the learned normal-reconstruction pattern

The final experiment constructs the same 242 values in two steps: first 230 values are built from 223 MLII waveform/statistical values plus seven base RR values; then eight causal RR-history values and four autoencoder residual values are appended. During live/session prediction, the backend builds the same final vector directly from 223 MLII waveform/statistical values, 15 RR values, and four autoencoder residual values.

5.6 How an Extra Trees model learns

Both supervised stages use the same type of architecture: an ensemble of *extremely randomised decision trees*. A tree asks a sequence of questions of the form “is feature $z_j \leq \tau$?” The answer sends a beat to a left or right child node. Randomly chosen candidate features and split values make different trees learn different partitions; among the candidate splits at a node, the algorithm chooses the one that best separates the weighted training labels [32].

5.6.1 Why some training rows have more weight

Rare arrhythmia classes and short patient records would otherwise contribute little to split decisions. For class c in subject cluster r , the final fitting code starts with a weight proportional to

$$w_{cr} = \frac{1}{n_c} \left(\frac{n_c}{R_c n_{cr}} \right)^\alpha, \quad (5.18)$$

where n_c is the total number of training beats of class c , R_c is the number of subject clusters containing c , and n_{cr} is the number of class- c beats in cluster r . Weights are divided by their overall mean and clipped to $[0.05, 20]$. The first factor increases the influence of rare classes. The second limits dominance by a subject that contributes many examples of the same class. The chosen exponent is $\alpha = 0.35$ for the binary model and $\alpha = 0.25$ for the subtype model. This weighting changes the tree's splitting calculation; it does not replicate beats or create new patient data.

5.6.2 The mathematical question at each tree node

Let S be the training beats that have reached a node, w_i the weight of beat i , and y_i its training label. The weighted share of class c is

$$P_c(S) = \frac{\sum_{i \in S} w_i \mathbf{1}[y_i = c]}{\sum_{i \in S} w_i}. \quad (5.19)$$

The node's entropy measures how mixed its labels are:

$$H(S) = - \sum_c P_c(S) \log_2 P_c(S), \quad 0 \log_2 0 := 0. \quad (5.20)$$

A pure node has entropy zero. If a proposed feature threshold divides S into left and right subsets S_L, S_R , its weighted entropy reduction is

$$\Delta H = H(S) - \frac{W_L}{W_S} H(S_L) - \frac{W_R}{W_S} H(S_R), \quad W_A = \sum_{i \in A} w_i. \quad (5.21)$$

The selected random candidate has the largest available reduction. A tree repeats this at its child nodes until its stopping conditions are reached. The selected final models have no fixed maximum depth and permit a leaf with one training beat; this makes them flexible, while ensemble averaging and the separate validation region help control the resulting instability.

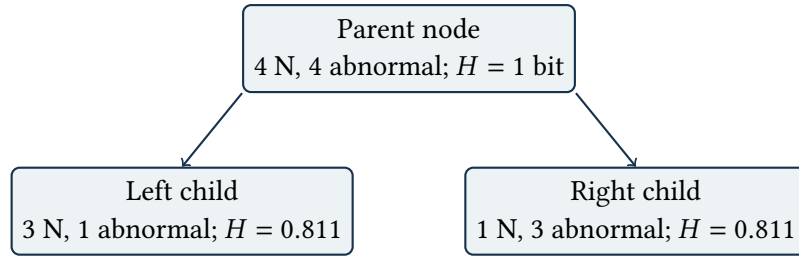


Figure 5.8: Illustrative unweighted binary split, separate from the actual experiment. Each child contains four beats, so Equation 5.21 gives $\Delta H = 1 - \frac{1}{2}(0.811) - \frac{1}{2}(0.811) = 0.189$ bit. A candidate placing two N and two abnormal beats in each child would give zero gain. Real fitting uses the sample weights above.

5.6.3 From a tree leaf to an ensemble score

Once a tree is fitted, a new feature row follows its learned questions to a leaf $L_b(z)$. The score for class c in tree b is the weighted share of training labels of class c in that leaf, $P_c(L_b(z))$. With B trees, the forest output is

$$Q_c(z) = \frac{1}{B} \sum_{b=1}^B P_c(L_b(z)). \quad (5.22)$$

This is an average of leaf scores, not a neural-network softmax. It can rank classes, but the final bundle contains no probability-calibration model; a score such as 0.8 should not be interpreted as an independently validated 80% clinical disease probability. Trees lower node impurity while they are grown. They do not have epochs, backpropagation, or a neural loss curve.

5.7 Binary abnormality detector

The first supervised model is the abnormality detector. It receives all 242 features and returns an abnormal-class score p_i for beat i . The selected detector is an Extra Trees classifier with:

- 240 trees;
- entropy split criterion;
- fully grown trees;
- minimum leaf size of one sample;
- square-root feature sampling at each split;
- record/class weighting with record-power $\alpha = 0.35$.

Its target is $y_i^{(B)} \in \{0, 1\}$, with zero for project-normal N and one for abnormal. This stage uses every eligible binary-labelled beat, including abnormal symbols for which the project has no supported subtype. Three candidate forest settings were trained on 66,285 earlier

beats and compared on 16,029 later validation beats. The selected structure was then refitted on 83,050 development beats. At each node the square-root feature rule uses $\lfloor \sqrt{242} \rfloor = 15$ candidate feature positions before selecting a random candidate threshold with the best available weighted entropy decrease. Different nodes and trees can therefore use different combinations of morphology, timing, and reconstruction evidence.

For tree b , let $p_b(z_i)$ be the abnormal weighted class share in the leaf reached by feature row z_i . The ensemble abnormal score is

$$p_i = \frac{1}{240} \sum_{b=1}^{240} p_b(z_i). \quad (5.23)$$

For example, if 180 trees returned an abnormal leaf score of one and 60 returned zero, the ensemble score would be $180/240 = 0.75$. Real leaves can also contain both classes, giving values between zero and one.

The decision threshold was selected on the 64–80% validation region of each development record. For each candidate threshold τ , sensitivity and specificity are calculated from validation labels, and their mean is the balanced accuracy:

$$\text{BA}(\tau) = \frac{1}{2} \left[\frac{TP(\tau)}{TP(\tau) + FN(\tau)} + \frac{TN(\tau)}{TN(\tau) + FP(\tau)} \right]. \quad (5.24)$$

The code ranks thresholds by balanced accuracy, followed by F1 and further tie-breaks. The selected fixed threshold is $\tau = 0.5708333$:

$$a_i = \mathbb{I}[p_i > \tau], \quad \tau = 0.5708333. \quad (5.25)$$

If $a_i = 0$, the final output is normal class N. No subtype classifier is used for that beat. The threshold controls the balance between missed abnormal beats and false alarms; it is selected after tree fitting, not learned by a gradient update.

5.8 Abnormal-subtype classifier

The second supervised model is used only after the binary gate marks a beat as abnormal. It is fitted only on supported abnormal classes 1–5, not on N and not on unsupported MIT–BIH symbols. The selected subtype model is an Extra Trees classifier with:

- 260 trees;
- entropy split criterion;
- fully grown trees;
- minimum leaf size of one sample;
- feature-sampling fraction 0.33 at each split;
- record/class weighting with record-power $\alpha = 0.25$.

This model’s target is one of five supported abnormal labels. It has no normal output. Its three candidate structures were fitted on 22,844 supported abnormal candidate beats and compared on 5,651 supported abnormal validation beats. The chosen structure was refitted on 28,758 supported abnormal development beats. The feature-sampling fraction 0.33 gives a nominal candidate count of $\lfloor 0.33(242) \rfloor = 79$ feature positions at a node. Entropy now sums over five classes, so a node with an exactly even five-class mixture would have $H = \log_2 5 \approx 2.322$ bits. A split lowers this entropy when its children become more concentrated in particular arrhythmia groups.

For an abnormal beat, the subtype classifier returns scores over the five supported abnormal outputs: PVC, PAC, LBBB, RBBB, and AFib. The final class score q_{ic} is Equation 5.22 with $B = 260$ and only these five training classes. The final subtype is

$$\hat{y}_i = \arg \max_{c \in \{1, \dots, 5\}} q_{ic}. \quad (5.26)$$

Thus the subtype forest does not learn a second normal-versus-abnormal decision. It resolves the abnormal branch by taking the largest of its five scores. Candidate subtype forests were selected by validation macro F1, which gives each of the five classes equal influence in the selection score, then by validation accuracy. The final saved bundle sets `reject_low_confidence=False`. Therefore the evaluated and installed final pipeline do not add a post-hoc “Unclassified abnormal” rejection step; every gated abnormal beat receives the subtype with the largest ensemble score.

The displayed subtype confidence is conditional on the abnormal branch. It is an ensemble score, not a calibrated probability of disease. Unsupported abnormalities can receive an incorrect supported subtype.

5.9 Final decision logic

The complete prediction rule is:

1. Validate that the input is a 512-sample MLII beat window and that 15 RR features are available.
2. Reconstruct the same window with the frozen autoencoder.
3. Build the 242-value vector from MLII shape, RR timing, and four autoencoder residual summaries.
4. Calculate the abnormal score p_i using the 240-tree binary detector.
5. If $p_i \leq 0.5708333$, output N with displayed score $1 - p_i$.
6. If $p_i > 0.5708333$, run the 260-tree subtype model and output the supported subtype with the highest score.

This is the same hierarchy used in the final temporal-holdout evaluation. The model does not use test labels while predicting. The labels are used only after prediction to compute accuracy, F1, confusion matrices, and subject-clustered uncertainty.

The full architecture can be written as one piecewise mathematical rule. Let z_i be the prepared 242-value row, p_i the binary ensemble score, and q_{ic} the subtype ensemble score. Define the supported abnormal class set $C = \{\text{PVC}, \text{PAC}, \text{LBBB}, \text{RBBB}, \text{AFib}\}$. Then

$$\hat{y}_i = \begin{cases} \text{N}, & p_i \leq 0.5708333, \\ \arg \max_{c \in C} q_{ic}, & p_i > 0.5708333. \end{cases} \quad (5.27)$$

For example, a beat with $p_i = 0.40$ is reported as N even if the subtype forest would have preferred PVC; that forest is not used for this beat. If $p_i = 0.70$ and its five subtype scores are (0.05, 0.10, 0.70, 0.10, 0.05) in the order above, the result is LBBB. These numbers illustrate the rule and are not measured results. The second stage's score is conditional on the first stage being passed, so the two stages must be evaluated both separately and together.

5.10 Mathematical training and inference procedure

The preceding sections define the individual components. This section brings them together as one reproducible procedure: learn a normal reconstruction function, freeze it, construct supervised features, fit and select the two forests, and apply the saved hierarchy to a new beat. The three models are trained sequentially; there is no joint loss that backpropagates a classification error through the complete system.

The first six worked examples below use deliberately small, hypothetical numbers to make each operation visible. They are teaching examples, not additional experimental observations. Whenever a window or forest is simplified, the full implementation still uses 512 waveform samples, 242 features, and the model sizes stated above. The final example connects the calculations into one complete prediction. Section 5.10.7 then follows real ECG segments through measured training calculations and saved-model prediction, with waveform figures and complete tree routes.

5.10.1 Notation, targets, and separation of data

Let $\mathcal{A}_{\text{fit}}$ and $\mathcal{A}_{\text{val}}$ denote the NSRDB normal-reference fitting and validation windows. They contain 288,673 windows from 15 subjects and 57,177 windows from three different subjects, respectively. These are consecutive four-second ECG1 windows, whereas the supervised examples are R-peak-aligned MLII windows. The common length is $T = 512$; the source channels and window alignment are not identical.

For supervised beat i , let $x_i \in \mathbb{R}^{512}$ be the prepared waveform, $\rho_i \in \mathbb{R}^{15}$ its RR measurements, r_i its subject cluster, $b_i \in \{0, 1\}$ its binary target, and c_i its subtype target. Define

$$C = \{\text{PVC}, \text{PAC}, \text{LBBB}, \text{RBBB}, \text{AFib}\}, \quad c_i \in \{\text{N}\} \cup C \cup \{\perp\}. \quad (5.28)$$

Here $\perp$ means an unsupported abnormal subtype: it still has $b_i = 1$ and can train the binary model. Targets follow the annotation mapping in Chapter 3; in particular, the project AFib target uses rhythm annotations. Neither a target nor a subject identifier is an input feature.

For record r with n_r accepted beats, use zero-based chronological index j , cuts $k_r = \lfloor 0.64n_r \rfloor$, $d_r = \lfloor 0.80n_r \rfloor$, and gap $g = 16$. The index sets are

$$\mathcal{I}_{\text{fit}}^{(r)} = \{j : 0 \leq j < k_r - g\}, \quad \mathcal{I}_{\text{val}}^{(r)} = \{j : k_r \leq j < d_r - g\}, \quad (5.29)$$

$$\mathcal{I}_{\text{dev}}^{(r)} = \{j : 0 \leq j < d_r - g\}, \quad \mathcal{I}_{\text{test}}^{(r)} = \{j : d_r \leq j < n_r\}. \quad (5.30)$$

Their unions over records define the corresponding study sets. Candidate fitting uses 66,285 beats and validation uses 16,029. Final refitting uses 83,050 development beats, including the earlier validation region and the gap that was needed only for candidate selection. The 16-beat exclusion before the temporal test remains in place. No test target enters these fitting or selection calculations in the final rerun.

Worked example: deciding which beats may teach the models

Suppose one record contains exactly 1,000 accepted beats. Then $k_r = 640$, $d_r = 800$, and $g = 16$. Using the same zero-based indices as Equation 5.30 gives:

Table 5.3: Illustrative allocation of a record containing 1,000 accepted beats.

Role	Beat indices	Number of beats
Candidate fitting	0–623	624
Gap before validation	624–639	16
Validation	640–783	144
Gap before temporal test	784–799	16
Temporal test	800–999	200
Final refitting	0–783	784

For example, beat 700 helps compare candidate models during validation. After selection it may join the final refitting set. Beat 850 does neither: the saved models predict it only during testing. If a fitting beat is labelled PVC, it teaches the binary model with $b_i = 1$ and the subtype model with $c_i = \text{PVC}$. A project-normal beat teaches only the binary model with $b_i = 0$. An unsupported abnormal beat teaches the binary model with $b_i = 1$ but is excluded from subtype fitting. This separates *when a beat is used* from *which model its label can train*.

5.10.2 Model 1: learning and freezing the autoencoder

Write the complete encoder–decoder as $A_\psi = g_\phi \circ f_\theta$, where $\psi = (\theta, \phi)$ collects its trainable parameters. At optimisation step k , a shuffled mini-batch $\mathcal{B}_k \subset \mathcal{A}_{\text{fit}}$ contains up to 256 windows. The training calculation is

$$\mathcal{L}_k(\psi) = \frac{1}{512|\mathcal{B}_k|} \sum_{x \in \mathcal{B}_k} \|x - A_\psi(x + \epsilon_x)\|_2^2, \quad \epsilon_x \sim \mathcal{N}(0, 0.03^2 I_{512}). \quad (5.31)$$

Here I_{512} is the identity matrix, so the added noise is independent across samples. Fresh noise is generated for each batch, and the clean window supplies the target. Backpropagation

calculates $\nabla_{\psi} \mathcal{L}_k$; Adam applies Equation 5.12 with learning rate 0.001. Thus the learned objects are convolutional weights, biases, and other network parameters, rather than stored arrhythmia labels.

After each of the two epochs, dropout is disabled and the network is evaluated on clean validation windows without a parameter update. If $\psi^{(e)}$ denotes the parameters after epoch e , checkpoint selection is

$$e^* = \arg \min_{e \in \{1,2\}} \frac{\sum_{x \in \mathcal{A}_{\text{val}}} \|x - A_{\psi^{(e)}}(x)\|_2^2}{512|\mathcal{A}_{\text{val}}|}, \quad \psi^* = \psi^{(e^*)}. \quad (5.32)$$

The recorded minimum occurs at epoch two. The saved A_{ψ^*} is then used in evaluation mode for every supervised example. MITDB labels cannot change ψ^* , and no artificial noise is added during feature extraction. The separately saved normal-residual threshold does not determine the final normal/abnormal decision.

Worked example: one reconstruction and one weight update

For arithmetic only, replace a 512-sample window by the four-sample clean target $x = (-1, 1, 1, -1)$. Suppose the added noise is $\epsilon = (0.03, -0.03, 0.02, -0.02)$. The network sees $\tilde{x} = (-0.97, 0.97, 1.02, -1.02)$, but must reproduce x . If its output is $\hat{x} = (-0.8, 0.7, 0.9, -0.8)$, the loss is

$$\mathcal{L}_{\text{example}} = \frac{(-0.2)^2 + (0.3)^2 + (0.1)^2 + (-0.2)^2}{4} = 0.045. \quad (5.33)$$

The output derivatives are $\partial \mathcal{L} / \partial \hat{x} = (0.10, -0.15, -0.05, 0.10)$. For instance, the second output is below its target, so its negative derivative indicates that increasing this output would locally reduce the error. Backpropagation carries these derivatives through the network to calculate how each parameter contributed to the error.

Suppose one parameter has value $w_0 = 0.500$ and its resulting gradient is $g_1 = -0.15$. At Adam's first step, with initially zero moments and $\beta_1 = 0.9$, $\beta_2 = 0.999$, the calculations are

$$m_1 = 0.1(-0.15) = -0.015, \quad v_1 = 0.001(-0.15)^2 = 0.0000225, \quad (5.34)$$

$$\hat{m}_1 = -0.15, \quad \hat{v}_1 = 0.0225, \quad (5.35)$$

$$w_1 = 0.500 - 0.001 \frac{-0.15}{\sqrt{0.0225 + 10^{-8}}} \approx 0.501. \quad (5.36)$$

This shows a concrete parameter change. In the real network, all trainable parameters receive their own updates, and a single update need not improve every sample. Repeated batches provide the learning signal. If illustrative clean-input validation losses after the two epochs were 0.060 and 0.050, epoch two would be saved. These example losses are separate from the measured losses reported earlier. Once saved, the network reconstructs new windows without repeating this training update.

5.10.3 Core calculation shared by training and prediction

Define $h(x_i) \in \mathbb{R}^{223}$ as the fixed waveform, spectrum, and statistics calculation, and $a(x_i, A_{\psi^*}(x_i)) \in \mathbb{R}^4$ as the four residual summaries defined earlier. Both forests receive exactly

$$z_i = \Phi_{\psi^*}(x_i, \rho_i) = \begin{bmatrix} h(x_i) \\ \rho_i \\ a(x_i, A_{\psi^*}(x_i)) \end{bmatrix} \in \mathbb{R}^{242}. \quad (5.37)$$

This makes the dependency explicit: the feature map depends on the frozen autoencoder, but not on b_i or c_i . The same feature order is required when fitting and predicting.

For example, the pooling operation used within h divides a sequence $v = (v_0, \dots, v_{L-1})$ into B equal bins with $m = \lfloor L/B \rfloor$ and calculates

$$[\text{pool}_B(v)]_j = \frac{1}{m} \sum_{\ell=0}^{m-1} v_{jm+\ell}, \quad 0 \leq j < B. \quad (5.38)$$

It uses the first Bm values. The full waveform gives 64 means of eight samples; the central 240 samples give 80 means of three samples. The central first-difference sequence has 239 values, of which the implementation uses the first 180 to form 60 means of three values. This truncation is part of the saved feature definition.

Using zero-based sample indices, the eight spectral values use the discrete Fourier transform $X_k = \sum_{t=0}^{511} x_t \exp(-2\pi i k t / 512)$ and frequencies $\nu_k = 128k/512$ Hz, where $i^2 = -1$. For a band $[\ell_b, u_b)$, the stored value is

$$P_b = \log \left(1 + \frac{\sum_{k: \ell_b \leq \nu_k < u_b} |X_k|^2}{\max\{\sum_{k: 0.5 \leq \nu_k \leq 50} |X_k|^2, 10^{-8}\}} \right). \quad (5.39)$$

The bands have edges 0.5, 3, 5, 8, 12, 20, 30, 40, 50 Hz. Together with 11 waveform statistics, these values give $64 + 80 + 60 + 8 + 11 = 223$.

RR features supply information that waveform shape alone cannot encode. With peak indices s_i at 128 Hz, define $\Delta_i = (s_i - s_{i-1})/128$ seconds. If μ_i and σ_i are the mean and population standard deviation of up to ten intervals ending at beat i , the seven base values are

$$\rho_i^{\text{base}} = \left(\Delta_i, \Delta_{i+1}, \mu_i, \frac{\Delta_i}{\mu_i}, \frac{\Delta_{i+1}}{\mu_i}, \frac{\sigma_i}{\mu_i}, \frac{60}{\Delta_i} \right). \quad (5.40)$$

Eight additional summaries use up to 20 intervals ending at the current beat: median, interquartile range, median absolute deviation, RMSSD, pNN50, median-normalised range, slope, and latest interval change. The base vector requires the next observed peak through Δ_{i+1} ; only the eight additional history summaries use exclusively current and earlier intervals.

Worked example: turning one beat into numerical evidence

Consider a new beat at $s_* = 1280$, with preceding peak 1184 and following peak 1392. Its raw window contains samples 1080–1591, placing the current peak at position 200. Suppose the

filtered window has mean 2.0 and standard deviation 0.5 in the signal's amplitude units. A filtered sample of 2.5 becomes $(2.5 - 2.0)/(0.5 + 10^{-8}) \approx 1.0$. Every sample is scaled using these same window statistics.

For one illustrative eight-sample group of the standardised waveform, $(-1, -1, 0, 0, 1, 1, 0, 0)$, the stored pooled value is $(-1 - 1 + 0 + 0 + 1 + 1 + 0 + 0)/8 = 0$. The next eight samples produce the next value, until 64 whole-window values are obtained. If one frequency band has power 20 and the total power used for normalisation is 100, its spectral feature is $\log(1 + 20/100) = \log(1.2) \approx 0.1823$.

Suppose the available preceding RR intervals, including the interval ending at the current beat, are $(0.875, 0.750, 0.875, 0.750)$ seconds. Their mean is 0.8125 and their population standard deviation is 0.0625. Since $\Delta_* = (1280 - 1184)/128 = 0.750$ and $\Delta_{*+1} = (1392 - 1280)/128 = 0.875$, the base RR vector becomes

$$\rho_*^{\text{base}} \approx (0.750, 0.875, 0.8125, 0.9231, 1.0769, 0.0769, 80). \quad (5.41)$$

The last value is $60/0.750 = 80$ beats per minute. For the same four-interval history, the median is 0.8125, each successive change has magnitude 0.125, and hence RMSSD is 0.125 seconds and pNN50 is 1.0. The latter means that all three observed changes exceed 0.05 seconds; it is stored as a fraction. With more history, the implementation uses up to the stated ten or twenty intervals for the respective calculations.

Next, reconstruct the full 512-sample window with the frozen autoencoder. Suppose its residuals give the following totals:

$$\sum_t |e_t| = 51.2, \quad \sum_t e_t^2 = 10.24, \quad \sum_{t=105}^{344} |e_t| = 36, \quad \sum_{t=2}^{512} |e_t - e_{t-1}| = 20.44. \quad (5.42)$$

These residual sums use the earlier one-based convention. Dividing by the appropriate numbers of samples gives

$$a_* = (51.2/512, 10.24/512, 36/240, 20.44/511) = (0.10, 0.02, 0.15, 0.04). \quad (5.43)$$

The 223 waveform values, all 15 RR values, and these four residual values are concatenated into z_* . Only selected entries are shown here; the model always receives all 242. A residual of 0.10 alone does not label this beat abnormal: the binary forest considers the combined evidence.

5.10.4 Model 2: fitting and selecting the binary detector

The binary training set is $\mathcal{D}_{B,\text{fit}} = \{(z_i, b_i, r_i) : i \in \mathcal{I}_{\text{fit}}\}$. Within this set, class and subject counts determine the weights in Equation 5.18. For each node containing index set S , a candidate feature j and cut t define

$$S_L(j, t) = \{i \in S : z_{ij} \leq t\}, \quad S_R(j, t) = S \setminus S_L(j, t), \quad (j^*, t^*) \in \arg \max_{(j,t) \in \mathcal{R}_S} \Delta H(j, t). \quad (5.44)$$

Here $\mathcal{R}_S$ is the randomly proposed set of admissible splits and ΔH is the weighted entropy reduction in Equation 5.21. Recursive splitting learns the node features, thresholds, and

weighted leaf class shares. Randomisation concerns candidate features and thresholds; the final configuration does not bootstrap training rows. The forest has no gradient update or epoch loop.

For binary candidate setting λ , let $B_\lambda(z)$ be its abnormal score and $\mathcal{T}_\lambda$ the distinct validation scores together with $\{0, 0.5, 1\}$. Threshold selection maximises validation balanced accuracy:

$$\tau_\lambda \in \arg \max_{t \in \mathcal{T}_\lambda} \text{BA}_{\text{val}}(\mathbf{1}[B_\lambda(z_i) > t], b_i). \quad (5.45)$$

Ties are resolved by F1, sensitivity, specificity, and closeness to 0.5, in that order. The three candidates are then compared by balanced accuracy and F1 at their selected thresholds. This selects 240 trees, entropy, unrestricted maximum depth, minimum leaf size one, square-root feature sampling, and weighting exponent 0.35. The selected threshold is stored at full precision and displayed as $\tau^* \approx 0.5708333$.

Finally, a new forest B^* is fitted with that setting on all 83,050 development beats. Weights are recomputed from the development labels and subject counts. The selected validation threshold is retained after refitting; it is not adjusted using the temporal-test labels.

Worked example: learning a split and choosing a gate

Suppose the training rows at a node have total normal weight 4 and total abnormal weight 4. These are sums of sample weights, not necessarily four beats per class. The parent entropy is $H(S) = 1$ bit. A random candidate asks whether the whole-window residual MAE is at most 0.12. Suppose its left child has normal and abnormal weights (3.5, 0.5), and its right child has weights (0.5, 3.5). Both children have entropy

$$H_{\text{child}} = -\frac{7}{8} \log_2 \frac{7}{8} - \frac{1}{8} \log_2 \frac{1}{8} \approx 0.5436. \quad (5.46)$$

Each child contains half the parent weight, giving $\Delta H = 1 - \frac{1}{2}(0.5436) - \frac{1}{2}(0.5436) = 0.4564$ bit. A competing split that leaves equal normal and abnormal weight in both children gives zero gain. The first candidate is therefore preferred between these two. Subsequent nodes repeat this calculation. This example shows how a threshold becomes a learned tree rule; 0.12 is an illustrative *feature split*, not the final abnormal-score threshold.

After fitting a whole candidate forest, suppose four validation beats have true targets (0, 0, 1, 1) and abnormal scores (0.20, 0.55, 0.60, 0.90). Applying the strict comparison $p > t$ gives:

Table 5.4: Illustrative validation comparison of three binary-score thresholds.

Threshold t	Predicted targets	Sensitivity	Specificity	Balanced accuracy
0.50	(0, 1, 1, 1)	1.00	0.50	0.75
0.55	(0, 0, 1, 1)	1.00	1.00	1.00
0.60	(0, 0, 0, 1)	0.50	1.00	0.75

The middle threshold wins this small example. At equality, a score of 0.55 does not pass a threshold of 0.55. The actual experiment searches its own validation scores and saves $\tau^* \approx 0.5708333$, rather than the toy value 0.55. Validation selects a setting and threshold; final refitting then learns new tree rules from the larger development set.

5.10.5 Model 3: fitting the abnormal-subtype classifier

The subtype training population is selected using the true supported labels:

$$\mathcal{D}_{C,\text{fit}} = \{(z_i, c_i, r_i) : i \in \mathcal{I}_{\text{fit}}, c_i \in C\}. \quad (5.47)$$

It contains 22,844 beats. Normal and unsupported abnormal targets are excluded, and inclusion does not depend on whether the binary model predicts the beat correctly. This prevents binary training errors from deciding which labelled examples the subtype model can learn from.

The same node-fitting rule in Equation 5.44 is applied with five target classes. For candidate γ , prediction is the class with the largest mean leaf score. Candidate selection uses 5,651 true supported abnormal validation beats and maximises

$$\text{MacroF1}_{\text{val}}(\gamma) = \frac{1}{5} \sum_{c \in C} \frac{2TP_c}{2TP_c + FP_c + FN_c}, \quad (5.48)$$

with a zero score for a zero denominator and accuracy as the tie-break. The selected setting uses 260 trees, entropy, unrestricted maximum depth, minimum leaf size one, feature fraction 0.33, and weighting exponent 0.25. It is refitted as C^* on the 28,758 supported abnormal development beats, with weights recomputed for that population. The stored model learns five abnormal alternatives; it has neither a normal class nor an unknown-class rejection rule.

Worked example: learning to distinguish abnormal subtypes

For a small node with equal effective sample weights, suppose the labels are two PVC, two PAC, one LBBB, one RBBB, and two AFib beats. Its class shares are $(1/4, 1/4, 1/8, 1/8, 1/4)$ and its entropy is 2.25 bits. If a candidate morphology split places the two PVC beats in one child and the remaining six beats in the other, the first child has zero entropy and the second has entropy approximately 1.9183. The gain is

$$\Delta H = 2.25 - \frac{2}{8}(0) - \frac{6}{8}(1.9183) \approx 0.8113 \text{ bits}. \quad (5.49)$$

This split helps isolate PVC. Later splits can separate the remaining classes. Real fitting uses the class-and-subject weights, but the same weighted-share arithmetic applies. No normal target appears in this node.

To illustrate model selection, suppose two completed candidate forests have validation F1 scores, in the order PVC, PAC, LBBB, RBBB, AFib, of

$$\text{candidate A: } (0.90, 0.60, 0.80, 0.85, 0.75), \quad \text{MacroF1}_A = 3.90/5 = 0.78, \quad (5.50)$$

$$\text{candidate B: } (0.85, 0.75, 0.80, 0.80, 0.80), \quad \text{MacroF1}_B = 4.00/5 = 0.80. \quad (5.51)$$

Candidate B wins even though A has the better PVC score, because the stated criterion gives every subtype equal influence. The winning configuration is then refitted using all supported abnormal development beats. Training uses the true subtype labels; at deployment the saved subtype model is called only after the binary gate has passed the beat.

5.10.6 Applying the saved models to an unseen beat

The result of training is the fixed collection

$$\mathcal{M}^* = \{A_{\psi^*}, \Phi_{\psi^*}, B^*, C^*, \tau^*\}. \quad (5.52)$$

For a previously unscored waveform u_* , preparation produces x_* using the same 128 Hz sampling, 512-sample geometry, local filtering, and standardisation as the supervised fitting data. Observed peaks supply ρ_* . Prediction then evaluates

$$\hat{x}_* = A_{\psi^*}(x_*), \quad z_* = \Phi_{\psi^*}(x_*, \rho_*), \quad p_* = B^*(z_*), \quad (5.53)$$

$$\hat{y}_* = \begin{cases} N, & p_* \leq \tau^*, \\ \arg \max_{c \in C} C_c^*(z_*), & p_* > \tau^*. \end{cases} \quad (5.54)$$

The autoencoder makes one reconstruction, and the binary trees route z_* through their stored split rules. Only a gated abnormal beat needs subtype routing. No label is required, no optimiser is called, and no split, leaf share, network weight, or threshold is updated. Window standardisation uses the new window's own mean and standard deviation; this deterministic preparation is not model retraining. The displayed score is $1 - p_*$ on the normal branch and $\max_c C_c^*(z_*)$ on the abnormal branch. The latter is the subtype forest's score, not the product of the two forest scores or a calibrated clinical probability.

Inference also requires sufficient signal context. If n_{obs} is the number of available 128 Hz samples, the beat window requires $n_{\text{obs}} \geq s_* + 312$, and the next peak s_{*+1} must already have been detected. Thus the waveform alone requires about 2.44 seconds of post-peak context; a longer next RR interval or acquisition and detection delays can increase the wait. The system is a delayed beat classifier. Database testing uses annotated beat locations, while live use detects peaks from the signal, adding a source of error not isolated by the temporal test.

Here “unseen” means absent from the fitting rows of the saved models, or newly arriving during use. The final temporal evaluation concerns later beats from known patients, and its test region had been inspected during earlier development. The equations specify how a new patient's beat would be processed, but do not establish equivalent accuracy on unseen patients. Likewise, because every gated abnormal beat receives a supported subtype, an unfamiliar abnormality can be detected by B^* yet mislabelled by C^* .

Worked example: the complete path from a new beat to a label

Return to the illustrative beat at sample 1280. The following sequence shows what changes at each step and what remains fixed:

1. **Wait for the required signal.** The backend needs the full window through sample 1591 and the detected following peak at 1392. Detecting the following peak alone is insufficient if the rest of the window has not yet arrived.
2. **Prepare and describe the beat.** Filtering and scaling produce x_* . Its RR measurements include the values in Equation 5.41. The saved autoencoder reconstructs x_* , giving the illustrative residual features (0.10, 0.02, 0.15, 0.04). These join the waveform measurements to form the complete $z_* \in \mathbb{R}^{242}$.
3. **Apply the binary forest.** Suppose 180 of the 240 trees return an abnormal leaf score of 1 and the other 60 return 0. Then $p_* = (180 \times 1 + 60 \times 0) / 240 = 0.75$. Since $0.75 > 0.5708333$, this beat passes to the subtype model. The real forest can also return fractional leaf scores.
4. **Apply the subtype forest.** Suppose the mean scores are (0.70, 0.10, 0.08, 0.07, 0.05) for PVC, PAC, LBBB, RBBB, and AFib, after averaging the 260 trees. The largest score is 0.70, so the reported subtype is PVC. These are hypothetical model outputs, not scores calculated from only the few feature entries printed in this example.
5. **Report without learning.** The output is an abnormal beat labelled PVC, with binary score 0.75 and displayed subtype score 0.70. It is not assigned the product $0.75(0.70) = 0.525$ as its displayed score. The saved weights, tree splits, and threshold remain unchanged.

For comparison, if another new beat has $p_* = 0.40$, the binary gate returns N with displayed score $1 - 0.40 = 0.60$, and its subtype calculation is skipped. Neither beat needs a known diagnosis to obtain a prediction. If an expert label becomes available later, it can be compared with the prediction for evaluation; that comparison does not automatically retrain any model.

5.10.7 End-to-end case study using real ECG segments

The preceding small examples make the arithmetic manageable. This case study now performs the operations on real archived ECG, using the research model itself. It connects a normal-reference training window, an earlier labelled PVC used for supervised fitting, and a later PVC withheld from fitting. The waveforms are measured signals, and the reported predictions, residuals, weights, and tree rules are calculated from the saved research artifacts.

Three kinds of evidence must be distinguished. The two-epoch autoencoder losses and model-selection scores come from the original training logs. The supervised forests were independently refitted on the complete original development populations, reproducing every saved feature split, threshold, and leaf value. A separate one-step autoencoder demonstration was run from fresh random weights on real normal windows; this exposes an actual gradient

update but is not a reconstruction of an unrecorded historical training step. The original saved models remain unchanged throughout.

1. Identify the signals and keep their roles separate

The normal-reference example is the first four seconds of ECG1 from NSRDB record 16272, one of the 15 fitting subjects. The supervised examples use MLII from MITDB record 105. These are different databases, channels, and training roles [6, 5]. Table 5.5 identifies the segments so that the reader can locate them independently.

Table 5.5: Real ECG segments used in the training-to-prediction case study.

Role	Source	Exact location and target
Autoencoder demonstration	NSRDB 16272, ECG1	Source samples 0–511 at 128 Hz; cached accepted window 0; clean waveform is the target.
Supervised fitting example	MITDB 105, MLII	Accepted-beat index 484; R peak 44,839 at 128 Hz (350.304688 s); binary target 1, subtype PVC.
Temporal-test example	MITDB 105, MLII	Accepted-beat index 2,066; R peak 187,009 at 128 Hz (1461.007813 s); annotation V, mapped to PVC.

All indices in this case study are zero-based unless a residual-sum formula explicitly uses the earlier one-based convention. Record 105 has 2,565 accepted beats, so its selection and test cuts are 1,641 and 2,052. Candidate fitting uses indices 0–1,624; validation uses 1,641–2,035; final refitting uses 0–2,035; and the test uses 2,052–2,564. The required 16-beat gaps precede validation during selection and the test during final refitting. Thus index 484 can influence the learned rules, whereas index 2,066 cannot influence this rerun’s fitting or model selection.

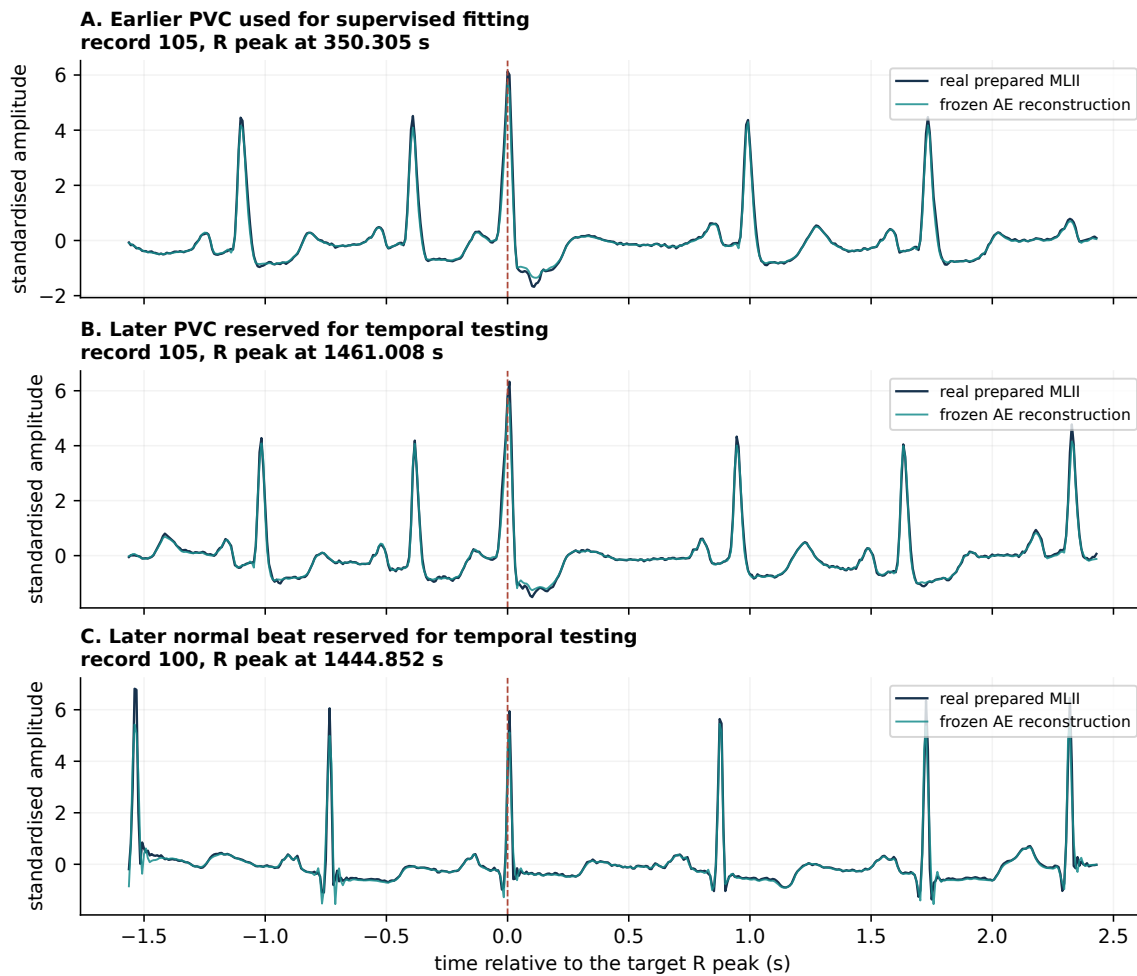


Figure 5.9: Actual prepared ECG windows and their saved-autoencoder reconstructions. The red line marks the target R peak; the other peaks are neighbouring beats within the same four-second window. Panels A and B are earlier and later PVC examples from record 105. Panel C is a normal test beat from record 100. The classifier labels the target beat, not every beat visible in the window.

The examples were selected for explanation after the research models were fixed. The test PVC is the first correctly classified PVC, in saved test order, from a record that also contains candidate-fitting PVCs. The earlier example is the middle candidate-fitting PVC in that record. This is a transparent case-selection rule, not a new accuracy estimate. Patient overlap and the earlier development exposure of the temporal test remain as described in Chapter 6.

2. Observe a real autoencoder training update

Training begins with normal data, before either classifier is fitted. The first archived window from record 16272 is a real 512-value target. For example, its first three clean samples are $(-0.075544, -0.143846, -0.188566)$. In the new demonstration, Gaussian noise with standard deviation 0.03 changes these inputs to $(-0.103119, -0.150460, -0.140566)$. The clean values

remain the targets. Figure 5.10 shows the full segment rather than only these three numbers.

Preserve the archived scaling. The saved first window has mean 0.005864 and standard deviation 0.530754. Consequently, its historical scaling cannot be described as unit-variance scaling of that individual window. Its waveform was matched to the filtered source samples using an affine transformation, with maximum absolute discrepancy below 1.1×10^{-7} . The demonstration uses the archived values directly; silently standardising them again would change the training example. The current normal-data preparation code specifies local standardisation when rebuilding a cache, but it also reuses existing caches. That code alone does not establish how an older cached array was scaled. This distinction does not alter the verified beat-local scaling of the supervised MLII examples.

The demonstration uses the first 256 accepted cached windows from fitting subject 16272, batch size 256, noise standard deviation 0.03, learning rate 0.001, and random seed 20260927. The network architecture is unchanged, but its weights are freshly initialised. The noisy-input batch loss is

$$\mathcal{L}_{\text{before}} = \frac{1}{131072} \sum_{i=1}^{256} \sum_{t=1}^{512} (x_{it} - A_{\psi_0}(x_i + \epsilon_i)_t)^2 = 0.449509. \quad (5.55)$$

Here $131072 = 256(512)$ is the number of scalar reconstruction errors in one batch. Every clean ECG sample contributes to the loss; no arrhythmia label is supplied.

Backpropagation gives the actual gradient -0.610003 for the final output bias. Its value before the update is -0.147403 . At this first Adam step, the moment calculations and bias correction give

$$m_1 = 0.1(-0.610003) = -0.0610003, \quad (5.56)$$

$$v_1 = 0.001(-0.610003)^2 \approx 0.000372103, \quad (5.57)$$

$$(\widehat{m}_1, \widehat{v}_1) \approx (-0.610003, 0.372103), \quad (5.58)$$

$$b_{\text{out},1} = -0.147403 - 0.001 \frac{-0.610003}{\sqrt{0.372103 + 10^{-8}}} \approx -0.146403. \quad (5.59)$$

This is one visible parameter change among all the network updates. Its negative gradient moves the bias upward to reduce the batch error locally. The filters and normalisation parameters receive their own gradients; changing only this bias would not reproduce the complete training step.

Re-evaluating the same noisy batch with the same dropout masks after the update gives MSE 0.319433. The displayed window's own MSE changes from 0.435649 to 0.313658. The controlled comparison shows that this particular update reduced the error. It does not imply that one step learns normal ECG adequately or that every future step reduces every window's error.

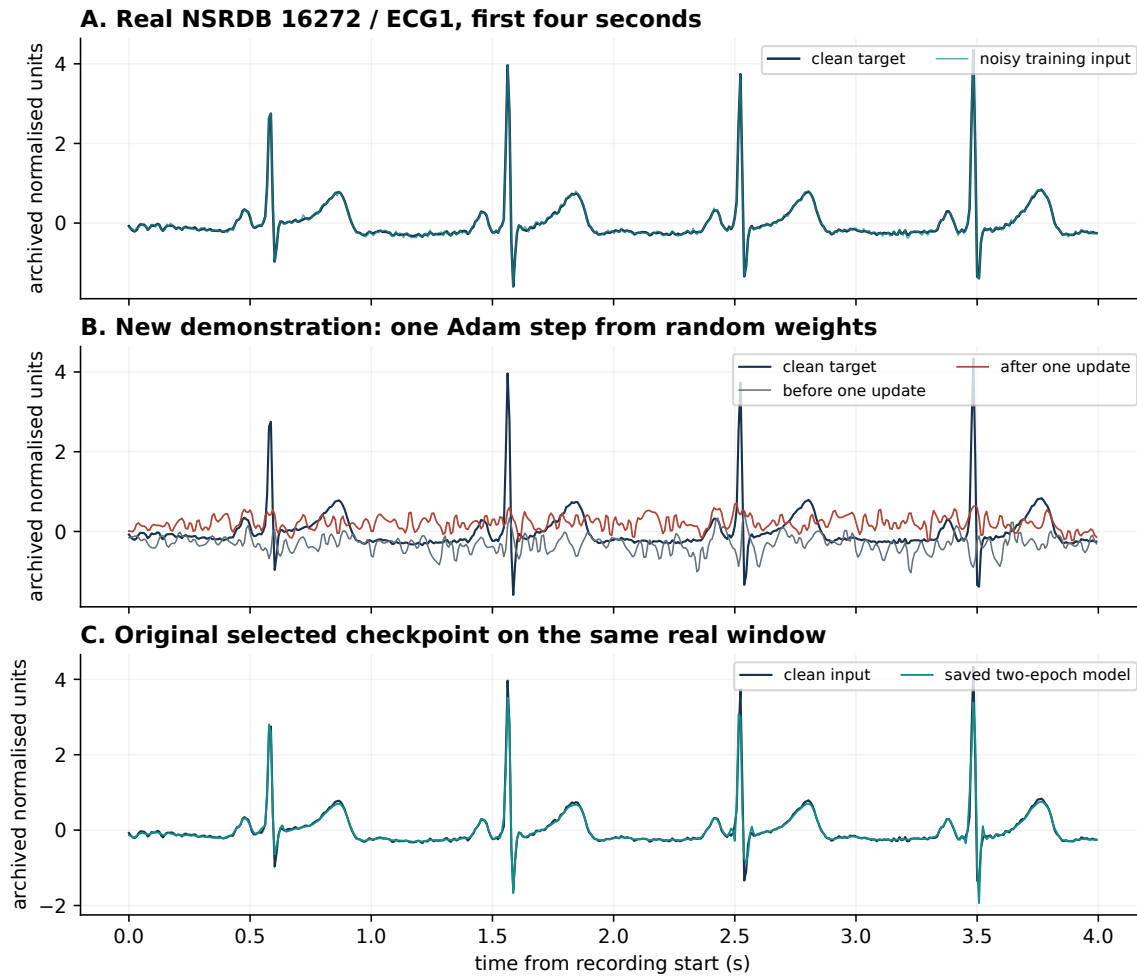


Figure 5.10: Real ECG1 samples from NSRDB 16272. A: clean target and newly corrupted input. B: actual outputs before and after one isolated Adam update from random weights. C: reconstruction by the original selected two-epoch checkpoint. Panel C is not the result of the single step in panel B. Archived normalised amplitudes are retained.

The original full training used 288,673 fitting windows, shuffled across 15 subjects, for two epochs. Each epoch therefore contained 1,128 batches, including a smaller final batch. Its recorded training MSE was 0.057639 and 0.041976; clean-input validation MSE on the three other subjects was 0.061195 and 0.061193. The second checkpoint was retained. On the displayed normal fitting window, that checkpoint has MSE 0.010463, as illustrated in panel C. This last number describes a fitting example, not unseen-subject performance. The independently measured validation loss remains the evidence used to choose the checkpoint.

3. Convert real labelled beats into supervised feature rows

Once the autoencoder is frozen, earlier MLII beats become the examples used to train the two forests. The fitting PVC at 350.304688 seconds supplies one 242-value row, binary target 1, and subtype target PVC. Its previous and next RR intervals are 0.398438 and 0.992188 seconds, and

its four residual features are

$$a_{\text{fit}} = (0.052354, 0.011190, 0.065462, 0.041013). \quad (5.60)$$

These numbers are computed from the waveform in panel A of Figure 5.9. Normal fitting beats supply the same kind of feature row with binary target 0. Other supported abnormalities supply binary target 1 and their own subtype target. The training populations contain many such rows; the forests are not fitted to just the displayed PVC.

The later test PVC follows the identical feature calculation, but its label is withheld from fitting. Its original annotation is V at source sample 525,964 in the 360 Hz recording. Resampling and index conversion give

$$s_* = \left\lfloor 525964 \frac{128}{360} \right\rfloor = 187009, \quad u_* = v[186809 : 187321]. \quad (5.61)$$

The slice endpoint is exclusive: samples 186,809 through 187,320 provide exactly 512 values. The R peak appears at local index 200. After filtering, the window mean and standard deviation are 0.018076 and 0.325109 mV. For example, the filtered sample at the target peak is 1.903376 mV, so

$$x_*[200] = \frac{1.903376 - 0.018076}{0.325109 + 10^{-8}} \approx 5.798973. \quad (5.62)$$

The values printed here are rounded; computations use full precision. Standardisation changes the amplitude units, while preserving the relative shape shown in Figure 5.11.

The first eight standardised samples have mean -0.024118 , giving feature entry 0. The mean of samples 216–223 is -1.222917 , giving entry 27. The 12–20 Hz log-relative-power measurement is 0.111020 at entry 208. These are examples of actual waveform-derived values, rather than manually assigned descriptions such as “wide QRS”. The remaining waveform entries are calculated by the same pooling, spectrum, and statistics rules.

The observed neighbouring peaks are 186,960 and 187,130. Their timing gives

$$\Delta_* = (187009 - 186960)/128 = 0.3828125 \text{ s}, \quad (5.63)$$

$$\Delta_{*+1} = (187130 - 187009)/128 = 0.9453125 \text{ s}. \quad (5.64)$$

The short preceding interval and longer following interval describe the timing around this annotated premature beat. They are measurements used by the model, not a stand-alone diagnostic rule. The local mean of the preceding ten intervals is 0.635156 seconds. Hence the previous/local-mean ratio is $0.3828125/0.63515625 = 0.602706$, the next/local-mean ratio is 1.488315, and instantaneous rate is $60/0.3828125 = 156.7347$ beats per minute. This rate is the reciprocal of one interval, not a long-term average rate.

Table 5.6: All 15 measured RR features for the real temporal-test PVC.

Base measurement	Value	History measurement	Value
Previous RR (s)	0.382813	Median (s)	0.656250
Next RR (s)	0.945313	IQR (s)	0.039063
Local mean RR (s)	0.635156	MAD (s)	0.019531
Previous / local mean	0.602706	RMSSD (s)	0.062884
Next / local mean	1.488315	pNN50 fraction	0.052632
RR coefficient of variation	0.136199	Range / median	0.476190
Instantaneous rate (bpm)	156.734695	Normalised trend	−0.004654
		Normalised latest change	−0.392857

The 20-interval history contains 19 successive differences. Exactly one exceeds 0.05 seconds in absolute value, so pNN50 is $1/19 = 0.052632$. This explains both the denominator and why the stored value is a fraction. The full history and all 242 feature entries are retained in the case-study audit file identified at the end of this subsection.

The frozen autoencoder reconstructs the same test window. Its measured absolute-residual sum is 27.777321 and squared-residual sum is 6.070283. Applying the residual equations gives

$$a_{*,1} = 27.777321/512 = 0.054253, \quad a_{*,2} = 6.070283/512 = 0.011856, \quad (5.65)$$

$$a_{*,3} = 14.046151/240 = 0.058526, \quad a_{*,4} = 30.254937/511 = 0.059207. \quad (5.66)$$

These four numbers are appended after the 223 waveform and 15 RR values. The archived feature row was independently rebuilt from the source signal, measured peak context, and saved checkpoint. Recomputed residuals agreed with the saved values within 2×10^{-6} , and the saved forests returned the same scores from the rebuilt row.

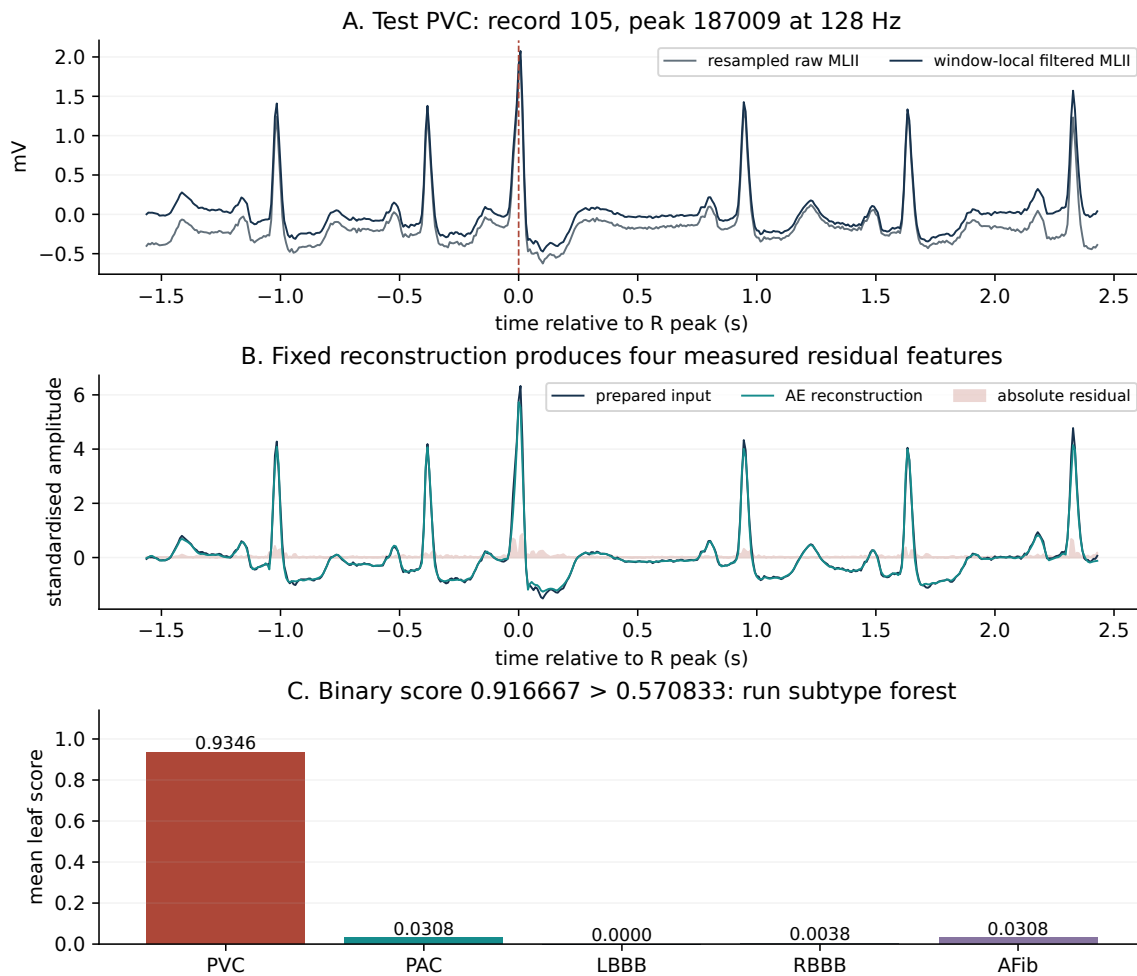


Figure 5.11: The same real test PVC passes through source-window filtering, standardisation, frozen autoencoder reconstruction, and the two supervised forests. The bottom panel reports measured subtype scores after the binary gate passes the beat. The expert V annotation is used for comparison, not supplied to prediction.

An important observation is that the autoencoder reconstructs this abnormal beat quite closely. Its MAE is 0.054253, while the displayed normal test beat has a larger MAE of 0.095564. Thus these real examples do not support the shortcut “larger reconstruction error always means abnormal”. The forests use the joint morphology, rhythm, and residual evidence. The autoencoder alone does not determine either the binary decision or the PVC subtype.

4. Learn the binary forest from real development rows

For final refitting, every eligible development row receives a binary target and a sample weight. The displayed fitting PVC belongs to the abnormal class, which contains 32,477 development beats across 42 subject clusters. Record 105 contributes 39 abnormal development beats. Its

unnormalised binary weight is therefore

$$\tilde{w}_B = \frac{1}{32477} \left(\frac{32477}{42(39)} \right)^{0.35} \approx 8.75923 \times 10^{-5}. \quad (5.67)$$

Dividing by the mean weight across all 83,050 binary fitting rows and applying the clipping rule gives $w_B = 4.073719$. The displayed beat therefore has more influence than a row with weight one. It is still one real beat; the weight does not create copies or add new observations.

Consider the first tree in the final binary forest. At its root, the total weight is 83,050 and the weighted class shares are (0.524047, 0.475953) for N and abnormal. Their entropy is 0.998331 bits. The fitted split asks whether whole-window mean bin 27 is at most 1.150551. The left child receives 82,227 rows with total weight 81,919.375851; the right child receives 823 rows with weight 1,130.624149. Their entropies are 0.997322 and 0.326286. Substituting actual fitted-node values gives

$$\begin{aligned} \Delta H_B = 0.998331 - \frac{81919.375851}{83050} (0.997322) \\ - \frac{1130.624149}{83050} (0.326286) \approx 0.010144 \text{ bits.} \end{aligned} \quad (5.68)$$

This modest reduction is a real first partition, not the final classification. The algorithm compares randomly proposed splits, retains the best available one, and repeats the operation inside each child. Different features become useful later in the tree. The saved tree records the winning rules; it does not retain the full historical list of rejected candidate splits.

The 240-tree forest repeats this learning process with different random choices. All trees receive the same weighted development population because bootstrap sampling is disabled. No derivative of the autoencoder is computed during tree fitting, and the two supervised targets never change the frozen autoencoder checkpoint.

5. Learn the subtype forest from real abnormal rows

Subtype fitting keeps only the 28,758 supported abnormal development beats. The displayed fitting PVC now belongs to a class containing 5,632 PVC beats across 32 subject clusters; record 105 contributes 35 PVC development beats. The subtype weight calculation is

$$\tilde{w}_C = \frac{1}{5632} \left(\frac{5632}{32(35)} \right)^{0.25} \approx 2.65888 \times 10^{-4}, \quad w_C = 1.693698. \quad (5.69)$$

The different final weight arises because both the target classes and the fitting population differ. The same feature row can therefore have weight 4.073719 in binary fitting and 1.693698 in subtype fitting.

At the root of the first subtype tree, the weighted shares for PVC, PAC, LBBB, RBBB, and AFib are (0.193628, 0.159765, 0.221019, 0.210692, 0.214895). The entropy is 2.312778 bits. This tree's first split uses RR pNN50, with threshold 0.678082. The left and right children contain

16,935 and 11,823 rows, have weights 18,528.763857 and 10,229.236143, and have entropies 2.111793 and 1.685630. Thus

$$\begin{aligned}\Delta H_C &= 2.312778 - \frac{18528.763857}{28758}(2.111793) \\ &\quad - \frac{10229.236143}{28758}(1.685630) \approx 0.352572 \text{ bits.}\end{aligned}\tag{5.70}$$

This calculation separates mixtures of abnormal subtypes. It never asks whether a beat is N, because N is absent from this fitting population. Further nodes combine frequency, morphology, and rhythm features until leaves store their weighted subtype shares. Averaging 260 such trees gives the final five-score vector.

6. Use validation to select models, then freeze the refits

The foregoing final-tree calculations follow model selection. During the original selection stage, the candidate forests learned from the earlier candidate-fitting regions and were compared on the later validation regions. Table 5.7 gives their recorded scores. These are original experiment measurements, not the small illustrative scores used earlier in this section.

Table 5.7: Recorded validation measurements selecting the real saved models.

Stage	Candidate	Threshold	Primary score	Secondary score
Binary	depth14	0.504692	BA 0.976183	F1 0.969381
Binary	depth22	0.536820	BA 0.985738	F1 0.981085
Binary	deep (selected)	0.570833	BA 0.987304	F1 0.983886
Subtype	depth22	–	Macro F1 0.978043	Accuracy 0.986197
Subtype	deep_sqrt	–	Macro F1 0.976246	Accuracy 0.985135
Subtype	deep_third (selected)	–	Macro F1 0.981126	Accuracy 0.988321

The deep binary candidate has the highest validation balanced accuracy, and the deep_third subtype candidate has the highest validation macro F1. Their settings are retained. Final refitting then uses all eligible earlier development rows, with fresh weights for those populations. The fixed binary threshold remains 0.5708333333333333; test labels do not choose it.

For this case study, the full 240-tree and 260-tree refits were reproduced in memory using their original final-fitting seeds 20260901 and 20260902. Every feature index, split threshold, and leaf-value array matched the saved forests exactly. Predictions over all 20,969 temporal-test rows also matched within numerical tolerance. This verifies the connection between the stated training procedure and the particular models used below. It does not add an independent test population or remove the original study’s design limitations.

7. Route the later real PVC through the learned trees

Now place the training labels aside. The later test beat contributes only its prepared feature row z_* . A learned tree does not compare it with a stored ECG picture; it applies its feature

thresholds in order. In the first binary tree, feature entry 27 is -1.222917 , so it satisfies $-1.222917 \leq 1.150551$ and takes the left branch. At the next node, entry 28 is -0.617664 , which is greater than -1.485152 , so it takes the right branch. Table 5.8 records every subsequent comparison in this tree. Feature entry 27 means the 28th value in the 242-entry vector; feature and node indices here follow the zero-based code.

Table 5.8: Complete route of the real test PVC through the first saved binary tree.

Node	Feature (zero-based index)	Beat value	Split value	Branch
0	27: whole-window mean bin 27	-1.222917	1.150551	left
1	28: whole-window mean bin 28	-0.617664	-1.485152	right
85	208: spectral band 4	0.111020	0.213205	left
86	69: central mean bin 5	-0.260181	-1.218844	right
550	208: spectral band 4	0.111020	0.066604	right
2368	93: central mean bin 29	-0.134006	0.291394	left
2369	234: RR pNN50	0.052632	0.466071	left
2370	159: central derivative bin 15	0.688649	-0.528727	right
2376	160: central derivative bin 16	-1.093958	-0.085592	left
2377	22: whole-window mean bin 22	-0.126941	0.644760	left
2378	67: central mean bin 3	-0.273355	1.523353	left
2379	3: whole-window mean bin 3	0.276538	2.012589	left
2380	221: skewness	2.938237	3.304656	left
2381	222: excess kurtosis	11.184209	13.890345	left
2382	241: AE residual-change MAE	0.059207	0.317291	left
2383	29: whole-window mean bin 29	0.124833	0.321360	left
2384	160: central derivative bin 16	-1.093958	-0.959581	left

The route ends at leaf 2385, which contains 23 development rows and has abnormal score 1. The other 239 trees route the same input through their own rules. For this beat, their total abnormal leaf score is 220, giving

$$p_* = \frac{220}{240} = 0.9166667 > 0.5708333. \quad (5.71)$$

The gate therefore requests a subtype. This is a prediction based on fitted rules; the V annotation has still not been used in the computation.

The first subtype tree begins with pNN50. The beat's value 0.052632 is at most 0.678082, so it goes left. At the next node, its 12–20 Hz spectral feature 0.111020 exceeds 0.100289, so it goes right. The complete route is shown in Table 5.9.

Table 5.9: Complete route of the real test PVC through the first saved subtype tree.

Node	Feature (zero-based index)	Beat value	Split value	Branch
0	234: RR pNN50	0.052632	0.678082	left
1	208: spectral band 4	0.111020	0.100289	right
541	208: spectral band 4	0.111020	0.142781	left
542	104: central mean bin 40	−0.993279	0.296236	left
543	104: central mean bin 40	−0.993279	−0.872055	left
544	103: central mean bin 39	−1.227315	−1.007380	left
545	91: central mean bin 27	0.056109	−0.586506	right
549	94: central mean bin 30	0.411630	0.175494	right

This route ends at leaf 555, with 143 development rows and score vector (1, 0, 0, 0, 0). Other trees can disagree. Across the full subtype forest, the class-score totals, in the order PVC, PAC, LBBB, RBBB, AFib, are (243, 8, 0, 1, 8). Dividing by 260 gives

$$q_* = \frac{(243, 8, 0, 1, 8)}{260} = (0.934615, 0.030769, 0, 0.003846, 0.030769). \quad (5.72)$$

The largest component is PVC. The final output is therefore an abnormal beat with subtype PVC, binary score 0.916667, and displayed subtype score 0.934615. These scores have different roles and are not multiplied for display. Only after this prediction is complete is it compared with the expert V annotation, which maps to PVC and confirms that this selected prediction is correct.

In a stream, this beat cannot be classified at the instant of its R peak. Its following R peak is needed for the 0.945313-second next RR interval, and the complete window extends through sample 187,320. That last sample occurs at 1463.437500 seconds, about 2.43 seconds after the target peak; buffering and peak detection can add further delay. These data-availability requirements apply even though evaluating the frozen models requires no training step.

8. Compare the normal branch and the other abnormal outputs

The real normal test beat from record 100 at 1444.851563 seconds provides a useful contrast. Its binary score is $13/240 = 0.054167$, below the gate. The final result is N with displayed score $1 - 0.054167 = 0.945833$. Runtime subtype prediction is skipped. If the subtype forest is queried separately for audit purposes, it prefers PAC for this normal beat, because that forest has no N output. This demonstrates why a subtype score cannot replace the preceding normal/abnormal decision.

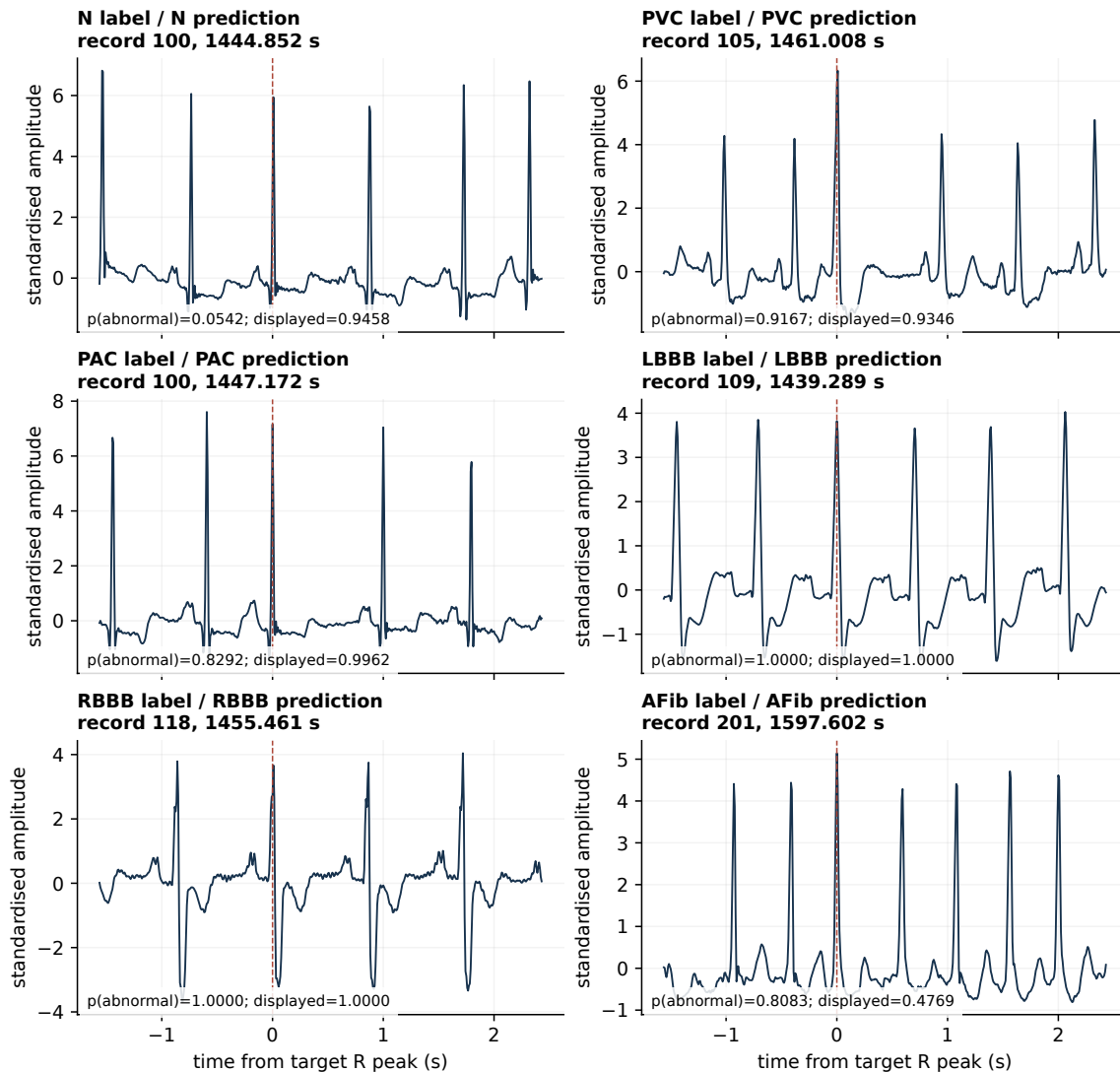


Figure 5.12: Real prepared four-second MLII windows illustrating the six project outputs. For each class, the first correctly classified test example in saved order is shown; the PVC additionally requires earlier fitting PVCs in the same record. The dashed line marks the target beat. Scores are actual frozen-model outputs. Selection for correct classification means this panel is an explanation of behaviour, not evidence of perfect accuracy.

Table 5.10: Measured predictions for the six real illustrative test segments.

Reference and output	Record	Peak time (s)	Binary score	Displayed score
N	100	1444.851563	0.054167	0.945833
PVC	105	1461.007813	0.916667	0.934615
PAC	100	1447.171875	0.829167	0.996154
LBBB	109	1439.289063	1.000000	1.000000
RBBB	118	1455.460938	1.000000	1.000000
AFib	201	1597.601563	0.808333	0.476923

The AFib example is especially informative: its subtype scores for AFib and PAC are 0.476923 and 0.453846. AFib wins narrowly, even though its score is below 0.5, because subtype selection uses the largest of five scores and the final bundle disables low-confidence rejection. The reference AFib label comes from the project’s rhythm-annotation mapping, including its stated AFIB/AFL grouping; a four-second plot alone is not used to establish that label. Neither a score of 1 nor agreement in these selected examples is a calibrated clinical certainty. The complete results, including errors, remain in Chapter 7.

The complete operational sequence is therefore visible in measured data: normal ECG supplies reconstruction targets; labelled earlier MLII beats supply weighted binary and subtype fitting rows; validation chooses the settings and gate; refitting produces fixed trees; and a later waveform passes through preparation, feature construction, the gate, and subtype selection. Prediction neither receives the expert label nor changes the network weights or tree rules.

All numerical values, the full 242-entry rows, source locations, training weights, and complete tree routes are retained in the audit file:

```
experiments/methodology_real_case.json
```

The figures and numerical audit can be regenerated using:

```
tools/generate_real_methodology_case.py
```

The audit records hashes confirming that the original autoencoder and forest files were not modified.

5.11 Why this architecture was selected

Table 5.11: Reason for each major final-architecture decision.

Decision	Reason
MLII-only supervised input	matches the signal available in 46 eligible MIT-BIH records and avoids hidden two-lead assumptions
R-peak alignment	puts comparable cardiac events in comparable positions inside every 512-sample window
Waveform and derivative bins	preserve beat morphology, including sharp QRS changes
Frequency powers and statistics	add compact information about signal energy, scale, and distribution
Fifteen RR features	represent local rate, premature beats, rhythm irregularity, and recent interval trend using information reproducible during streaming
Normal-only autoencoder	supplies a learned measure of disagreement with normal ECG structure without requiring arrhythmia labels
Two-stage hierarchy	separates binary detection from subtype evaluation; unsupported labels enter binary scoring only, although runtime predictions still assign a supported subtype
Extra Trees models	handle mixed feature scales and non-linear interactions with reproducible tabular-model training
Temporal split with gap	evaluates later signal from each record while removing direct boundary overlap between development and test beats

Chapter 6 describes model selection and performance evaluation.

6 Experimental Method

The main experiment evaluated later MLII beats from patients whose earlier labelled ECG was available during development. Beats were split in chronological order within each record. This design measures performance on known patients. The final segment had been viewed during earlier development, so the reported results form a retrospective evaluation.

6.1 Method overview

Figure 6.1 shows how accepted beats are allocated to model selection, final refitting, and testing. The normal autoencoder was already frozen from NSRDB. The supervised experiment then used only MITDB MLII beats. Candidate models were selected with earlier temporal regions, refitted before the test boundary, and evaluated only on the final 20% of accepted beats.

Chronological split within each MITDB record

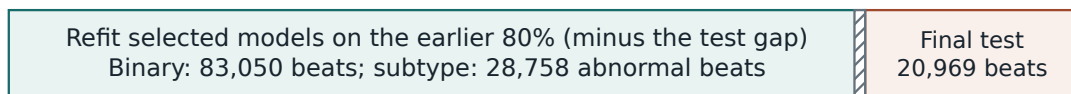
Percentages refer to accepted-beat order, not elapsed recording time.

A Select model settings



Validation chooses the model candidates and the abnormality threshold.

B Refit, freeze, then evaluate



The model and threshold remain fixed while scoring the final test segment.

Hatched markers: 16 beats excluded immediately before each marked boundary. Markers are enlarged for visibility. The 64% gap is restored during final refitting.

Interpretation: later beats from known patients; the test had been seen in earlier project development, so this is a retrospective evaluation.

Figure 6.1: Chronological selection, refitting, and testing by accepted-beat order. Selection excludes 16 beats before both boundaries; refitting retains only the exclusion before the test. The test region was observed during earlier development.

6.2 Research objective and success conditions

The final experiment asks:

After the model has access to the earlier ECG of each patient, how accurately does it classify that patient's later ECG?

The primary outcome is whole-system six-class accuracy on the final 20% temporal holdout. The planned success conditions were:

1. pooled whole-system accuracy above 90%;
2. lower bound of the equal-subject 95% confidence interval above 90%; and
3. total confidence-interval width no greater than three percentage points.

The secondary target was recall above 90% for each supported abnormal group. Macro F1 and per-class results were also examined because overall accuracy can hide poor performance on a smaller class.

6.3 Input data and feature construction

The method uses one ECG channel at every stage, but the two data sources have different roles. NSRDB supplies normal-reference windows for the frozen normal autoencoder. MITDB supplies MLII beat windows, binary labels, six-class labels, and the final temporal test. No NSRDB beat is scored in the final test.

Table 6.1: Exact evidence supplied to the final supervised models.

Evidence block	Count	Role
MLII morphology and spectrum	223 values	Describes the beat-centred 512-sample MLII waveform. This is the direct ECG evidence for both supervised stages.
RR timing context	15 values	Describes previous RR, next RR, local timing, and recent rhythm history. In live use the beat is scored only after this next-RR context is available; labels are never used during prediction.
Frozen autoencoder residuals	4 values	Summarises how the frozen NSRDB-trained normal autoencoder reconstructs the same MLII beat.
Total per MITDB beat	242 values	The binary detector and subtype classifier receive the same one-lead feature vector.

This feature construction matches the saved final model bundle: `lead_count` is one, `required_lead` is MLII, `rr_feature_count` is 15, and `ae_feature_count` is four. The method therefore does not use a second MITDB lead during training, validation, temporal testing, or deployment.

6.4 Normal-reference autoencoder training procedure

The autoencoder was trained *before* either supervised model. All 18 NSRDB records supplied first-channel ECG1 data. After filtering, four-second windowing, archived amplitude scaling, and quality checks, 15 entire subjects contributed 288,673 fitting windows and three separate subjects contributed 57,177 validation windows. The split is by subject, so adjacent windows from the same recording cannot appear on opposite sides of this validation boundary.

The real-data audit in Section 5.10.7 preserves the original cached normal windows. Their scaling is not uniformly zero mean and unit variance within each window, so it must be distinguished from the verified local standardisation used for supervised MLII beats.

One training step takes a shuffled batch of 256 clean windows, adds fresh Gaussian noise with standard deviation 0.03 to the inputs, reconstructs them, and computes mean squared error against the original clean windows. PyTorch backpropagates this error through the convolutional encoder and decoder; Adam updates their weights with learning rate 0.001. The training code repeats these steps for two complete epochs. After each epoch it disables dropout and measures clean-input MSE on the three subjects excluded from fitting. It saves the checkpoint with the lowest validation MSE. The precise objective and gradient logic are in Equations 4.14 and 4.15; Figure 4.5 contrasts this loop with tree fitting. The measured loss curve appears in Figure 5.5.

After checkpoint selection, the model is frozen. For every MITDB MLII window, its reconstruction is compared with the input to create four residual measurements. No MITDB class label updates the autoencoder. The saved normal-residual threshold is descriptive for the normal-reference model; the final classifier uses residual measurements as features and uses the separately validated binary-tree threshold for its final gate.

6.5 Temporal split

The eligible supervised records are the 46 MITDB records that contain MLII. Records 102 and 104 were excluded before fitting because their available leads are V5 and V2, not MLII. Records 201 and 202 are treated as one subject cluster for uncertainty analysis because they belong to the same person.

Every eligible record is kept in chronological order. Percentages refer to accepted-beat counts, not exact elapsed recording time. With n_r accepted beats, the cuts are $\lfloor 0.64n_r \rfloor$ and $\lfloor 0.80n_r \rfloor$:

1. the first 64% is the candidate-training region;
2. a 16-beat gap is removed before validation during candidate selection;

3. the 64–80% region is the validation region;
4. a 16-beat gap is removed before the final temporal test; and
5. the final 20% is the fixed temporal test.

For final fitting, the selected binary and subtype models are refitted on the earlier 80% development region after the 16-beat boundary gap before the test is removed. The test segment itself remains intact.

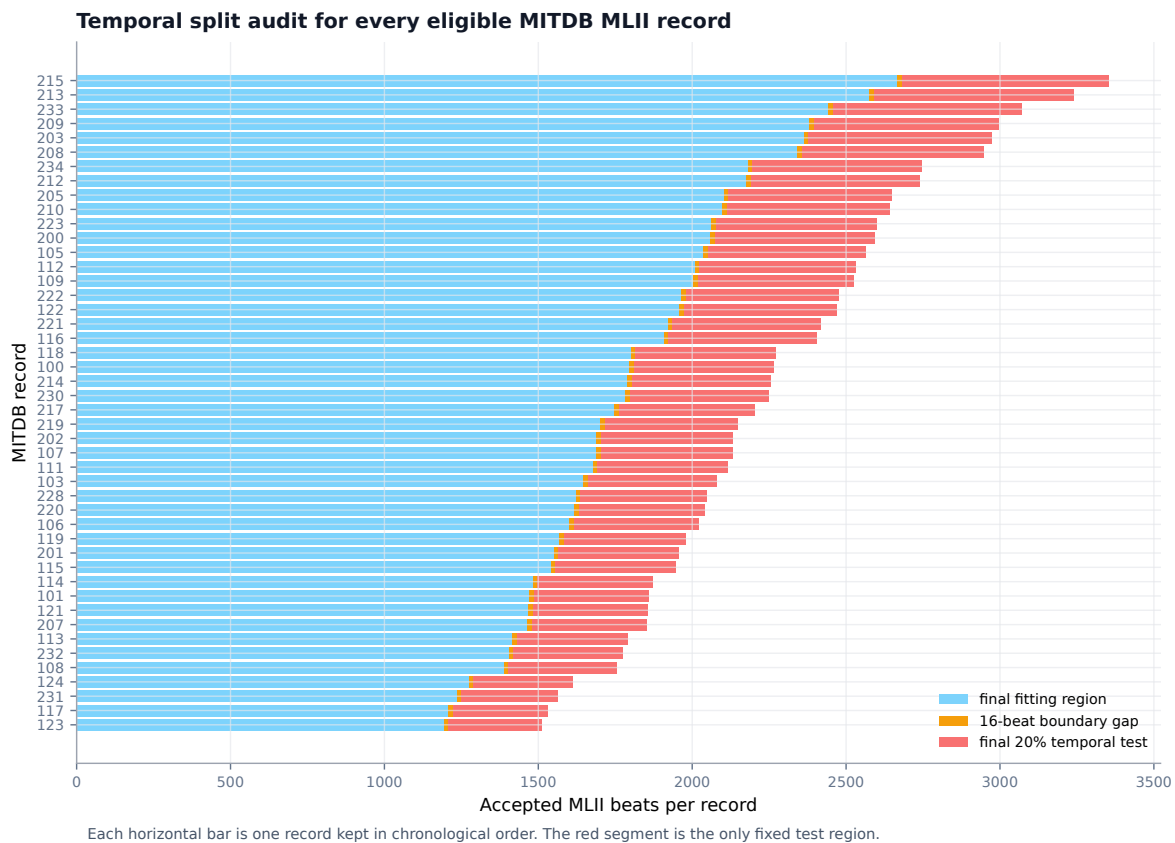


Figure 6.2: Temporal split audit for every eligible MITDB MLII record. Blue beats are available for final fitting, yellow beats are the boundary gap before the test, and red beats are the held-out final 20% temporal test.

The final held-out test contains 20,969 accepted beats. These are later beats from known patients, not beats from unseen patients. The method prevents signal overlap across the test boundary, but it intentionally keeps patient overlap.

6.6 Model selection and final fitting

Model selection uses only the earlier temporal validation region. In this rerun, the final 20% test region does not choose the candidate, threshold, or hyperparameters.

Three binary Extra Trees candidates were fitted on the earlier candidate region and ranked by validation balanced accuracy, then by F1. Three subtype Extra Trees candidates were fitted only on supported abnormal candidate beats and ranked by validation macro F1, then by accuracy. Figure 6.3 shows the validation scores that selected the final pair.

The fitting sequence for each candidate is explicit:

1. Build one 242-value row per accepted MLII beat, keeping the label and subject identifier outside the row. The autoencoder is already frozen.
2. Select candidate-training rows from the first 64% of each record, subject to the 16-beat gap before validation. For the binary stage, use every normal/abnormal target. For the subtype stage, keep only supported abnormal targets.
3. Compute class-and-record sample weights within these selected rows. At each tree node, propose random feature thresholds and choose the one giving the largest weighted entropy decrease in Equation 4.19. Continue until that candidate's depth and leaf-size rules stop growth, then average all tree outputs.
4. Score the candidate on the 64–80% validation region. For each binary candidate, choose an abnormal-score threshold using validation balanced accuracy and tie-breaks; compare candidates by balanced accuracy then F1. Compare subtype candidates by macro F1 then accuracy.
5. Fix the winning structures and threshold. Refit the binary model on all 83,050 eligible development beats and the subtype model on 28,758 supported abnormal development beats. Apply the frozen pair once to the final temporal segment.

Tree fitting is a recursive partitioning procedure, so the supervised models have no epoch-by-epoch training-loss curve. Their training criterion is node entropy; the evidence used for model selection is held-out validation performance. Node entropy and validation error therefore measure different aspects of model fitting.

Validation-region model selection

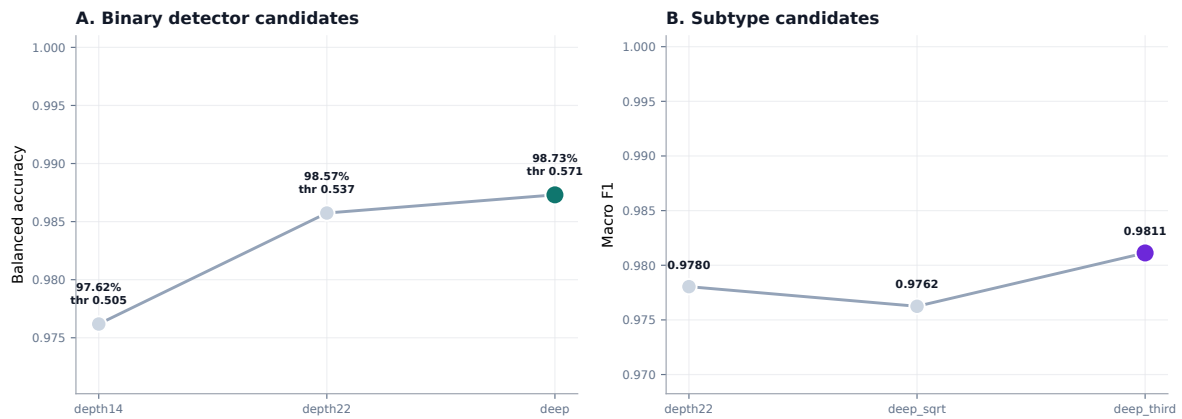


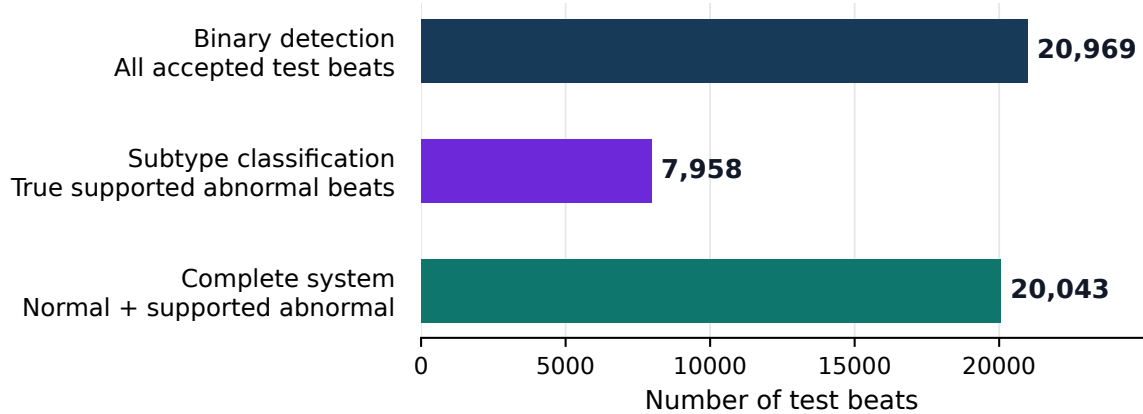
Figure 6.3: Model selection on the 64–80% validation region. The binary detector is selected by balanced accuracy and F1; the subtype classifier is selected by macro F1 and accuracy.

The selected binary detector was the fully grown 240-tree candidate with square-root feature sampling, minimum leaf size one, and record-weighting power 0.35. Its validation-selected abnormal threshold was 0.5708333. The selected subtype classifier was the fully grown 260-tree candidate with a feature-sampling fraction of 0.33, minimum leaf size one, and record-weighting power 0.25. After selection, the binary model was refitted on all development beats, whereas the subtype model was refitted only on supported abnormal development beats.

6.7 Evaluation order

The deployed decision is hierarchical. First, the binary detector estimates whether a beat is abnormal. If the abnormal probability is not above the fixed threshold, the final output is N. If the beat passes the gate, the subtype classifier chooses PVC, PAC, LBBB, RBBB, or AFib.

Three distinct evaluation populations



These groups overlap. Unsupported abnormal labels enter only binary evaluation.

Figure 6.4: Populations used for the three evaluation questions. Conditional subtype scoring uses true supported abnormal beats independently of the gate. Complete-system scoring uses the gate and subtype model together. The populations overlap; unsupported abnormal labels enter only the binary evaluation.

The temporal evaluation uses three sets of beats:

1. binary abnormality detection on all 20,969 held-out beats, including unsupported abnormal MITDB symbols;
2. conditional subtype classification on 7,958 true supported abnormal beats; and
3. whole-system N/PVC/PAC/LBBB/RBBB/AFib classification on 20,043 supported held-out beats.

The 926 unsupported abnormal beats are useful for testing the binary normal/abnormal gate, but they are not scored in the six-class whole-system confusion matrix.

6.8 Metrics

For binary detection, an abnormal beat is the positive class. TP and TN are correctly classified abnormal and N beats; FP is an N beat predicted as abnormal, and FN is an abnormal

beat predicted as N. The metrics are

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad (6.1)$$

$$\text{Sensitivity} = \frac{TP}{TP + FN}, \quad (6.2)$$

$$\text{Specificity} = \frac{TN}{TN + FP}, \quad (6.3)$$

$$\text{Balanced accuracy} = \frac{\text{Sensitivity} + \text{Specificity}}{2}. \quad (6.4)$$

For each output class c , the remaining classes are treated as negative when calculating

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \quad (6.5)$$

$$\text{Recall}_c = \frac{TP_c}{TP_c + FN_c}, \quad (6.6)$$

$$F1_c = \frac{2 \text{Precision}_c \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}. \quad (6.7)$$

Macro F1 is the unweighted mean of class F1 values. It gives PAC the same weight as N even though N contains many more beats.

Two accuracy summaries are reported. Pooled beat accuracy gives every beat equal weight. Equal-subject accuracy first calculates accuracy within each of the 45 eligible subject clusters and then gives every subject equal weight:

$$A_{\text{macro}} = \frac{1}{45} \sum_{s=1}^{45} A_s. \quad (6.8)$$

The equal-subject mean is used for uncertainty analysis so that subjects with more beats do not receive more weight. For conditional subtype accuracy, only subjects with supported abnormal test beats have a defined score; the mean uses that smaller set of subjects.

6.9 Subject-clustered confidence interval

The primary 95% confidence interval uses a non-parametric percentile bootstrap:

1. keep every beat from one subject together;
2. treat records 201 and 202 as one subject cluster;
3. sample 45 subject clusters with replacement;
4. calculate the equal-subject mean accuracy;
5. repeat 10,000 times using seed 20260801; and
6. take the 2.5th and 97.5th percentiles.

These intervals describe variation across the evaluated subject clusters with the fitted model held fixed. They do not include retraining variability, uncertainty from earlier test exposure, or a new population shift. Resampling individual beats can create an artificially narrow interval because adjacent beats from the same recording are correlated. The clustered bootstrap keeps that dependence [23].

6.10 Exploratory tests against the 90% target

The confidence interval is the primary statistical evidence. Two non-parametric tests provide supplementary, exploratory analyses. The exact one-sided sign test asks whether more than half of subjects exceed 90%. The one-sided Wilcoxon signed-rank test asks whether subject accuracies are shifted above 90%, under the additional symmetry assumption of a signed-rank location test. Accuracy is bounded and has ceiling effects, so the Wilcoxon analysis is secondary. The sign test concerns the fraction of subjects above the target, not pooled beat accuracy. These tests are interpreted together with the confidence interval, interval width, per-class recall, macro F1, and subject accuracy distribution.

6.11 Reproducibility and leakage controls

The final experiment saves the temporal boundary audit for every record, the candidate scores, selected models, probability threshold, final confusion matrix, per-class counts, per-subject accuracies, bootstrap seed, and headline metrics. The rerun used seed 20260801 and did not use the final 20% segment for candidate selection, threshold selection, or final refitting.

The final 20% segment had been observed during earlier system development, so the evaluation is a retrospective rerun on previously observed data. This prior exposure can influence design choices and cause optimism even when the present selection code excludes test rows. It is not a confirmatory first-use test, and the bootstrap does not correct development-selection bias. Patient overlap is also intentional: the test is later ECG from known patients. Therefore, the method supports a patient-adapted temporal-performance claim, not a patient-independent generalisation claim.

The next chapter reports each evaluation population separately and examines errors at both stages of the hierarchy.

7 Results and Statistical Validation

This chapter presents the retrospective temporal results, their uncertainty, and the limited external INCART check. Binary detection, conditional subtype classification, and whole-system classification are reported separately because they use different sets of beats. All results use the local window-preparation procedure described in Chapter 6. The saved experiment and prediction files are listed in the appendix.

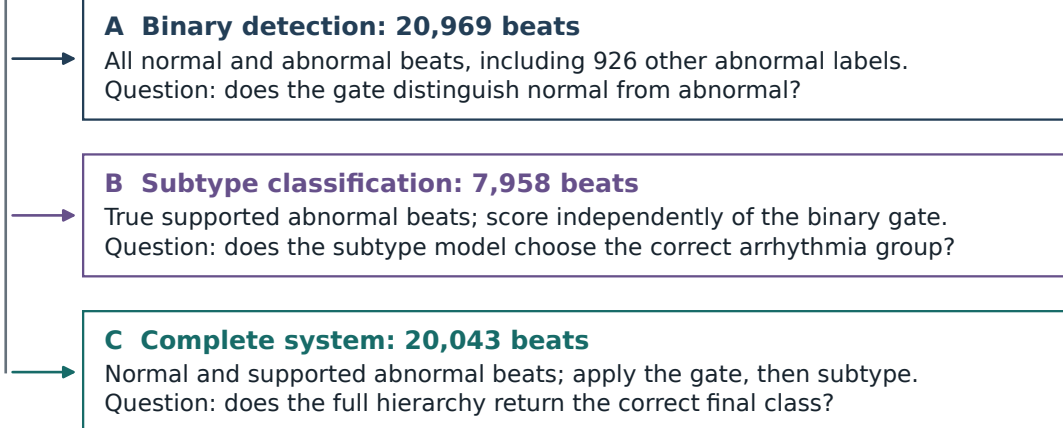
7.1 Evaluation populations and sample sizes

The final dataset contains 104,755 accepted beats from 46 MITDB MLII records, representing 45 subject clusters. Records 102 and 104 lack MLII and are excluded. Records 201 and 202 form one subject cluster. The first and last eligible beat in each record are excluded because a preceding or following accepted R peak is unavailable. Compared with the original cache, this removes 92 beats before splitting. Percentages refer to accepted-beat order.

One temporal test set; three evaluation questions

20,969 later beats from 45 patient clusters

12,085 normal + 7,958 supported abnormal + 926 other abnormal



Supported arrhythmias: PVC, PAC, LBBB, RBBB and AFib.

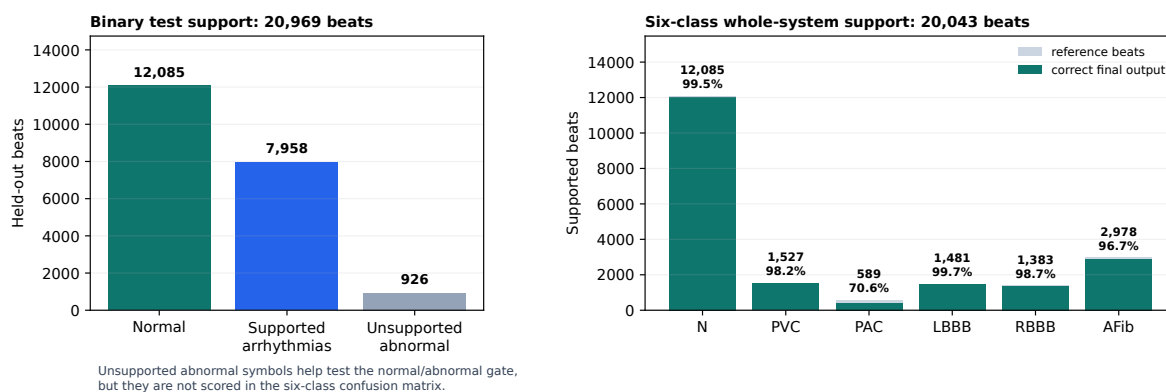
The three groups overlap; each answers a separate evaluation question.

Figure 7.1: Evaluation groups for the corrected retrospective test. Conditional subtype scoring bypasses the binary gate; complete-system scoring applies the hierarchy. Counts overlap, and records 201 and 202 share one patient cluster.

Table 7.1: Distinct denominators for the three evaluation questions.

Evaluation	Support and interpretation
Binary detection	20,969 beats: 12,085 N and 8,884 abnormal, including 926 unsupported abnormal beats.
Conditional subtype	7,958 true supported abnormal beats, scored by the subtype model regardless of the binary gate.
Whole system	20,043 beats with supported labels, evaluated through both stages of the hierarchy.

What data was used for the final temporal test

**Figure 7.2:** Reference-class composition of the corrected temporal test.

7.2 Model selection

Candidate selection uses the earlier 64–80% region after the boundary exclusions. The chosen binary model has 240 fully grown Extra Trees, square-root feature sampling, minimum leaf size one, entropy criterion, and subject/class weighting power 0.35. Its validation balanced accuracy is 98.73%. The chosen subtype model has 260 trees, feature sampling 0.33, minimum leaf size one, entropy criterion, and weighting power 0.25. Its validation macro F1 is 0.9811.

The binary threshold is 0.5708333333. The selected models are refitted on 83,050 development beats and 28,758 supported abnormal development beats, respectively. The final test does not change this threshold.

7.3 Main temporal results

Table 7.2: Corrected temporal results. Confidence intervals estimate equal-subject accuracy, not pooled beat accuracy.

Outcome	Pooled	Subject mean	95% CI	Width
Binary accuracy	98.81%	98.84%	98.01–99.45%	1.44 pp
Conditional subtype	97.63%	94.77%	89.03–99.08%	10.05 pp
Whole-system accuracy	98.07%	98.03%	96.29–99.25%	2.95 pp

In Table 7.2, CI means confidence interval and pp means percentage points. Interval widths are calculated from unrounded bounds; subtracting the displayed bounds may therefore give a slightly different value.

Binary balanced accuracy is 98.69%, sensitivity is 97.91%, specificity is 99.47%, precision is 99.27%, and F1 is 0.9858. Conditional subtype macro F1 is 0.9574; whole-system macro F1 is 0.9524.

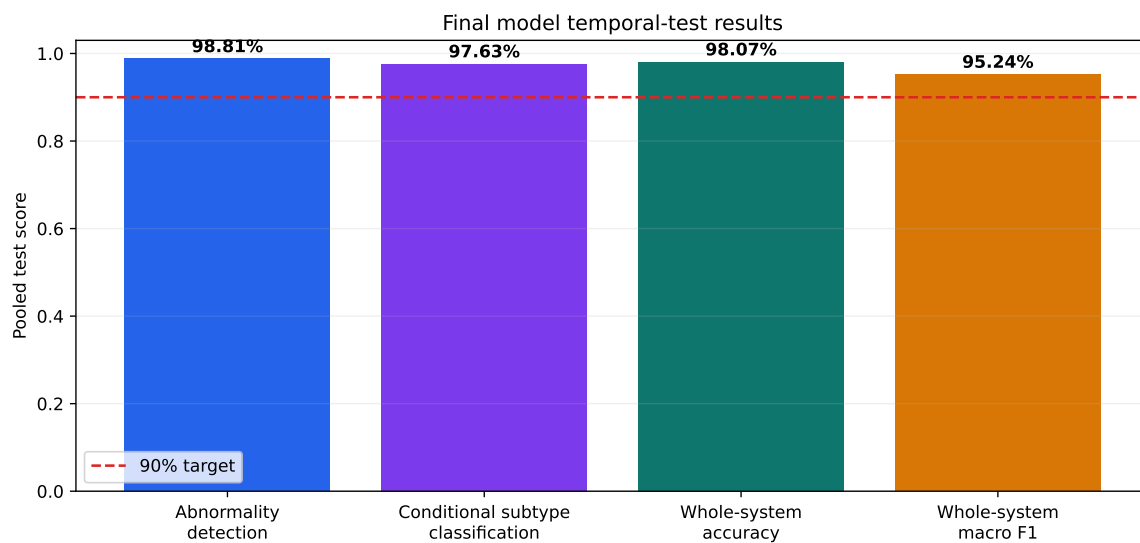


Figure 7.3: Pooled temporal scores after the preprocessing correction.

For the whole system, all three primary targets were met: pooled accuracy was above 90%, the lower confidence bound for equal-subject accuracy was above 90%, and interval width was at most three percentage points. These are descriptive outcomes of the retrospective study. Prior test exposure prevents interpreting them as an independent confirmation of these targets.

7.4 Uncertainty and subject variation

The bootstrap resamples 45 complete subject clusters with equal weight, 10,000 times using seed 20260801. It holds the fitted model fixed. For subtype accuracy, subjects without supported abnormal test beats have no subtype score and are omitted from that replicate's mean.

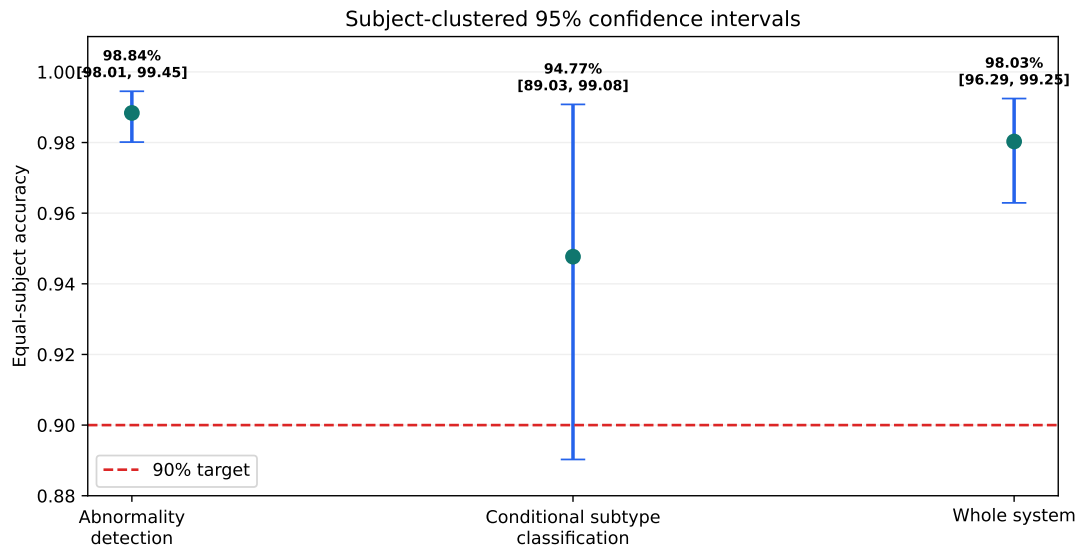


Figure 7.4: Equal-subject accuracy and percentile bootstrap intervals. The conditional subtype interval remains wide even though the whole-system width target is met.

Forty-three of 45 subject clusters exceed 90% whole-system accuracy. The exact one-sided sign test gives $p = 2.94 \times 10^{-11}$ against the null that no more than half of subjects exceed the target. The secondary Wilcoxon statistic is 987, with $p = 4.24 \times 10^{-8}$. Its location-shift interpretation requires additional symmetry assumptions. Neither test corrects selection optimism or establishes new-patient performance.

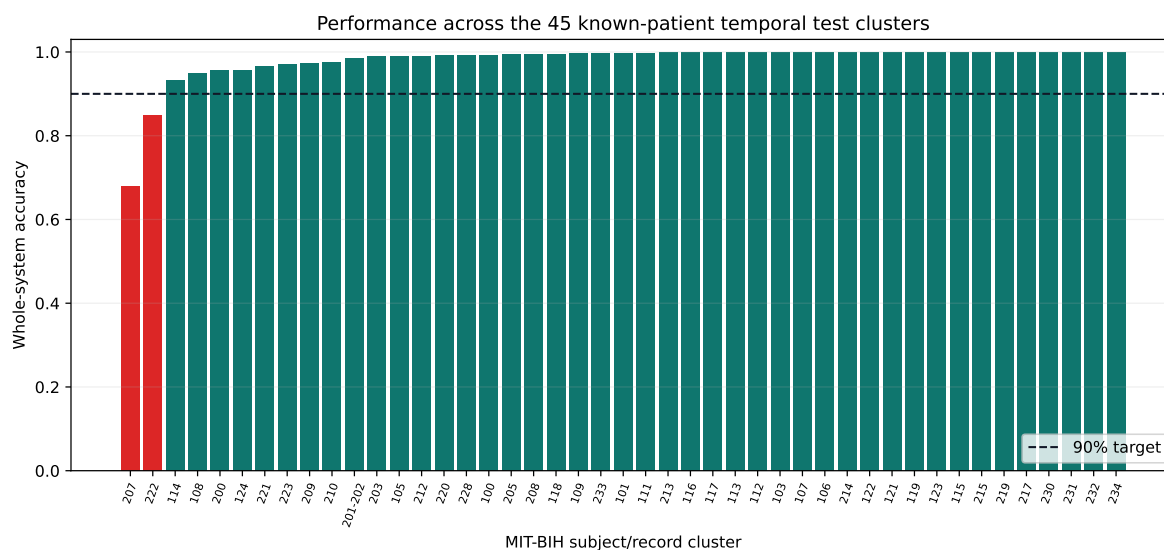


Figure 7.5: Corrected whole-system accuracy across 45 known-patient clusters.

The below-target clusters are 207 (67.92%) and 222 (84.85%). Cluster 207 has high binary accuracy but poorer subtype classification. Cluster 222 has binary accuracy 85.45%, so failures in the gate also matter. This variation is not visible from pooled accuracy alone.

7.5 Errors by stage and class

The gate detects 8,698 of 8,884 abnormal beats and correctly assigns N to 12,021 of 12,085 N beats. Thus it produces 186 missed abnormal beats and 64 false alarms. When evaluated without the gate, the subtype model correctly classifies 7,769 of 7,958 supported abnormal beats. Gate errors can assign N to an abnormal beat even when the subtype model would have classified it correctly.

Table 7.3: Whole-system performance by supported abnormal output group.

Group	True	Correct	Precision	Recall	F1
PVC	1,527	1,499	92.36%	98.17%	0.9517
PAC	589	416	98.58%	70.63%	0.8229
LBBB	1,481	1,476	96.72%	99.66%	0.9817
RBBB	1,383	1,365	99.78%	98.70%	0.9924
AFib	2,978	2,879	98.26%	96.68%	0.9746

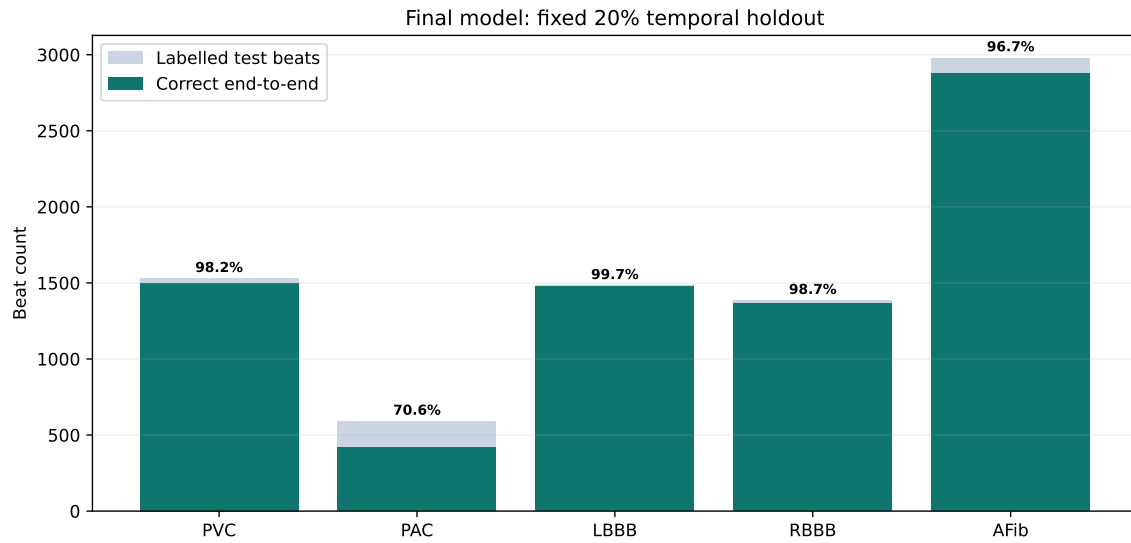


Figure 7.6: True and correctly classified supported abnormal beats. PAC remains below the 90% recall target.

Of 589 PAC-group beats, 549 pass the binary gate, 441 receive the correct subtype when the gate is ignored, and 416 are correct end to end. PAC errors therefore arise in both stages. AFib denotes the project's AFIB/AFL rhythm grouping on normal-like beat symbols; the 96.68% recall is not an episode-level or AF-only sensitivity estimate.

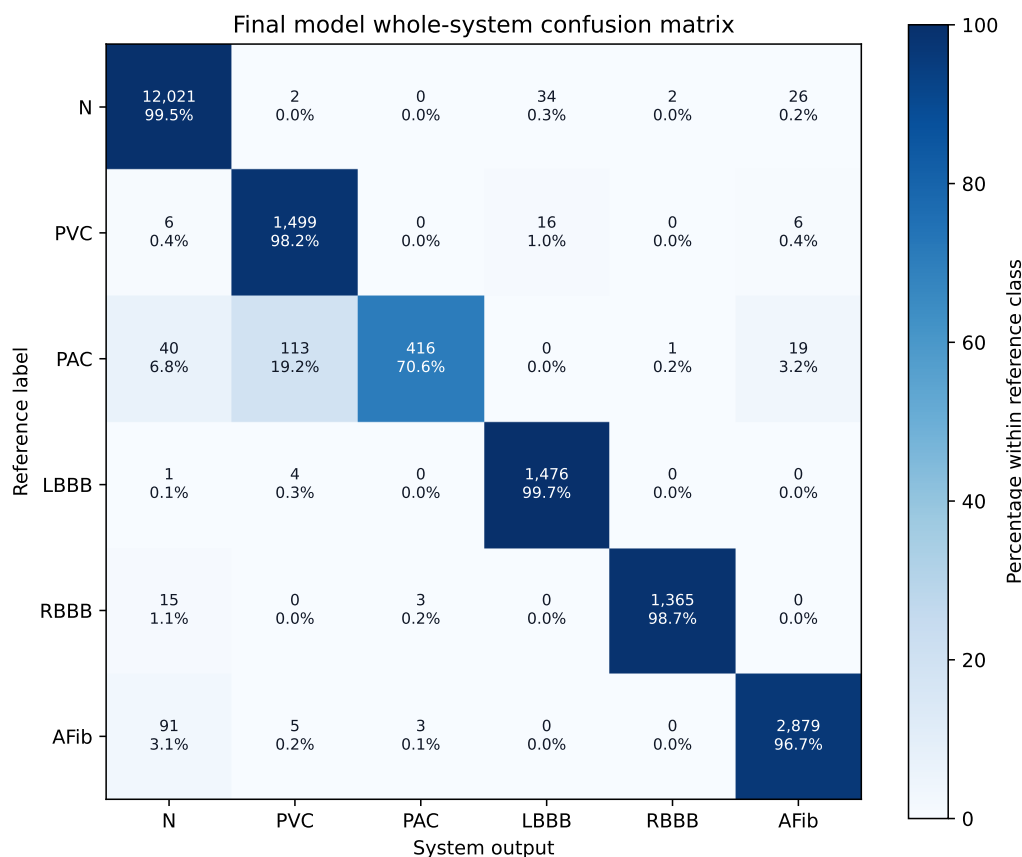


Figure 7.7: Whole-system confusion matrix for 20,043 supported temporal beats.

The largest abnormal-subtype confusion is 113 PAC beats assigned to PVC. The gate sends 91 AFib-group and 40 PAC-group beats to N. Among reference N beats, 34 become LBBB and 26 become AFib. Unsupported abnormal beats are excluded from this matrix, but a deployed gated beat still receives a supported subtype because the model has no unknown-class rejection rule.

7.6 Limited external INCART check

After model fitting, the fixed hierarchy and threshold were applied to complete INCART records I01 and I02. These were the two lead-II recordings already cached locally at the start of the external evaluation. Selection was by local availability, not measured performance. This is a convenience sample. INCART contains 75 half-hour records from 32 source patients, with 12 leads sampled at 257 Hz [35]. I01 and I02 both carry the header identifier *patient 1*: the two records represent one independent source patient.

Standard lead II is a practical external counterpart to MITDB MLII, not an identical lead. This check combines source, acquisition and lead differences. Scoring uses annotated beats, not live R-peak detections.

Table 7.4: Corrected model on complete INCART I01 and I02 recordings.

Quantity	Value
Records / source patients	2 / 1
Accepted annotated beats	5,416
Reference N / PVC	4,845 / 571
Fixed abnormal threshold	0.5708333333
Binary accuracy	94.11%
Binary balanced accuracy	96.55%
Sensitivity / specificity	99.65% / 93.46%
Binary precision / F1	64.22% / 0.7811
AUROC / average precision	0.9914 / 0.9152
TP / TN / FP / FN	569 / 4,528 / 317 / 2
Whole-system accuracy	92.85%
End-to-end PVC recall	87.74%

Only N and PVC occur in the reference labels. The binary model detects 569 of 571 PVC beats but produces 317 false alarms among 4,845 N beats. End to end, 501 PVC beats receive the correct label. Binary accuracies by record are 94.47% for I01 and 93.74% for I02. High binary sensitivity therefore coexists with lower precision and subtype errors.

Macro F1 over the six predefined labels is 0.3039 when the four classes absent from the reference labels are assigned F1 values of zero. The mean over the two observed reference classes is 0.9116. These summaries are not directly comparable to a six-class test containing reference examples of every class. PAC, LBBB, RBBB and AFib sensitivity cannot be estimated from this subset. A patient-cluster population confidence interval is not reported because there is only one source patient.

7.7 Answers to the research questions

- RQ1** The frozen autoencoder supplies four residual features, but their independent benefit has not been tested through a controlled comparison.
- RQ2** Binary, conditional-subtype and whole-system accuracies are 98.81%, 97.63% and 98.07%, respectively, on their respective evaluation sets.
- RQ3** Equal-subject whole-system accuracy is 98.03%, with a 96.29–99.25% interval and two below-target subject clusters.
- RQ4** The whole-system targets were met, but PAC recall remained below 90%. INCART evaluation covered only one patient and two reference classes; broader external validation remains incomplete.

8 Discussion and Conclusion

8.1 Main findings

The single-lead hierarchy achieved 98.07% pooled six-class accuracy on later accepted MITDB MLII beats from known patients. Equal-subject accuracy is 98.03%, with a subject-clustered 95% interval of 96.29–99.25%. The interval width was 2.95 percentage points, meeting the target of at most three percentage points. The width was calculated from unrounded bounds. However, PAC recall is 70.63%, and two subject clusters remain below 90% accuracy.

A limited check of complete INCART I01 and I02 records gives 94.11% binary accuracy and 92.85% whole-system accuracy. Both records belong to the same source patient. Binary precision falls to 64.22% because 317 normal beats become false alarms. The lower precision shows that false alarms remain a major issue in this external sample. The one-patient design limits the scope of this finding.

8.2 Interpretation of the findings

The high pooled accuracy shows that the system classified most supported beats correctly in this temporal evaluation. It does not show uniform performance across classes or patients. PAC recall was 70.63%, compared with more than 96% for each of the other supported abnormal groups. The 113 PAC-to-PVC errors show that subtype confusion was a major source of failure. Both the binary gate and subtype model therefore need improvement for this group.

Conditional subtype accuracy measures how well the second model classifies true supported abnormal beats when the gate is ignored. Whole-system accuracy also includes the gate and the larger N group. The two percentages have different denominators, so their difference is not a direct measure of the cost of the gate. The class-level counts provide a clearer view of those errors. Likewise, the wide conditional-subtype confidence interval shows substantial variation across subjects despite high pooled accuracy.

The use of waveform and RR features follows earlier heartbeat-classification research [3]. Normal-reference reconstruction adds another source of information, but this experiment does not isolate its benefit. Differences in labels, leads, and patient splits also prevent a direct accuracy ranking against the studies reviewed in Chapter 2.

The INCART check shows a further limitation. High sensitivity was accompanied by 317 false alarms and binary precision of 64.22%. Thus, detecting most abnormal beats did not ensure high precision among the alerts. Changes in recording conditions, lead configuration, and the proportion of abnormal beats may contribute to this difference, but the one-patient sample cannot separate their effects.

8.3 Achievement of the objectives

- The prototype integrates a frozen NSRDB autoencoder and two supervised MLII ensembles, with the same local model-window preparation at training and use.
- Pooled whole-system accuracy, the equal-subject lower confidence bound, and the interval-width target are met in the corrected retrospective run.
- The requirement that every supported abnormal group exceed 90% recall is not met because of PAC.
- Generalisation across independent external patients, all supported subtypes, and real AD8232 recordings is not established.

8.4 Effect of the preprocessing revision

The original supervised cache standardised complete MITDB recordings before window extraction. Its scale statistics therefore included the test region, and sensor chunks used a different scale. The corrected system filters and standardises each four-second model window and requires both neighbouring R peaks to be observed. Tests verify agreement between offline and streaming feature construction when the same peak locations are supplied.

The original 96.86% result is retained as a historical artifact, not the headline result of the corrected system. The preprocessing changes, removal of two edge beats per record, and refitting mean that the difference between the two runs is not a controlled estimate of one component's causal benefit.

The autoencoder remains the original frozen checkpoint. Its decoder resizes up-block tensors to their skip lengths and finally interpolates from 256 to 512 samples. Chapter 5 reports these implemented dimensions. Altering this trained architecture would require a separate training study.

8.5 Limitations

1. Earlier labelled ECG from the same patients is available during supervised development. The result does not establish accuracy for patients whose data were absent from model development.
2. The final segment was observed during earlier development. Correcting the preprocessing does not restore test independence. Earlier development choices may still make the results optimistic. Neither the bootstrap nor the reported p -values corrects this bias.
3. The bootstrap estimates subject variation with the fitted model fixed. It does not include retraining variability or extrapolation to a new cohort.
4. NSRDB is a nominally normal source cohort, with no beat-annotation screening in this preparation. Its ECG1 anatomical lead is unspecified. The two-epoch training record

does not establish optimal convergence.

5. No controlled ablation quantifies the independent benefit of the four autoencoder residuals.
6. Output labels are project groups. AFib includes flutter on normal-like beat symbols; PVC includes ventricular escape beats; N includes atrial and junctional escape beats. No AF episode detection is evaluated.
7. Unsupported abnormal beats enter binary testing but are excluded from six-class scoring. The deployed gate has no validated unknown-class rejection.
8. Temporal and INCART scoring use annotated R peaks. Live peak detection introduces errors beyond those metrics. The prepared window needs roughly 2.44 seconds of post-R signal and the next observed beat, plus acquisition and computation delay.
9. INCART evaluation covers 5,416 beats but only one source patient and N/PVC reference support. More beats from this same patient cannot supply independent-patient evidence or validate absent classes.
10. The software requires the correct lead declaration, but cannot verify physical electrode placement. Signal checks reject non-finite/flat inputs and reported lead-off; they do not certify clinical signal quality.

8.6 Further research

The next experiment should preregister a patient-disjoint development, validation and test allocation; freeze preprocessing and thresholds before external scoring; and recruit enough independent subjects for every target class. PAC and external false alarms need particular attention. A controlled comparison should evaluate the hierarchy with and without residual features, and prospective sensor testing should measure R-peak accuracy, classification errors, signal quality and end-to-end latency. Future work should treat the already viewed temporal and INCART records as development data and reserve separate patients for final evaluation.

8.7 Conclusion

This study developed a reproducible single-lead ECG prototype and evaluated its binary, subtype, and complete classification stages. The retrospective results met the aggregate targets, but PAC recall and external false alarms remain important weaknesses. It remains a research system, not a validated clinical diagnostic device [24].

Bibliography

- [1] P. Kligfield, L. S. Gettes, J. J. Bailey, R. Childers, B. J. Deal, E. W. Hancock, G. van Herpen, J. A. Kors, P. Macfarlane, D. M. Mirvis, O. Pahlm, P. Rautaharju, and G. S. Wagner, “Recommendations for the standardization and interpretation of the electrocardiogram: Part i,” *Journal of the American College of Cardiology*, vol. 49, no. 10, pp. 1109–1127, 2007.
→ [p1], [p24]
- [2] G. B. Moody and R. G. Mark, “The impact of the MIT-BIH arrhythmia database,” *IEEE Engineering in Medicine and Biology Magazine*, vol. 20, no. 3, pp. 45–50, 2001.
→ [p1], [p6], [p11]
- [3] P. de Chazal, M. O’Dwyer, and R. B. Reilly, “Automatic classification of heartbeats using ECG morphology and heartbeat interval features,” *IEEE Transactions on Biomedical Engineering*, vol. 51, no. 7, pp. 1196–1206, 2004.
→ [p1], [p6], [p7], [p9], [p22], [p91]
- [4] A. L. Goldberger, L. A. N. Amaral, L. Glass, J. M. Hausdorff, P. C. Ivanov, R. G. Mark, J. E. Mietus, G. B. Moody, C.-K. Peng, and H. E. Stanley, “Physiobank, physiotoolkit, and PhysioNet: Components of a new research resource for complex physiologic signals,” *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000.
→ [p6], [p11]
- [5] G. B. Moody and R. G. Mark, “MIT-BIH arrhythmia database.” <https://physionet.org/content/mitdb/1.0.0/>, 2005.
→ [p6], [p11], [p13], [p22], [p60]
- [6] G. B. Moody, “MIT-BIH normal sinus rhythm database.” <https://physionet.org/content/nsrdb/1.0.0/>, 1999.
→ [p6], [p9], [p11], [p22], [p43], [p60]
- [7] S. Mousavi and F. Afghah, “Inter- and intra-patient ECG heartbeat classification for arrhythmia detection: A sequence to sequence deep learning approach,” in *IEEE International Conference on Acoustics, Speech and Signal Processing*, pp. 1308–1312, 2019.
→ [p6], [p7], [p9], [p22]
- [8] J. Zhang, A. Liu, D. Liang, X. Chen, and M. Gao, “Interpatient ECG heartbeat classification with an adversarial convolutional neural network,” *Journal of Healthcare Engineering*, vol. 2021, p. 9946596, 2021.
→ [p6], [p7], [p9], [p30]
- [9] H. Huang, J. Liu, Q. Zhu, R. Wang, and G. Hu, “Detection of inter-patient left and right bundle branch block heartbeats in ECG using ensemble classifiers,” *BioMedical Engineering OnLine*, vol. 13, p. 72, 2014.
→ [p7], [p10]

- [10] S. Kiranyaz, T. Ince, and M. Gabbouj, “Real-time patient-specific ECG classification by 1-d convolutional neural networks,” *IEEE Transactions on Biomedical Engineering*, vol. 63, no. 3, pp. 664–675, 2016. → [p7]
- [11] U. R. Acharya, H. Fujita, O. S. Lih, Y. Hagiwara, J. H. Tan, and M. Adam, “Automated detection of arrhythmias using different intervals of tachycardia ECG segments with convolutional neural network,” *Information Sciences*, vol. 405, pp. 81–90, 2017. → [p7]
- [12] A. Y. Hannun, P. Rajpurkar, M. Haghpanahi, G. H. Tison, C. Bourn, M. P. Turakhia, and A. Y. Ng, “Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network,” *Nature Medicine*, vol. 25, pp. 65–69, 2019. → [p7]
- [13] A. H. Ribeiro, M. H. Ribeiro, G. M. M. Paixao, D. M. Oliveira, P. R. Gomes, J. A. Canazart, M. P. S. Ferreira, C. R. Andersson, P. W. Macfarlane, W. Meira, T. B. Schön, and A. L. P. Ribeiro, “Automatic diagnosis of the 12-lead ECG using a deep neural network,” *Nature Communications*, vol. 11, p. 1760, 2020. → [p7]
- [14] G. E. Hinton and R. R. Salakhutdinov, “Reducing the dimensionality of data with neural networks,” *Science*, vol. 313, no. 5786, pp. 504–507, 2006. → [p7], [p9], [p26]
- [15] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, “Extracting and composing robust features with denoising autoencoders,” in *Proceedings of the 25th International Conference on Machine Learning*, pp. 1096–1103, 2008. → [p8], [p9], [p26], [p27], [p40]
- [16] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009. → [p8], [p26]
- [17] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, “LSTM-based encoder–decoder for multi-sensor anomaly detection,” in *ICML 2016 Workshop on Anomaly Detection*, 2016. → [p8]
- [18] M. Thill, W. Konen, H. Wang, and T. Bäck, “Temporal convolutional autoencoder for unsupervised anomaly detection in time series,” *Applied Soft Computing*, vol. 112, p. 107751, 2021. → [p8], [p9]
- [19] J.-H. Jang, T. Y. Kim, H.-S. Lim, and D. Yoon, “Unsupervised feature learning for electrocardiogram data using the convolutional variational autoencoder,” *PLOS ONE*, vol. 16, no. 12, p. e0260612, 2021. → [p8]
- [20] L. Shan, Y. Li, H. Jiang, P. Zhou, J. Niu, R. Liu, Y. Wei, J. Peng, H. Yu, X. Sha, and S. Chang, “Abnormal ECG detection based on an adversarial autoencoder,” *Frontiers in Physiology*, vol. 13, p. 961724, 2022. → [p8]
- [21] T. Saito and M. Rehmsmeier, “The precision–recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets,” *PLOS ONE*, vol. 10, no. 3, p. e0118432, 2015. → [p8], [p10], [p34]

- [22] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, "Class-balanced loss based on effective number of samples," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 9268–9277, 2019. → [p8], [p32]
- [23] C. M. Rutter, "Bootstrap estimation of diagnostic accuracy with patient-clustered data," *Academic Radiology*, vol. 7, no. 6, pp. 413–419, 2000. → [p8], [p10], [p34], [p82]
- [24] G. S. Collins, K. G. M. Moons, P. Dhiman, R. D. Riley, A. L. Beam, B. Van Calster, M. Ghassemi, X. Liu, J. B. Reitsma, M. van Smeden, A.-L. Boulesteix, J. C. Camaradou, L. A. Celi, S. Denaxas, A. K. Denniston, B. Glocker, R. M. Golub, H. Harvey, G. Heinze, M. M. Hoffman, A. P. Kengne, E. Lam, N. Lee, E. W. Loder, L. Maier-Hein, B. A. Mateen, M. D. McCradden, L. Oakden-Rayner, J. Ordish, R. Parnell, S. Rose, K. Singh, L. Wynants, and P. Logullo, "TRIPOD+AI statement: Updated guidance for reporting clinical prediction models that use regression or machine learning methods," *BMJ*, vol. 385, p. e078378, 2024. → [p8], [p10], [p93]
- [25] PhysioNet, "Frequently asked questions: What do the signal names MLII, v2, ... mean?," <https://archive.physionet.org/faq.shtml>. Accessed September 23, 2026. → [p13]
- [26] S. Butterworth, "On the theory of filter amplifiers," *Experimental Wireless and the Wireless Engineer*, vol. 7, pp. 536–541, 1930. → [p15], [p23], [p24]
- [27] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in python," *Nature Methods*, vol. 17, pp. 261–272, 2020. → [p15], [p23], [p33]
- [28] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778, 2016. → [p27], [p42]
- [29] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention*, pp. 234–241, 2015. → [p27]
- [30] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014. → [p27], [p42]
- [31] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations*, 2015. → [p28], [p43]
- [32] P. Geurts, D. Ernst, and L. Wehenkel, "Extremely randomized trees," *Machine Learning*, vol. 63, pp. 3–42, 2006. → [p30], [p46]
- [33] Analog Devices, *AD8232: Single-Lead, Heart Rate Monitor Front End Data Sheet*. Analog Devices, Inc., 2020. → [p32]

-
- [34] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019. → [p33]
- [35] E. Yakushenko, “St petersburg INCART 12-lead arrhythmia database.” <https://physionet.org/content/incartdb/1.0.0/>, 2008. → [p89]

A Reproducibility Record

A.1 Experiment files

Paths in this appendix are relative to `Final_Report_Research/` unless they begin with `backend/` or `Final_Report_Research/`; those paths are relative to the repository root. The main temporal experiment is stored in:

- `experiments/corrected_temporal_holdout.json`;
- `experiments/corrected_temporal_holdout.joblib`; and
- `tools/final_temporal_holdout.py`.

The normal model is documented by `experiments/nsrdb_autoencoder_training.json`, produced by `tools/train_nsrdbencoder.py`. Its deployable checkpoint is `backend/app/storage/models/model_nsrdbencoder_primary_healthy.pt`. The deployable copy is registered by the backend model registry and stored in the backend model directory.

The real-data training and inference case study is reproducible from `tools/generate_real_methodology_case.py`. Its numerical audit is `experiments/methodology_real_case.json`, including source sample indices, all 242 feature values for each selected beat, actual tree routes, complete-forest refit checks, and unchanged model-file hashes. The new one-step autoencoder demonstration is identified separately from the original two-epoch training logs. Archived NSRDB amplitudes are preserved.

A.2 Frozen constants

A.3 Recorded evaluation targets

The saved JSON file records that all three whole-system targets were met: pooled accuracy above 90%, a lower 95% confidence bound above 90%, and an interval width of at most three percentage points. These checks describe the retrospective evaluation; they do not establish independent validation.

A.4 Safety statement

The installed artifact is a research model. The final experiment uses patient overlap, one MITDB lead, and annotated R-peak locations during evaluation. It is not a certified medical device and must not be used to diagnose or treat a patient.

Table A.1: Settings used for the final experiment.

Setting	Value
Experiment seed	20260801
Healthy-autoencoder seed	20260802
Autoencoder fitting/validation subjects	15 / 3 NSRDB records
Autoencoder fitting/validation windows	288,673 / 57,177
Bootstrap seed and replicates	20260801; 10,000
Development/test boundary	80% / 20% per record
Candidate training/validation	approximately 64% / 16% per record
Boundary gap	16 beats before validation and test
Binary threshold	0.5708333333
Binary ensemble	240 fully grown Extra Trees
Subtype ensemble	260 fully grown Extra Trees
Binary/subtype input sizes	242 / 242
Independent uncertainty units	45 subject clusters

A.5 Verification and historical record

The following files retain the results and verification record:

- Original August results:
`experiments/final_temporal_holdout.json`.
- Revised temporal predictions:
`experiments/corrected_test_predictions.npz`.
- Independent verification:
`experiments/final_verification.json`.
- External INCART results:
`experiments/corrected_external_incant.json`.
- External INCART predictions:
`experiments/corrected_external_predictions.npz`.

Model SHA-256 digests link the evaluated bundle to its deployed copy.

Use the backend Python environment to run the verification script at `tools/verify_final_results.py` within the report directory. The supervised training entry point is `tools/final_temporal_holdout.py`. It uses `tools/corrected_dataset.py` and the shared preparation module at `backend/app/ml/beat_preparation.py`. The NSRDB checkpoint is frozen and is not retrained by this entry point. Running training again replaces the corrected artifacts, so preserve the current experiment directory before any future research change.